

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

etsinf

Escola Tècnica Superior d'Enginyeria Informàtica

Escuela Técnica Superior de Ingeniería Informática
Universidad Politécnica de Valencia

De señal acústica a partitura: transcripción automática de piano mediante aprendizaje profundo

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

Autor: Jorge Rodríguez González

Tutor: Alfons Juan Císcar

Curso 2025-2026

Resum

La transcripció automàtica de música (AMT) constitueix un dels problemes centrals de la recuperació d'informació musical (MIR), l'objectiu de la qual és convertir un senyal d'àudio en una representació simbòlica de la música interpretada. En el cas del piano, l'objectiu és obtindre a partir de l'àudio una representació MIDI que capture amb precisió els esdeveniments de nota –to, inici, final i dinàmica– presents en la interpretació. Aquest treball aborda la replicació i l'avaluació de dos sistemes de l'estat de l'art per a la transcripció automàtica. En primer lloc, s'entrena un model MFT-FiLM (*Multitask Frequency-Time Feature-wise Linear Modulation*) que combina convolucions espectrals i xarxes recurrents bidireccionals per a l'estimació conjunta d'*onsets*, *offsets*, *frames* i velocitats. A continuació, s'entrena un model Transkun, un sistema basat en *transformers* que aborda la transcripció com un problema de traducció de seqüències. Tots dos models s'avaluen de forma quantitativa sobre el conjunt de test de MAESTRO seguint el protocol estàndard (`mir_eval`). Addicionalment, s'avalua la capacitat de generalització de tots dos models sobre el *dataset* PiJAMA, la distribució de repertori i àudio del qual difereix de MAESTRO, i s'exploren modificacions experimentals orientades a la millora del rendiment dels models.

Paraules clau: Transcripció automàtica, xarxes convolucionals, *transformers*, *onset*, *offset*, *frame*, modulació lineal, MIDI, espectrograma, MAESTRO, PiJAMA, piano

Resumen

La transcripción automática de música (AMT) constituye uno de los problemas centrales de la recuperación de información musical (MIR), cuyo objetivo es convertir una señal de audio en una representación simbólica de la música interpretada. En el caso del piano, el objetivo es obtener a partir del audio una representación MIDI que capture con precisión los eventos de nota –tono, inicio, final y dinámica– presentes en la interpretación. Este trabajo aborda la replicación y evaluación de dos sistemas del estado del arte para la transcripción automática. En primer lugar, se entrena un modelo MFT-FiLM (*Multitask Frequency-Time Feature-wise Linear Modulation*) que combina convoluciones espectrales y redes recurrentes bidireccionales para la estimación conjunta de *onsets*, *offsets*, *frames* y velocidades. A continuación, se entrena un modelo Transkun, un sistema basado en transformers que aborda la transcripción como un problema de traducción de secuencias. Ambos modelos se evalúan de forma cuantitativa sobre el conjunto de test de MAESTRO siguiendo el protocolo estándar (`mir_eval`). Adicionalmente se evalúa la capacidad de generalización de ambos modelos sobre el *dataset* PiJAMA, cuya distribución de repertorio y audio difiere de MAESTRO, y se exploran modificaciones experimentales orientadas a la mejora del rendimiento de los modelos.

Palabras clave: Transcripción automática, redes convolucionales, *transformers*, *onset*, *offset*, *frame*, modulación lineal, MIDI, espectrograma, MAESTRO, PiJAMA, piano

Abstract

Automatic Music Transcription (AMT) constitutes one of the core problems in Music Information Retrieval (MIR), aiming to convert an audio signal into a symbolic representation of the performed music. In the case of the piano, the goal is to obtain a MIDI representation from the audio that accurately captures the note events –pitch, onset, offset, and dynamics– present in the performance. This work addresses the replication and evaluation of two state-of-the-art systems for automatic transcription. First, an MFT-FiLM

(Multitask Frequency-Time Feature-wise Linear Modulation) model is trained, which combines spectral convolutions and bidirectional recurrent networks for the joint estimation of onsets, offsets, frames, and velocities. Next, a Transkun model is trained, a *transformer*-based system that approaches transcription as a sequence translation problem. Both models are quantitatively evaluated on the MAESTRO test set following the standard protocol (`mir_eval`). Additionally, the generalization capability of both models is evaluated on the PiJAMA *dataset*, whose repertoire and audio distribution differs from MAESTRO, and experimental modifications aimed at improving the models' performance are explored.

Key words: Automatic transcription, convolutional networks, *transformers*, *onset*, *offset*, *frame*, linear modulation, MIDI, spectrogram, MAESTRO, PiJAMA, piano

Índice general

Índice general v
Índice de figuras vii
Índice de tablas vii

Lista de Acrónimos ix
1 Introducción 1
1.1 Motivación . . . 1
1.2 Objetivos . . . 1
1.3 Estructura de la memoria . . . 1
2 Preliminares 3
2.1 Aprendizaje automático . . . 3
2.2 Aprendizaje profundo . . . 4
2.3 Transformers . . . 5
2.4 Transcripción automática de música . . . 7
2.5 Representación digital del audio . . . 7
2.6 Representaciones simbólicas de la música . . . 8
2.7 MIDI2ScoreTransformer . . . 10
3 MAESTRO 13
3.1 Introducción . . . 13
3.2 Origen de los datos . . . 13
3.3 Adquisición y anotación . . . 15
3.4 Características del *dataset* . . . 15
3.5 Evolución del *dataset* . . . 17
3.6 Distribución y licencia . . . 17
3.7 Limitaciones . . . 17
3.8 Relevancia para este trabajo . . . 18
3.9 Evaluación estándar . . . 18
4 MFT-FiLM 21
4.1 Introducción . . . 21
4.2 Transformada Q Constante . . . 21
4.3 Arquitectura general . . . 24
4.4 Módulos FiLM, FT-FiLM e IFT-FiLM . . . 25
4.5 Función de pérdida y eficiencia . . . 28
4.6 Experimentos . . . 29
4.7 Conclusiones . . . 35
5 Transkun 37
5.1 Introducción . . . 37
5.2 Arquitectura general . . . 37
5.3 Espectrograma de entrada, submuestreo y *event tracks* . . . 39
5.4 Transformer encoder no jerárquico . . . 40
5.5 Scoring por producto interior y semi-CRF . . . 42
5.6 Predicción de atributos y procesado por segmentos . . . 44

5.7 Experimentos 46
5.8 Conclusiones 53

6 Evaluación fuera del clasicismo: *jazz* **55**
6.1 Introducción 55
6.2 Análisis del comportamiento sobre MAESTRO 55
6.3 PiJAMA 59
6.4 Experimentos 62
6.5 Conclusiones 67

7 Conclusiones **69**

Bibliografía **71**

Apéndices

A Configuración del experimento MFT-FiLM **75**
A.1 Entorno de ejecución y dependencias 75
A.2 Rendimiento del entrenamiento en el clúster 76
A.3 Parámetros de configuración del experimento 76
A.4 Herramientas de preprocesamiento de audio y MIDI 77

B Configuración del experimento Transkun **79**
B.1 Entorno de ejecución y dependencias 79
B.2 Preparación del conjunto MAESTRO 79
B.3 Configuración del modelo y del entrenamiento 80

Índice de figuras

2.1 Arquitectura de un perceptrón multicapa con dos capas ocultas. 4
2.2 Convolución 1D con kernel de tamaño $K = 3$ y stride $s = 1$. 5
2.3 Arquitectura *Transformer encoder–decoder* original [27]. 6
2.4 Fragmento de *performance* MIDI. 9
2.5 Fragmento de *piano roll*. 9
2.6 Fragmento en notación de partitura estándar (MusicXML). 10

3.1 Sistema de sensores de un Yamaha Disklavier 14

4.1 Comparación de representaciones tiempo–frecuencia 23
4.2 Esquema general del modelo MFT-FiLM 24
4.3 Arquitectura detallada de los módulos FT-FiLM e IFT-FiLM 26
4.4 Convergencia de la pérdida durante la primera época 31
4.5 Evolución de la pérdida de entrenamiento y validación por época 32
4.6 Evolución de la métrica F_1 en el conjunto de validación 33
4.7 Precisión y *recall* en el conjunto de validación 34

5.1 Esquema general del modelo Transkun 38
5.2 Bloques constitutivos del *transformer encoder* de Transkun 41
5.3 Pérdida durante la primera época de entrenamiento de Transkun 47
5.4 Evolución de la pérdida de entrenamiento y validación de Transkun . . . 48
5.5 MSE de velocidad sobre entrenamiento de Transkun 49
5.6 MSE de *offset* sobre entrenamiento de Transkun 49
5.7 Métricas *Note w/ Offset* de validación de Transkun 50

6.1 Intervalos LSD por compositor ordenados cronológicamente 57
6.2 Intervalos LSD ordenados cronológicamente para los cinco compositores principales en MAESTRO v3.0.0 58
6.3 Comparación espectral entre repertorio clásico/romántico y *jazz* 61
6.4 Salidas de MFT-FiLM y Transkun sobre fragmento de *jazz* 65
6.5 Salidas de MFT-FiLM y Transkun procesadas por MIDI2ScoreTransformer 66

Índice de tablas

3.1 Top 20 compositores en MAESTRO v3.0.0 16
3.2 Estadísticas de MAESTRO v3.0.0 por partición 16
3.3 Evolución de MAESTRO entre las versiones V1, V2 y V3 17

3.4 Condiciones de aceptación de un evento predicho como verdadero positivo según la variante de métrica de nota empleada. 20

4.1 Resultados del F_1 de MFT-FiLM sobre MAESTRO v3.0.0 30
4.2 Resultados de la precisión de MFT-FiLM sobre MAESTRO v3.0.0 30
4.3 Resultados del *recall* de MFT-FiLM sobre MAESTRO v3.0.0 30

5.1 Resultados del F_1 de Transkun sobre MAESTRO v3.0.0 51
5.2 Resultados de la precisión de Transkun sobre MAESTRO v3.0.0 52
5.3 Resultados del *recall* de Transkun sobre MAESTRO v3.0.0 52

6.1 Cobertura de audio en las particiones de PiJAMA 62
6.2 Efecto de la tolerancia temporal en PiJAMA 63
6.3 Resultados comparativos de MFT-FiLM y Transkun en MAESTRO y PiJAMA 64

A.1 Parámetros de configuración establecidos en `constant_macro.py` para el entrenamiento de MFT-FiLM. 77

B.1 Parámetros principales de configuración empleados en el experimento Transkun. 80

Lista de Acrónimos

AMT *Automatic Music Transcription* (Transcripción Automática de Música)

ASAP *Aligned Scores and Performances*

CC *Control Change* (mensaje de control MIDI)

CQT *Constant-Q Transform* (Transformada Q Constante)

CSV *Comma-Separated Values*

MEI *Music Encoding Initiative*

MusicXML *Music Extensible Markup Language*

FiLM *Feature-wise Linear Modulation* (Modulación Lineal por Características)

FG-LSTM *Frequency-Grouped Long Short-Term Memory*

JSON *JavaScript Object Notation*

LSTM *Long Short-Term Memory*

MAESTRO *MIDI and Audio Edited for Synchronous Tracks and Organization*

MAPS *MIDI Aligned Piano Sounds*

MIDI *Musical Instrument Digital Interface*

MIR *Music Information Retrieval* (Recuperación de Información Musical)

MTL *Multitask Learning* (Aprendizaje Multitarea)

PCM *Pulse Code Modulation* (Modulación por Impulsos Codificados)

PiJAMA *Piano jazz with Automatic MIDI Annotations*

SLURM *Simple Linux Utility for Resource Management*

STFT *Short-Time Fourier Transform* (Transformada de Fourier de Tiempo Corto)

VRAM *Video Random Access Memory*

WAV *Waveform Audio File Format*

BCE *Binary Cross-Entropy* (Entropía Cruzada Binaria)

CCE *Categorical Cross-Entropy* (Entropía Cruzada Categórica)

ReLU *Rectified Linear Unit*

GELU *Gaussian Error Linear Unit*

ECC *Error-Correcting Code*

CRF *Conditional Random Field* (Campo Aleatorio Condicional)

ViT *Vision Transformer*

YOLOS *You Only Look at One Sequence*

RMSNorm *Root Mean Square Layer Normalization*

FFN *Feed-Forward Network* (Red Neuronal de Avance)

MAP *Maximum A Posteriori* (Máximo a Posteriori)

MLE *Maximum Likelihood Estimation* (Estimación de Máxima Verosimilitud)

MLP *Multilayer Perceptron* (Perceptrón Multicapa)

MSE *Mean Squared Error* (Error Cuadrático Medio)

NLL *Negative Log-Likelihood* (Log-Verosimilitud Negativa)

LSD *Least Significant Difference* (Diferencia Mínima Significativa)

CAPÍTULO 1
Introducción

1.1 Motivación

1.2 Objetivos

1.3 Estructura de la memoria

CAPÍTULO 2
Preliminares

2.1 Aprendizaje automático

El aprendizaje automático es la rama de la inteligencia artificial que estudia el diseño de algoritmos capaces de mejorar su rendimiento en una clase de tareas a partir de la experiencia. La definición formal más extendida, recogida por Murphy [36], es la de Mitchell: *"se dice que un programa aprende de la experiencia E con respecto a una clase de tareas T y una métrica de rendimiento R, si su rendimiento en las tareas de T, medido por R, mejora con la experiencia E"*. Esta definición unifica bajo un mismo marco algoritmos de naturaleza muy diversa, desde la regresión lineal hasta las arquitecturas de aprendizaje profundo empleadas en este trabajo.

La característica que distingue al aprendizaje automático de la programación clásica es que el comportamiento del sistema no se especifica mediante reglas escritas explícitamente por un programador, sino que se *induce* a partir de los datos. El modelo aprende a representar la estructura del problema ajustando un conjunto de parámetros internos de forma que sus predicciones se aproximen a los valores esperados sobre el conjunto de entrenamiento, con el objetivo de que dicha capacidad se generalice a datos no vistos [36].

En función de la naturaleza de los datos disponibles y del objetivo de aprendizaje, se distinguen tres paradigmas fundamentales [36, 9]:

- **Aprendizaje supervisado** (*supervised learning*). El conjunto de entrenamiento está formado por pares entrada–salida $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^{N}$, donde cada entrada $\mathbf{x}_n$ tiene asociada una etiqueta o valor objetivo y_n proporcionado por un supervisor externo. El objetivo es aprender una función $f : \mathcal{X} \to \mathcal{Y}$ que generalice correctamente a entradas no vistas.

- **Aprendizaje no supervisado** (*unsupervised learning*). Solo se dispone de las entradas $\mathcal{D} = \{\mathbf{x}_n\}_{n=1}^{N}$, sin etiquetas. El objetivo es descubrir la estructura latente de los datos: agrupaciones, distribuciones subyacentes o representaciones comprimidas. Técnicas como los modelos de mezcla de gaussianas, los *autoencoders* o los modelos generativos profundos pertenecen a este paradigma [36]. Un caso particular de especial relevancia en la actualidad es el *aprendizaje autosupervisado* (*self-supervised learning*), en el que las etiquetas de entrenamiento se generan automáticamente a partir de los propios datos mediante tareas pretexto (como predecir una parte enmascarada de la entrada), sin necesidad de anotación humana. Este enfoque es la base del preentrenamiento de modelos como wav2vec o BERT [36].

- **Aprendizaje por refuerzo** (*reinforcement learning*). Un agente aprende a tomar decisiones secuenciales interactuando con un entorno y recibiendo una señal de recom-

pensa escalar que indica la calidad de sus acciones. A diferencia de los paradigmas anteriores, no existe un conjunto de pares entrada–etiqueta predefinido: la señal de aprendizaje emerge de la interacción dinámica entre el agente y el entorno [36]. Aunque este paradigma no se emplea en el presente trabajo, completa la taxonomía fundamental del aprendizaje automático.

Junto a estos tres paradigmas, Murphy [9] identifica el **aprendizaje semi-supervisado** (*semi-supervised learning*) como un escenario de creciente interés práctico, en el que se combinan un conjunto reducido de datos etiquetados con una cantidad mayor de datos sin etiquetar. Este enfoque es especialmente valioso en dominios donde la obtención de anotaciones precisas es costosa, ya que permite aprovechar la abundancia de datos no etiquetados para mejorar la capacidad de generalización del modelo.

2.2 Aprendizaje profundo

El aprendizaje profundo (*deep learning*) es una rama del aprendizaje automático que emplea redes neuronales artificiales con múltiples capas de procesamiento para aprender representaciones jerárquicas de los datos directamente a partir de los datos crudos, sin necesidad de extracción manual de características [36].

El **perceptrón multicapa** (*Multilayer Perceptron*, MLP) es la arquitectura más elemental y constituye el bloque de construcción sobre el que se edifican las redes profundas modernas [36]. Está formado por una capa de entrada, una o más capas ocultas y una capa de salida. Cada capa aplica una transformación afín seguida de una función de activación no lineal σ:

$$\mathbf{h}^{(l)} = \sigma\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) \tag{2.1}$$

donde $\mathbf{W}^{(l)}$ y $\mathbf{b}^{(l)}$ son la matriz de pesos y el vector de sesgo de la capa l, y $\mathbf{h}^{(0)} = \mathbf{x}$ es la entrada. La composición de múltiples transformaciones no lineales permite aproximar funciones de complejidad arbitraria [36]. La Figura 2.1 ilustra la arquitectura de un MLP con dos capas ocultas.

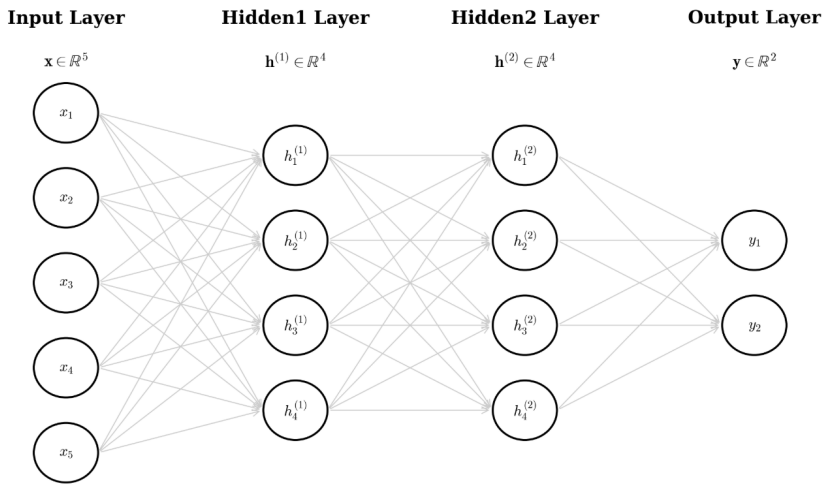


Figura 2.1: Arquitectura de un perceptrón multicapa con dos capas ocultas.

Una extensión natural del MLP que explota la estructura local de los datos son las **redes neuronales convolucionales** (*Convolutional Neural Networks*, CNN). En lugar de

conectar cada neurona con todas las de la capa anterior, emplean filtros de peso compartido que se desplazan a lo largo de la entrada [36]. En su variante unidimensional (Conv1D), cada filtro de tamaño K calcula en cada posición t:

$$y_t = \sigma\left(\sum_{k=0}^{K-1} w_k \cdot x_{t+k} + b\right) \tag{2.2}$$

El parámetro *stride* controla el desplazamiento entre posiciones consecutivas, actuando como factor de submuestreo. El peso compartido reduce el número de parámetros e introduce invariancia a la traslación: el mismo patrón local se detecta independientemente de su posición en la entrada [36]. La Figura 2.2 ilustra esta operación.

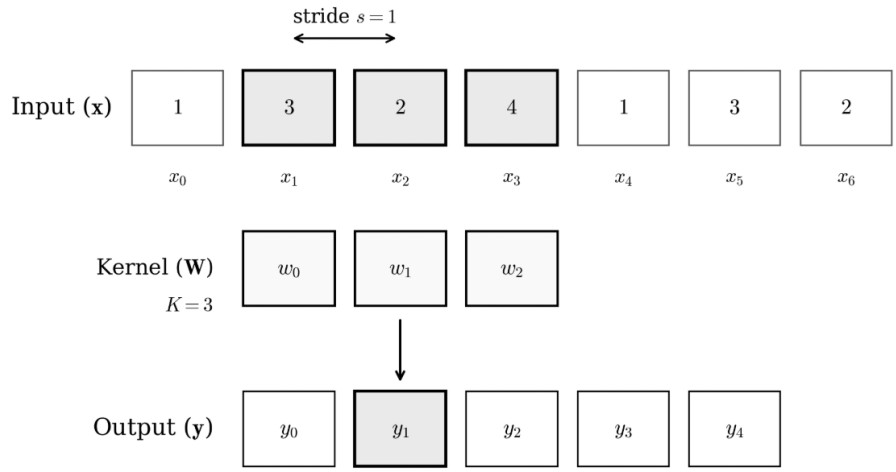


Figura 2.2: Convolución 1D con kernel de tamaño $K = 3$ y stride $s = 1$.

La capacidad representacional de las arquitecturas anteriores se puede aprovechar de forma más eficiente mediante el **aprendizaje multitarea** (*Multitask Learning*, MTL), un paradigma en el que un único modelo —típicamente una red profunda con capas compartidas— se optimiza simultáneamente para varias tareas relacionadas [9]. La función de pérdida conjunta se define como:

$$\mathcal{L}_{\text{total}} = \sum_{t=1}^{T} \lambda_t \mathcal{L}_t \tag{2.3}$$

donde $\mathcal{L}_t$ es la pérdida de la tarea t y λ_t su peso de ponderación. Las capas compartidas actúan como regularizador implícito, forzando al modelo a aprender representaciones internas útiles para todas las tareas a la vez. No obstante, un diseño inadecuado puede producir *transferencia negativa*: situaciones en las que el entrenamiento conjunto perjudica alguna tarea debido a gradientes conflictivos entre ellas [8].

2.3 Transformers

Transformers es una arquitectura de red neuronal propuesta por Vaswani et al. [27] que prescinde por completo de la recurrencia y la convolución, sustituyéndolas por un mecanismo de **autoatención** (*self-attention*) capaz de modelar dependencias entre cualquier par de posiciones de una secuencia en un único paso de cómputo. Esta propiedad, junto con su alta capacidad de paralelización durante el entrenamiento, ha convertido al Transformer en la arquitectura dominante en prácticamente todos los campos del aprendizaje profundo [36].

El mecanismo de autoatención opera sobre tres matrices derivadas de la entrada: consultas $\mathbf{Q}$, claves $\mathbf{K}$ y valores $\mathbf{V}$. La atención se calcula como una suma ponderada de los valores, donde los pesos se obtienen mediante el producto escalar entre consultas y claves, escalado por la raíz cuadrada de su dimensión d_k [27]:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{2.4}$$

En la práctica, este mecanismo se aplica en paralelo H veces con proyecciones lineales independientes, dando lugar a la **atención multi-cabeza** (*multi-head attention*), que permite al modelo atender simultáneamente a información procedente de distintos subespacios de representación [27].

La arquitectura original, ilustrada en la Figura 2.3, sigue un diseño *encoder–decoder*.

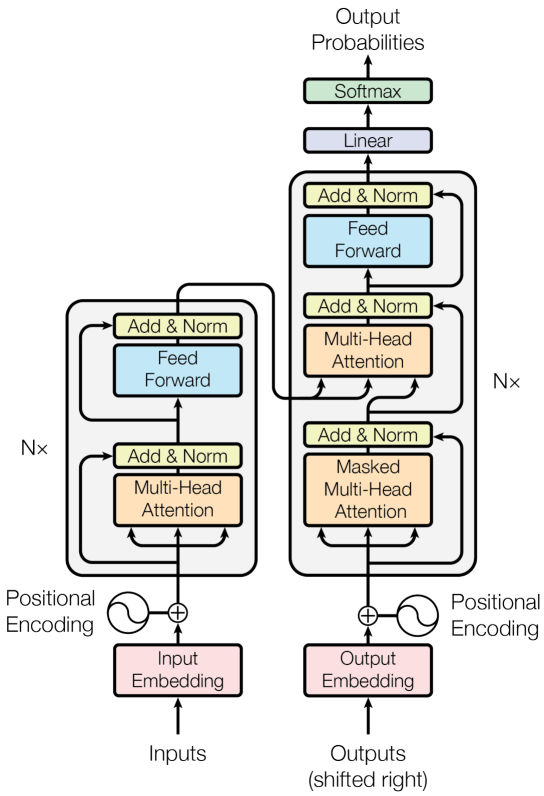


Figura 2.3: Arquitectura *Transformer encoder–decoder* original [27].

Como se puede observar en la Figura 2.3, el encoder procesa la secuencia de entrada y produce una representación contextualizada de cada posición; el decoder genera la secuencia de salida de forma autoregresiva, atendiendo tanto a su propia salida parcial como a la representación del encoder mediante atención cruzada [27]. Cada bloque del encoder y del decoder combina una subcapa de atención multi-cabeza con una *Feed-Forward Network* (FNN), envueltas en conexiones residuales y normalización de capa [36].

Dado que la autoatención es invariante al orden, se incorporan **codificaciones posicionales** (*positional encodings*) para inyectar información sobre la secuencia [27]. Con esta estructura modular, el Transformer establece un marco robusto y eficiente para el modelado contextual de datos secuenciales.

2.4 Transcripción automática de música

La transcripción automática de música (*Automatic Music Transcription*, AMT) es la tarea de convertir una señal de audio musical en una representación simbólica de la música interpretada [37]. Constituye uno de los problemas centrales de la recuperación de información musical (*Music Information Retrieval*, MIR) y combina desafíos propios del procesamiento de señales, la teoría musical y el aprendizaje automático.

Desde una perspectiva formal, AMT puede entenderse como un problema de *transducción de secuencias*: dada una señal de audio discreta $\mathbf{x} \in \mathbb{R}^T$, el sistema debe producir una secuencia de eventos simbólicos que describan con precisión la música contenida en dicha señal. La dificultad intrínseca del problema reside en la naturaleza **polifónica** del audio musical: múltiples instrumentos o voces suenan simultáneamente, sus armónicos se superponen en el dominio de la frecuencia y la señal resultante no es una suma trivialmente separable de sus componentes [37].

En el caso del piano, que es el instrumento de referencia en este trabajo, la tarea se descompone habitualmente en la estimación conjunta de cuatro atributos para cada nota: su **tono** (*pitch*), su instante de inicio (*onset*), su instante de finalización (*offset*) y su **dinámica** o velocidad (*velocity*) [37]. La estimación precisa del offset es especialmente difícil en presencia del pedal de sustain, cuyo efecto prolonga la resonancia del instrumento más allá del instante físico de liberación de la tecla.

Los primeros sistemas de AMT se basaban en métodos de descomposición espectral no negativos (NMF) y modelos probabilísticos como los modelos de Markov ocultos [37]. Con la irrupción del aprendizaje profundo, los sistemas actuales del estado del arte abordan la tarea mediante redes neuronales que procesan representaciones tiempo–frecuencia del audio y estiman directamente las activaciones de nota. En este contexto, el flujo de trabajo más extendido en la literatura consiste en dos etapas diferenciadas: en primer lugar, el sistema produce un **performance MIDI**, es decir, una representación simbólica que captura con precisión los eventos de nota tal y como fueron interpretados; en segundo lugar, dicho performance MIDI se transforma en una **partitura** (*score*), que constituye la representación musical canónica legible por un músico. Esta separación permite especializar cada etapa de forma independiente.

2.5 Representación digital del audio

Para que una señal de audio pueda ser procesada por un sistema de aprendizaje automático, debe ser previamente convertida a una representación numérica adecuada.

El audio analógico se digitaliza mediante un proceso de muestreo y cuantización. El muestreo consiste en registrar el valor de la señal a intervalos regulares con una frecuencia f_s denominada **frecuencia de muestreo**; el teorema de Nyquist–Shannon establece que para representar fielmente una señal de ancho de banda B es necesario que $f_s \geq 2B$ [41]. En audio de calidad CD, $f_s = 44{,}1\,\text{kHz}$, suficiente para cubrir el rango de audición humana. Cada muestra se cuantiza mediante codificación PCM (*Pulse Code Modulation*), representando su amplitud como un entero de 16 o 24 bits [41].

Respecto al análisis de frecuencias, este se realiza mediante la transformada de Fourier. Sin embargo, las señales musicales son **no estacionarias**: su contenido frecuencial varía a lo largo del tiempo. La **Transformada de Fourier de Tiempo Corto** (*Short-Time Fourier Transform*, STFT) resuelve este problema segmentando la señal en ventanas temporales cortas y aplicando la transformada de Fourier a cada una de ellas [41]:

$$X(m,k) = \sum_{n=0}^{N-1} x(n+mH)\, w(n)\, e^{-j2\pi kn/N} \tag{2.5}$$

donde $w(n)$ es la función de ventana, N es el tamaño de la ventana, H es el salto entre ventanas consecutivas (*hop size*) y m es el índice de trama. El módulo al cuadrado $|X(m,k)|^2$ constituye el **espectrograma**, una representación bidimensional tiempo–frecuencia de la energía de la señal. La STFT distribuye las bandas de frecuencia de forma **lineal**, lo que implica una resolución excesiva en frecuencias agudas y escasa en frecuencias graves, algo que no se corresponde con la percepción auditiva humana ni con la estructura de la música tonal.

Para alinear la representación frecuencial con la percepción auditiva, se aplica un banco de filtros triangulares distribuidos en escala **mel** [38]. La escala mel es una escala perceptual en la que distancias iguales corresponden a diferencias de tono percibidas como iguales por el oído humano. Cada filtro integra la energía del espectrograma en una banda de frecuencias, y el resultado se transforma a escala logarítmica, produciendo el **espectrograma log-mel**:

$$S_{\text{mel}}(m,b) = \log\left(1 + \sum_{k} H_b(k)\, |X(m,k)|^2\right) \tag{2.6}$$

donde $H_b(k)$ es la respuesta del b-ésimo filtro mel. Esta representación compacta y perceptualmente motivada es ampliamente utilizada como entrada en sistemas de reconocimiento de voz, clasificación de música y transcripción automática [38].

2.6 Representaciones simbólicas de la música

La música puede representarse de formas muy distintas en función del nivel de abstracción y del uso que se quiera hacer de ella. En el contexto de la transcripción automática, las dos representaciones simbólicas fundamentales son el ***performance* MIDI** y la **partitura musical**. Comprender la diferencia entre ambas es esencial para entender la arquitectura de los sistemas de AMT modernos.

MIDI (*Musical Instrument Digital Interface*) es un protocolo de comunicación estándar para instrumentos electrónicos, definido por la MIDI Manufacturers Association [40]. A diferencia del audio, que representa la presión sonora como una señal continua, MIDI representa la música como una secuencia de **eventos discretos**. Los eventos más relevantes para la transcripción de piano son:

- **Note On / Note Off:** indican el inicio y el final de una nota, identificada por su número de tono (*pitch*) en el rango $[0, 127]$, donde cada unidad corresponde a un semitono.
- **Velocidad (*velocity*):** valor entero en $[0, 127]$ asociado al evento Note On que codifica la intensidad de pulsación de la tecla y determina la dinámica de la nota.
- **Mensajes de control (CC):** regulan parámetros continuos como la posición de los pedales. El pedal de sustain (CC64), el pedal sostenuto (CC66) y el pedal una corda (CC67) son especialmente relevantes en la transcripción de piano, ya que modifican la duración sonora de las notas más allá del instante físico de liberación de la tecla.

Este protocolo se usa principalmente para representar *performance* MIDI, que es un archivo que registra una interpretación musical tal y como fue ejecutada, capturando con precisión los tiempos absolutos de cada evento en segundos o *ticks*. En él no existe ninguna noción de compás, figura musical, alteración o armadura: los eventos simplemente ocurren en instantes de tiempo concretos. La Figura 2.4 ilustra un fragmento de *performance* MIDI con sus eventos de nota y pedal.

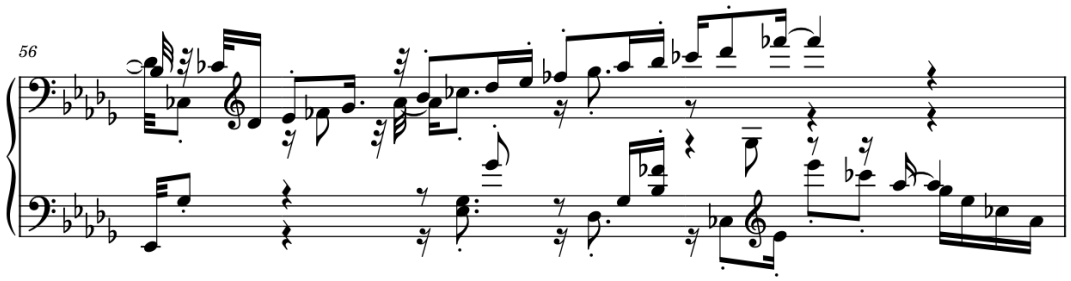


Figura 2.4: Fragmento de *performance* MIDI.

Una forma habitual de visualizar un *performance* MIDI es el *piano roll*, una representación matricial bidimensional en la que el eje horizontal corresponde al tiempo, el eje vertical a los 88 tonos del piano estándar, y cada celda indica si una nota está activa en un instante dado. La Figura 2.5 muestra el *piano roll* correspondiente al fragmento de la Figura 2.4.

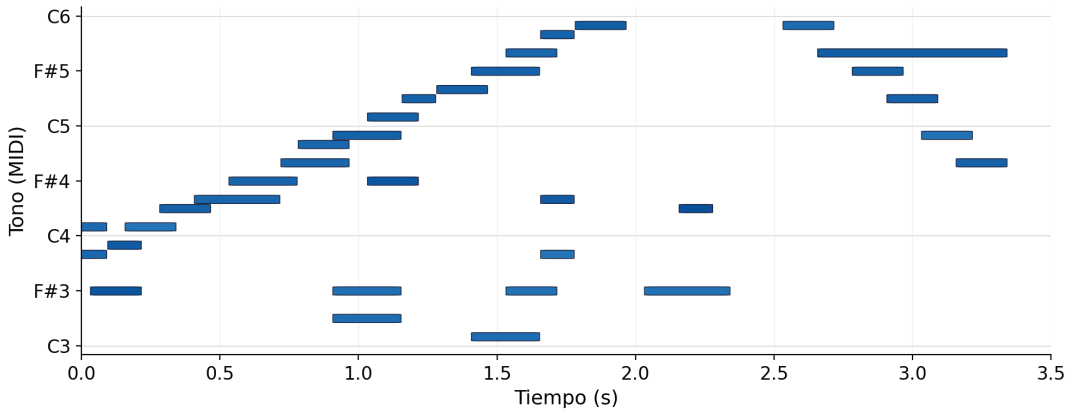


Figura 2.5: Fragmento de *piano roll*.

Como última forma de representación se usa la **partitura musical** (*score*). A diferencia del *performance* MIDI, la partitura no describe cuándo ocurre exactamente cada evento en términos de tiempo absoluto, sino qué debe tocarse en términos de notación musical: figuras de duración (redonda, blanca, negra, etc.), compases, armaduras, alteraciones, ligaduras, indicaciones de dinámica y ornamentos. Esta representación es la que produce y lee un músico, y su obtención automática a partir de audio constituye el objetivo último de la AMT completa (*audio-to-score*) [37].

MusicXML es el formato estándar de intercambio para partituras digitales [39]. Representa la partitura como un documento XML jerárquico en el que se codifican los elementos estructurales de la notación: pentagramas, compases, voces, notas con su figura y altura, silencios, articulaciones y detalles interpretativos. La Figura 2.6 muestra el mismo fragmento musical en notación de partitura estándar tal y como se visualiza a partir de un archivo MusicXML.

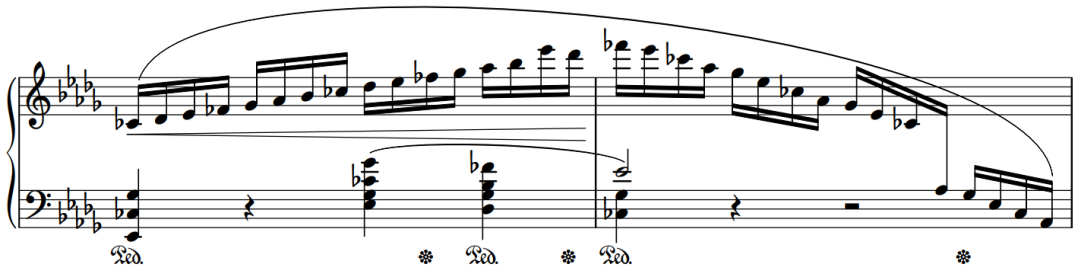


Figura 2.6: Fragmento en notación de partitura estándar (MusicXML).

La diferencia fundamental entre *performance* MIDI y partitura radica en el nivel de abstracción musical: el primero es una descripción *física* de la interpretación (tiempos exactos, velocidades continuas, pedal) mientras que la segunda es una descripción *simbólica e intencional* de la obra figuras, compases, voces, ornamentos, etc.

2.7 MIDI2ScoreTransformer

La herramienta *midi2scoreTransformer* [35] realiza la conversión automática de archivos *performance MIDI* a partituras legibles en formato MusicXML mediante un modelo *transformer encoder–decoder* especializado. Su objetivo es transformar una secuencia de notas cruda, descrita únicamente por altura, tiempos de inicio y fin y velocidad, en una notación musical compacta y clara, similar a la que produciría un copista humano. En el contexto de este TFG, *midi2scoreTransformer* se emplea como etapa de postprocesado de los sistemas de transcripción automática de piano: a partir de las salidas en formato MIDI de interpretación, se generan partituras MusicXML que facilitan el análisis visual y la discusión de resultados por parte de lectores no especialistas en MIR o procesamiento de audio.

Para poder aplicar un *transformer* de forma eficiente, el sistema introduce un esquema de tokenización paralelo tanto para la entrada (performance MIDI) como para la salida (MusicXML) [35]. En la entrada, cada nota se representa mediante cuatro flujos de tokens que codifican la altura MIDI, el instante de inicio, la duración (ambos cuantizados mediante *buckets* con transformación logarítmica) y la velocidad, lo que reduce la longitud de secuencia manteniendo la información relevante. En la salida, el modelo predice desde cero una secuencia de notas de partitura enriquecida con múltiples atributos: altura y tiempo musical cuantizados dentro del compás, duración musical, información estructural (mano o pentagrama, número de voz, dirección de plica, alteraciones) y detalles interpretativos como staccatos, notas de adorno y trinos. Esta representación paralela permite que cada nota ocupe un único paso temporal, y combina la posición relativa dentro del compás con la longitud del compás anterior para reconstruir tiempos absolutos y barras de compás, recuperando ligaduras y silencios durante la conversión posterior a MusicXML [35].

En el núcleo de la herramienta se utiliza un modelo tipo RoFormer, es decir, un *transformer* con codificaciones posicionales rotatorias (*Rotary Positional Embeddings*), dispuesto en una arquitectura estándar *encoder–decoder* autoregresiva. Los distintos flujos de tokens de cada nota se embeben por separado y se suman en un espacio común de dimensión fija antes de alimentar las pilas de atención. La arquitectura integra técnicas como normalización de capa previa a la atención (*pre-norm*) y funciones de activación de tipo SwiGLU en las capas de alimentación directa, lo que mejora la estabilidad del entrenamiento y la capacidad del modelo para capturar dependencias de largo alcance, cruciales para estimar el compás y mantener la coherencia de notación a lo largo de la pieza. El entrenamiento

se plantea como una tarea de clasificación múltiple en cada paso temporal, con una función de pérdida basada en entropía cruzada sobre todos los atributos de salida y sobre un token especial de espaciado que permite alinear los flujos paralelos; para la generación se emplea decodificación autoregresiva *greedy*[35].

El flujo de trabajo de inferencia puede resumirse en las siguientes etapas:

1. Tokenización del archivo *performance MIDI* procedente bien de una interpretación real, bien de un sistema de transcripción, extrayendo los flujos de tokens de entrada descritos.

2. Traducción de dichos tokens a los flujos de salida de MusicXML mediante el Transformer *encoder–decoder*.

3. Reconstrucción de la partitura en formato MusicXML, donde se derivan tiempos absolutos, compases y ligaduras, se insertan silencios en los huecos entre notas y se generan los elementos de notación final.

El resultado es una partitura legible en cualquier editor estándar y adecuada para el análisis cualitativo y cuantitativo de las transcripciones generadas en este trabajo.

Respecto a la implementación, el código original de *midi2scoreTransformer* [35] se apoyaba en clases internas de versiones anteriores de la biblioteca `transformers` de HuggingFace, lo que generaba incompatibilidades con versiones recientes. Para utilizar la herramienta en el entorno actual, se ha refactorizado el módulo RoFormer del proyecto sustituyendo la herencia de dichas clases internas por una jerarquía de módulos puros de PyTorch que replica la lógica del modelo original. Esta reestructuración mantiene intactos los nombres de las capas y la definición matemática, de modo que el *checkpoint* preentrenado se puede cargar sin modificaciones y el sistema funciona con las versiones actuales de las bibliotecas, garantizando su estabilidad futura y reutilización como componente de postprocesado en este TFG.

CAPÍTULO 3
MAESTRO

3.1 Introducción

Uno de los principales factores que limitan el rendimiento de los sistemas de AMT basados en el aprendizaje profundo es la disponibilidad de datos de entrenamiento etiquetados con alta precisión y en cantidad suficiente para que los modelos sean capaces de generalizar. Hasta 2018, los *datasets* disponibles para esta tarea, como **MAPS** (*MIDI Aligned Piano Sounds*), no superaban las 70 horas de audio, lo que suponía una barrera significativa para el entrenamiento de nuevos modelos [1].

En el ámbito de este trabajo, **Google Magenta** publicó el *dataset* **MAESTRO** [6] (*Midi and Audio Edited for Synchronous Tracks and Organization*), que supuso un cambio de paradigma al ofrecer casi 200 horas de grabaciones de piano de alta calidad con alineación precisa entre audio y etiquetas MIDI. La publicación de MAESTRO demostró inmediatamente su impacto: el modelo **Onsets and Frames**, entrenado sobre MAESTRO alcanzó un **F1 de 67,43 %** en la evaluación sobre grabaciones Disklavier del conjunto MAPS, superando notablemente el **50,22 %** obtenido por el mismo modelo entrenado sobre MAPS únicamente [5, 6]. Desde entonces, MAESTRO se ha consolidado como el estándar de facto para la evaluación y comparación de sistemas de transcripción automática de piano.

En este trabajo se utiliza la versión **V3.0.0** [3] del *dataset*, publicada en 2020, que constituye una versión más depurada y es actualmente la referencia en la comunidad investigadora.

Este capítulo analiza el *dataset* MAESTRO. La Sección 3.2 describe su origen en la *International Piano-e-Competition* y el uso de pianos Yamaha Disklavier. El proceso de adquisición y anotación se detalla en la Sección 3.3, mientras que sus características técnicas, evolución y licenciamiento se exponen en las Secciones 3.4, 3.5 y 3.6, respectivamente. Posteriormente, se discuten sus limitaciones (Sección 3.7) y su relevancia para este trabajo (Sección 3.8). Finalmente, la Sección 3.9 presenta el protocolo de evaluación estándar para la transcripción automática de piano.

3.2 Origen de los datos

Los datos que constituyen MAESTRO provienen de grabaciones recogidas durante diez ediciones de la *International Piano-e-Competition*, una competición de piano de nivel internacional celebrada anualmente en la que pianistas virtuosos interpretan sobre pianos de cola **Yamaha Disklavier** equipados con un sistema integrado de captura MIDI de alta precisión [3]. Esta competición es la fuente exclusiva de todas las grabaciones del

dataset, lo que dota al conjunto de datos de una coherencia excepcional en términos de calidad de la interpretación, condiciones acústicas y equipamiento.

En cuanto al instrumento empleado para la captación, el elemento técnico que garantiza la calidad de MAESTRO es el piano Yamaha Disklavier, un piano de cola acústico convencional al que se le ha añadido un sistema electrónico de captura que registra, con alta precisión temporal, el movimiento de cada tecla y pedal durante la interpretación [6]. Este sistema está formado por un conjunto de sensores situados bajo las teclas y pedales, que detectan la posición y velocidad de cada elemento, así como actuadores que permiten reproducir automáticamente la interpretación a partir de los datos MIDI [4]. A diferencia de los sintetizadores o controladores MIDI, el Disklavier produce sonido acústico real al mismo tiempo que genera los datos MIDI, lo que garantiza que el audio captado por los micrófonos y el MIDI registrado sean inherentemente sincrónicos [6].

La Figura 3.1 ilustra la disposición de estos componentes bajo el teclado. La clave tecnológica que aporta valor a este *dataset* reside en el uso de **sensores ópticos sin contacto**. Al evitar cualquier fricción o resistencia mecánica adicional, el sistema logra escanear continuamente el movimiento del martillo y la tecla para medir la dinámica (velocidad) y los instantes de *onset* y *offset* con absoluta fidelidad, todo ello preservando intacta la experiencia táctil del intérprete [4].



Figura 3.1: Sistema de sensores de un Yamaha Disklavier. En la parte superior se aprecia el teclado con sus teclas blancas y negras. En la franja central, bajo cada tecla, se dispone una fila de **sensores ópticos** cuyos emisores de luz infrarroja (visibles como puntos rojos) detectan de forma continua y sin contacto el desplazamiento del martillo y de la tecla a lo largo de su recorrido. A ambos lados se observan los mecanismos de acción con sus amortiguadores y pines de guía. Al no introducir ninguna fricción ni resistencia mecánica adicional, estos sensores pueden medir con precisión el instante exacto en que cada tecla es pulsada (*onset*) y liberada (*offset*), así como la velocidad de pulsación, que determina la dinámica (intensidad) de la nota en el protocolo MIDI. Esta arquitectura de medición sin contacto es la que permite que el *dataset* MAESTRO ofrezca una alineación entre el audio grabado y las etiquetas MIDI de aproximadamente ± 3 ms, muy por debajo del umbral de percepción temporal humano [4].

La alineación resultante entre la señal de audio y las etiquetas MIDI en MAESTRO es de aproximadamente ± 3 ms, muy inferior al umbral de percepción temporal humano (estimado en 20 ms), lo que hace que las etiquetas se consideren alineadas a nivel de fonación. Esta precisión es fundamental para entrenar modelos de transcripción: cualquier desalineación sistemática entre el *onset* de una nota en el audio y su etiqueta MIDI correspondiente introduce ruido en los gradientes durante el entrenamiento y degrada el rendimiento de los modelos.

3.3 Adquisición y anotación

En primer lugar, el audio se graba en calidad CD o superior, con una frecuencia de muestreo de 44,1 a 48 kHz y una profundidad de bit de 16 bits PCM[1] estéreo. Esta calidad es suficiente para representar con fidelidad toda la gama de frecuencias audibles relevantes para el piano (entre 28 Hz y 4 kHz aproximadamente para los fundamentales, con armónicos que se extienden hasta 16-20 kHz). Las grabaciones se almacenan sin compresión con pérdida, lo que preserva toda la información espectral original.

De forma simultánea a la captura del audio, el sistema MIDI del Disklavier captura automáticamente los siguientes eventos para cada interpretación:

- Velocidades de pulsación de teclas (*key strike velocities*, canal MIDI, nota, velocidad 0-127): codifican la dinámica de cada nota individual.
- Posición del pedal *sustain* (Control Change CC64): el pedal más utilizado en el piano, que prolonga el sonido de una tecla.
- Posición del pedal *sostenuto* (Control Change CC66): permite mantener seleccionadas únicamente las notas que están sonando en el momento de su activación.
- Posición del pedal *una corda* (Control Change CC67): pedal izquierdo, que reduce el volumen y modifica el timbre.

La inclusión de los tres pedales hace a MAESTRO uno de los pocos *datasets* de transcripción de piano con **anotación de pedal completa**, lo que resulta especialmente relevante para sistemas que aspiran a una transcripción expresiva o a la síntesis de audio realista a partir de MIDI.

Cada grabación se segmenta automáticamente en piezas musicales individuales y se anota con los siguientes metadatos:

- Compositor canónico (nombre normalizado).
- Título de la obra.
- Año de la grabación.
- Partición sugerida (train/validation/test).

Estos metadatos permiten analizar el repertorio, controlar posibles solapamientos entre particiones y reproducir de forma precisa los protocolos de evaluación propuestos por los autores [3].

3.4 Características del *dataset*

MAESTRO v3.0.0 está formado por un total de 1 276 grabaciones de interpretaciones de piano, que suman 198,7 horas de audio y más de 7,4 millones de eventos MIDI [3]. El repertorio incluye obras de 60 compositores distintos que se asocian principalmente a la música clásica occidental entre los siglos XVII y XX.

[1] PCM (*Pulse Code Modulation*) es una técnica de representación digital de señales analógicas basada en el muestreo y la cuantización, ampliamente utilizada en sistemas de audio sin compresión.

Compositor	Grabaciones	Obras únicas	Duración (h)
Frédéric Chopin	201	107	26,22
Franz Schubert	186	96	35,82
Ludwig van Beethoven	146	95	26,92
Johann Sebastian Bach	145	93	13,77
Franz Liszt	131	91	20,27
Sergei Rachmaninoff	59	50	6,66
Robert Schumann	49	31	15,72
Claude Debussy	45	41	5,97
Joseph Haydn	40	29	4,72
Wolfgang Amadeus Mozart	38	35	5,60
Alexander Scriabin	35	26	5,35
Domenico Scarlatti	31	29	1,71
Felix Mendelssohn	28	8	4,85
Johannes Brahms	26	23	8,06
Franz Schubert / Franz Liszt	10	7	0,69
Isaac Albéniz	7	6	0,87
Modest Mussorgsky	6	2	3,17
Pyotr Ilyich Tchaikovsky / Mikhail Pletnev	6	6	0,80
Pyotr Ilyich Tchaikovsky	6	6	0,71
Muzio Clementi	6	6	0,87

Tabla 3.1: Los 20 compositores con mayor número de grabaciones en MAESTRO v3.0.0 [3].

La Tabla 3.1 muestra la distribución típica de competiciones internacionales de piano, que tienden a centrarse en el período Romántico y en los grandes compositores del clasicismo tardío.

Respecto a las particiones, estas están predefinidas y publicadas con el *dataset*, lo que garantiza la reproducibilidad de los experimentos y permite la comparación directa de distintos modelos. Esto es fundamental en un campo como AMT, en el que pequeñas diferencias en la partición de datos pueden producir variaciones significativas en los resultados reportados. En la Tabla 3.2 se detallan las estadísticas principales por partición.

Partición	Grabaciones	Duración (h)	Tamaño (GB)	Notas (M)
Train	962	159,2	96,3	5,66
Validation	137	19,4	11,8	0,64
Test	177	20,0	12,1	0,74
Total	**1,276**	**198,7**	**120,2**	**7,40**

Tabla 3.2: Estadísticas de MAESTRO v3.0.0 desglosadas por partición.

La política de particionado distingue el *dataset* de otras colecciones de audio-MIDI. El criterio establecido por los autores es el siguiente: una misma composición no puede aparecer simultáneamente en más de una partición, aunque haya sido interpretada por distintos pianistas en periodos diferentes [3].

Esta decisión de diseño es crucial para garantizar la validez de los experimentos. Si múltiples interpretaciones de una misma obra aparecieran en entrenamiento y test, un modelo podría aprender características tímbricas o estructurales específicas de esa obra y obtener resultados artificialmente buenos en la evaluación sin haber generalizado correctamente. De esta forma, los modelos se evalúan sobre obras no vistas en entrenamiento, lo que constituye una medida mucho más exigente y realista de rendimiento.

3.5 Evolución del *dataset*

MAESTRO ya va por su tercera versión desde su publicación original en 2018. En la Tabla 3.3 podemos ver características y principales cambios de cada versión.

Versión	Año	Grabaciones	Horas	Notas (M)	Principal novedad
V1.0.0	2018	1,184	172,3	6,18	Publicación original [3]; datos 2004–2016.
V2.0.0	2019	1,282	201,2	7,13	Añade edición 2018 [3]; incorpora pedales *sostenuto* y *una corda*.
V3.0.0	2020	1,276	198,7	7,04	Elimina 6 grabaciones con acompañamiento orquestal [3].

Tabla 3.3: Comparativa de las tres versiones publicadas de MAESTRO.

La transición de V2.0.0 a V3.0.0 merece especial atención. Contrariamente a lo que podría esperarse, la versión más reciente no amplía el *dataset*, sino que realiza una corrección de calidad eliminando 6 grabaciones que no eran de piano solo. La presencia de estas grabaciones en versiones anteriores constituía una inconsistencia que podía afectar negativamente al entrenamiento de modelos especializados en piano solo. Esta corrección hace que las particiones de V3.0.0 no sean compatibles con las de V2.0.0, por lo que todos los experimentos de este trabajo se realizan íntegramente sobre V3.0.0 para garantizar la comparabilidad con los resultados publicados recientemente.

3.6 Distribución y licencia

MAESTRO se distribuye a través de Google Magenta en tres modalidades de descarga [3]:

- **Audio + MIDI** (120,2 GB sin comprimir): contiene los archivos WAV y MIDI completos, junto con los metadatos en formato CSV y JSON.
- **Solo MIDI** ($\sim$ 57 MB): para tareas que no necesiten el audio.
- **Comprimido con pérdida** (MP3/FLAC): versión reducida para almacenamiento limitado.

El *dataset* se distribuye con licencia *Creative Commons Attribution Non-Commercial Share Alike 4.0 International* (CC BY-NC-SA 4.0), que implica:

- Atribución obligatoria: cualquier uso debe citar la fuente original.
- Prohibido su uso con fines comerciales.
- Las obras derivadas deben distribuirse bajo la misma licencia.

Esta licencia es compatible con el uso académico en el contexto de un Trabajo de Fin de Grado.

3.7 Limitaciones

A pesar de sus fortalezas, MAESTRO presenta una serie de limitaciones que debemos tener en cuenta al interpretar los resultados de los sistemas entrenados sobre él.

Sesgo de repertorio. El repertorio es casi exclusivamente música clásica occidental de los siglos XVII-XX. Los modelos entrenados con MAESTRO no generalizan bien a géneros como el *jazz*, el pop o la música contemporánea con técnicas extendidas. Esto se debe a que las distribuciones de patrones armónicos, rítmicos y de velocidad son muy diferentes a las de la música clásica.

Sesgo de instrumento. Todas las grabaciones se realizaron sobre pianos Yamaha Disklavier del mismo modelo y en condiciones acústicas muy similares. Es por esto que los modelos entrenados con MAESTRO muestran una degradación significativa de rendimiento cuando se evalúan sobre grabaciones de pianos distintos en entornos con reverberación variable. Esto es especialmente relevante para aplicaciones del mundo real, donde el instrumento y la acústica son variables.

Ausencia de anotación de partitura. MAESTRO proporciona MIDI, pero no partitura en formato estándar como MusicXML o MEI [3]. Para tareas de transcripción audio-a-partitura (*Audio-to-Score*), como parte del flujo de las tareas de AMT, es necesario complementar MAESTRO con *datasets* que incluyan partituras anotadas, como ASAP (*Aligned Scores and Performances*).

3.8 Relevancia para este trabajo

MAESTRO v3.0.0 es el *dataset* principal empleado para el entrenamiento y evaluación de los modelos de transcripción automática de piano desarrollados en este trabajo.

Los modelos de referencia del estado del arte, incluyendo los modelos empleados en este trabajo, reportan sus resultados sobre las particiones fijas de MAESTRO, lo que hace directamente comparables los experimentos aquí presentados con los resultados publicados en la literatura.

Su volumen de casi 200 horas con más de **7 millones de eventos MIDI** proporciona datos suficientes para el entrenamiento de arquitecturas basadas en *transformers* sin riesgo de sobreajuste, siempre y cuando se apliquen las técnicas de regularización adecuadas. Todos estos eventos tienen una precisión de alineación de ± 3 ms, garantizando la calidad de las etiquetas *onset*, *offset* y velocidad, que son las variables objetivo de los modelos de transcripción empleados.

Finalmente, las particiones fijas y predefinidas garantizan la reproducibilidad de los experimentos.

3.9 Evaluación estándar

La evaluación cuantitativa de los sistemas de transcripción automática de piano entrenados sobre MAESTRO sigue un protocolo consolidado en la comunidad, implementado mediante la librería `mir_eval` [7]. Este protocolo es el empleado de forma invariante por todos los modelos del estado del arte que reportan resultados sobre este *dataset*, incluyendo los modelos reproducidos en este trabajo, MFT-FiLM y Transkun. Es por esto que en esta sección veremos cómo se evalúan ambos modelos.

Antes de aplicar las métricas, la salida de cada modelo, ya sea en forma de *piano roll* (MFT-FiLM) o en forma de intervalos decodificados (Transkun) se convierte en un conjunto de eventos MIDI discretos de la forma $\{(\text{tono}, t_{\text{onset}}, t_{\text{offset}}, v)\}$, donde el tono es un valor entero en $[21, 108]$ (las 88 teclas del piano estándar), los tiempos se expresan en segundos y la velocidad $v \in [0, 127]$ es un entero MIDI. La evaluación opera directamente sobre estos eventos discretos extraídos del MIDI de salida, comparándolos con

los eventos del MIDI de *ground truth*, sin acceder en ningún momento a la representación interna del modelo, de forma que se garantiza la independencia de las métricas respecto a la arquitectura empleada.

Las métricas previamente mencionadas se organizan en dos categorías en función del nivel de abstracción al que operan: nivel de trama y nivel de nota.

Las **métricas a nivel de trama** (*frame-level*) cuantifican la capacidad del modelo para estimar qué teclas están activas en cada instante temporal, con independencia de cómo se delimiten los eventos. Para ello, tanto la predicción como el *ground truth* se convierten en matrices binarias de dimensiones $T \times 88$, con resolución temporal infinitesimal (equivalente a un piano roll continuo), comparando el estado de activación de cada tecla en cada instante.

Se calculan la **precisión** (P), la **exhaustividad** (R, *Recall*) y la **puntuación** F_1:

$$P = \frac{VP}{VP + FP} \qquad R = \frac{VP}{VP + FN} \qquad F_1 = \frac{2 \cdot P \cdot R}{P + R} \tag{3.1}$$

donde VP son los verdaderos positivos (instantes y teclas correctamente activadas), FP los falsos positivos (activaciones predichas incorrectamente) y FN los falsos negativos (activaciones del *ground truth* no detectadas). Estas métricas de trama son las más directamente relacionadas con la audición subjetiva de la transcripción, ya que capturan el porcentaje de tiempo en que la armonía reconstruida es correcta. En términos musicales, una precisión alta indica que el modelo casi no introduce notas espurias que no aparecen en la grabación original, mientras que una exhaustividad alta implica que rara vez se olvida de activar notas que sí están presentes en la interpretación.

Por otro lado, las **métricas a nivel de nota** (*note-level*) operan sobre los eventos discretos y evalúan la capacidad del modelo para segmentar correctamente los límites temporales de las notas. Hay tres variantes de complejidad creciente:

- **Nota con *onset*** (*Note Onset*): una nota predicha se considera un verdadero positivo si y solo si su tono es correcto y su instante de inicio está dentro de una tolerancia de ± 50 ms respecto al *onset* de *ground truth* correspondiente.
- **Nota con *onset* y *offset*** (*Note w/ Offset*): además de las condiciones anteriores, el instante de finalización de la nota predicha debe encontrarse dentro de una tolerancia igual al máximo entre 50 ms y el 20 % de la duración de la nota de referencia. Esta condición menos restrictiva para el *offset* refleja la mayor dificultad de detección del final de una nota, especialmente en presencia de pedal de *sustain*, cuyo efecto prolonga la resonancia del instrumento más allá del instante de liberación de la tecla.
- **Nota con *onset*, *offset* y velocidad** (*Note w/ Offset & Velocity*): añade una condición sobre la estimación de la dinámica. La velocidad predicha debe satisfacer $|v_{\text{pred}} - v_{\text{ref}}| \leq 0{,}1$ tras escalar ambos valores al intervalo $[0, 1]$, lo que equivale a una tolerancia absoluta de aproximadamente 12,7 unidades MIDI. Esta es la métrica más exigente y la más representativa de la calidad de una transcripción completa de *performance* MIDI.

Desde la perspectiva de un intérprete, estas métricas de nota se aproximan mejor a la noción de "partitura correcta": no basta con que las notas suenen en algún momento, sino que deben empezar y terminar en puntos temporales razonables y con una intensidad coherente con la ejecución original.

Tabla 3.4: Condiciones de aceptación de un evento predicho como verdadero positivo según la variante de métrica de nota empleada.

Métrica	**Tono correcto**	**Tolerancia *onset***	**Condición adicional**
Frame	✓	—	—
Note Onset	✓	± 50 ms	—
Note w/ Offset	✓	± 50 ms	Tolerancia *offset*†
Note w/ Offset & Vel.	✓	± 50 ms	Tolerancia *offset*† + $\lvert\Delta v\rvert \leq 0{,}1$

† máx(50 ms, 20 % de la duración de la nota)

La Tabla 3.4 resume las condiciones de aceptación de cada variante.

Todos los valores de las métricas se **promedian sobre el número de piezas** del conjunto de evaluación, en lugar de calcularse de forma global sobre todas las notas. Este criterio es adoptado de forma uniforme por todos los trabajos de referencia [5, 13, 18].

Los valores de F_1 son los indicadores principales de rendimiento en la literatura de AMT, siendo la ***Note Onset*** F_1 la métrica más citada para comparar modelos, y la ***Note w/ Offset & Velocity*** F_1 la más representativa de la calidad de la transcripción completa.

CAPÍTULO 4
MFT-FiLM

4.1 Introducción

La transcripción de música de piano polifónico mediante redes neuronales se aborda principalmente como un problema de detección multitarea (*Multitask Learning*, o MTL)[10, 5], en el que se estiman de forma conjunta el tono a nivel de *frame*, los *onsets*, *offsets* y la velocidad de las notas. Esta formulación permite explotar la fuerte correlación entre estos eventos para mejorar la precisión global del sistema, pero también pone de manifiesto las limitaciones de las arquitecturas multitarea clásicas, que pueden sufrir transferencia negativa o una fusión ineficaz de la información cuando las tareas comparten parámetros de manera demasiado rígida o combinan características sin un mecanismo de ponderación adecuado [13].

En este proyecto, la primera alternativa que se ha implementado es el modelo **MFT-FiLM** [13], que propone un enfoque más flexible y eficiente basado en el acondicionamiento mediante FiLM para fusionar dependencias espectrales y temporales, así como para intercambiar información entre las distintas ramas de tarea. El uso de módulos FT-FiLM e IFT-FiLM permite ajustar dinámicamente las representaciones intermedias en función del contexto, obteniendo un modelo con un número de parámetros reducido que mantiene un rendimiento competitivo en la transcripción de piano a partir de representaciones adaptadas a la percepción musical.

En este capítulo, la Sección 4.2 introduce la representación tiempo–frecuencia empleada, basada en la Transformada Q Constante (CQT). A continuación, la Sección 4.3 describe la arquitectura general del sistema y detalla los módulos FiLM, FT-FiLM e IFT-FiLM. La Sección 4.5 presenta la función de pérdida conjunta y analiza la eficiencia computacional del modelo. La Sección 4.6 recoge el preprocesamiento de los datos, el procedimiento de entrenamiento y los resultados alcanzados. Finalmente, la Sección 4.7 sintetiza brevemente el trabajo realizado en este capítulo y destaca los principales resultados obtenidos.

4.2 Transformada Q Constante

El primer paso, antes de que la señal de audio entre en la red neuronal, es aplicar una cadena de preprocesamiento. El audio se convierte a mono y se remuestrea a 16 kHz. El oído humano no percibe las frecuencias de forma lineal: la diferencia perceptual entre 100 Hz y 200 Hz es la misma que entre 1000 Hz y 2000 Hz, ya que en ambos casos la segunda es el doble de la primera (diferencia de una octava musical). Las transformaciones tiempo–frecuencia clásicas como la **Transformada de Fourier de tiempo corto** (STFT) distribuyen las bandas de forma **lineal**, lo que implica que las frecuencias graves que-

dan con poca resolución y las frecuencias agudas con resolución excesiva, algo que no se corresponde ni con la estructura de la música ni con la percepción auditiva.

Es por esto por lo que se usa la **Transformada Q Constante** (*Constant-Q Transform*, CQT), que distribuye las bandas de frecuencia en escala logarítmica, alineándose con la percepción musical y con la disposición de las teclas en el piano [13]. En la CQT, cada *bin* cubre una banda proporcional a su frecuencia central, manteniendo una razón constante entre frecuencia y resolución (Q constante), lo que se traduce en ventanas temporales más largas en bajas frecuencias y más cortas en altas. Esto permite representar de forma más uniforme la energía armónica a lo largo de todo el rango del piano y facilita que la red neuronal aprenda patrones armónicos y melódicos coherentes con la notación musical.

En la Figura 4.1 se puede apreciar este efecto para un mismo fragmento de audio (*Prelude op. 28 no. 16, Chopin*, interpretación original), que presenta una distribución bastante uniforme sobre el piano, con presencia tanto en el registro grave como en el agudo. La STFT lineal agrupa la energía armónica en muy pocas bandas graves, mientras que la CQT asigna un número constante de *bins* por octava, distribuyendo la información de forma mucho más uniforme a lo largo del rango del instrumento. Esta diferencia visual resume la ventaja práctica de la CQT como representación de entrada para la transcripción automática de piano.

La CQT presenta los siguientes parámetros de configuración:

- Frecuencia mínima de 27,5 Hz correspondiente a la tecla A0 (la tecla más grave de un piano estándar).
- 48 *bins* por octava, con un total de 352.
- Ventana de *Hanning* y salto de trama (*hop length*) de 20 ms.

El espectrograma CQT de dimensiones $(T \times 352)$, donde T es el número de *frames*, se remodela a (C_0, T, F) con $C_0 = 4$ y $F = 88$. De esta forma, los 352 *bins* se distribuyen en cuatro grupos de 88 frecuencias, en correspondencia biunívoca con las 88 teclas del piano [13]. Esta representación constituye la entrada de la capa lineal inicial del modelo.

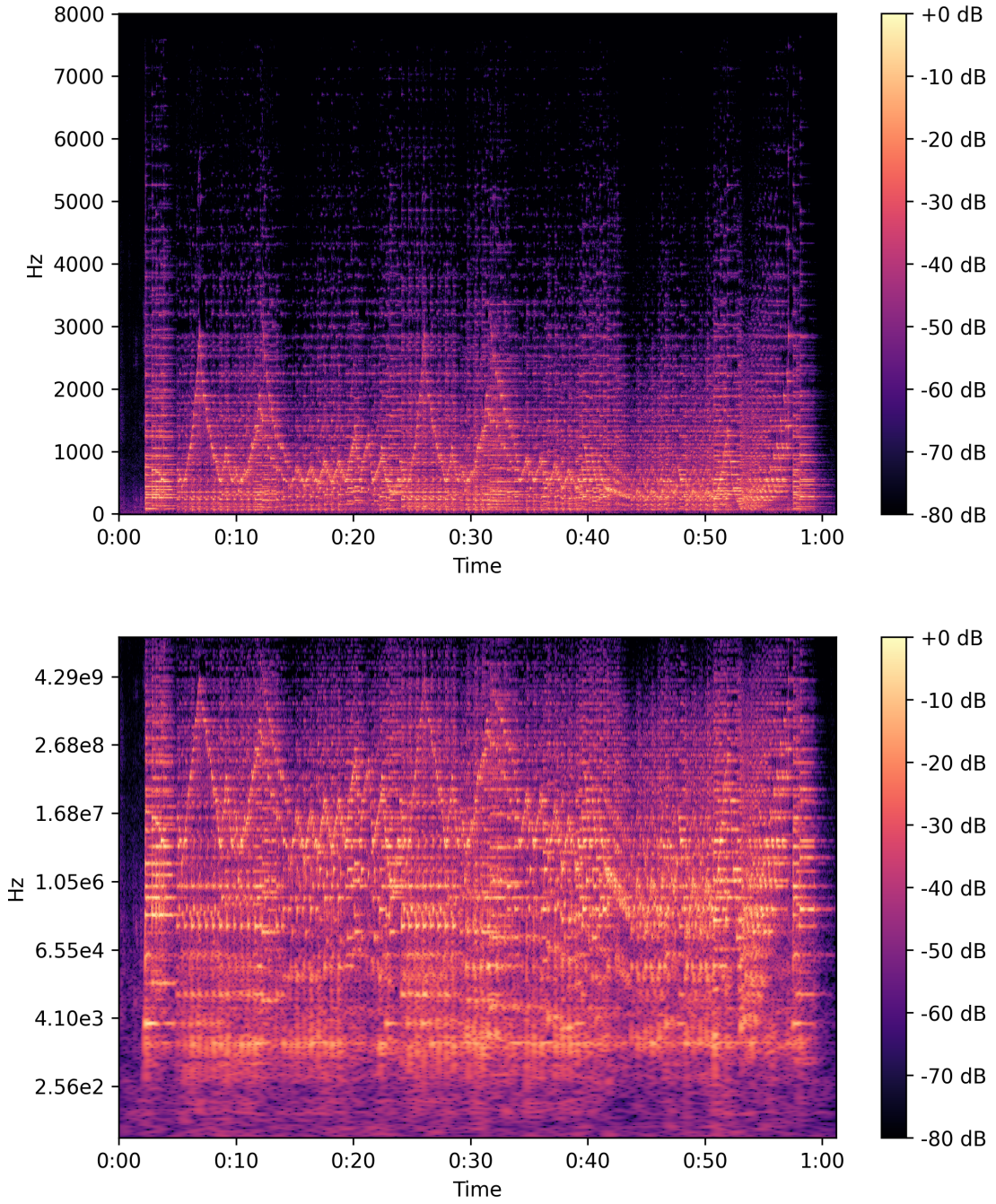


Figura 4.1: Comparación entre un espectrograma STFT con eje de frecuencia lineal y un espectrograma CQT con eje logarítmico, ambos generados a partir del mismo fragmento de audio de piano (*Prelude op. 28 no. 16, Chopin*). En la parte superior se muestra la STFT con escala lineal de frecuencia, donde la energía de los armónicos graves se concentra en muy pocas bandas y las frecuencias agudas aparecen con una resolución excesiva. En la parte inferior se muestra la CQT con escala logarítmica, que asigna un número constante de *bins* por octava y reparte la energía de forma mucho más uniforme a lo largo del teclado, alineando las bandas de frecuencia con las notas discretas del piano. Esta figura ilustra visualmente la motivación principal para emplear la CQT como representación tiempo–frecuencia de entrada en el modelo MFT-FiLM, ya que proporciona una base más adecuada para el aprendizaje de patrones armónicos y la posterior estimación de activaciones de nota.

4.3 Arquitectura general

El modelo MFT-FiLM aborda la transcripción automática de piano como un problema de aprendizaje multitarea con cuatro salidas simultáneas: la activación de teclas a nivel de *frame*, los *onsets*, los *offsets* y la velocidad de las notas [13]. La Figura 4.2 muestra el esquema general del sistema, que sirve de referencia para la descripción del flujo de información que se desarrolla en esta sección.

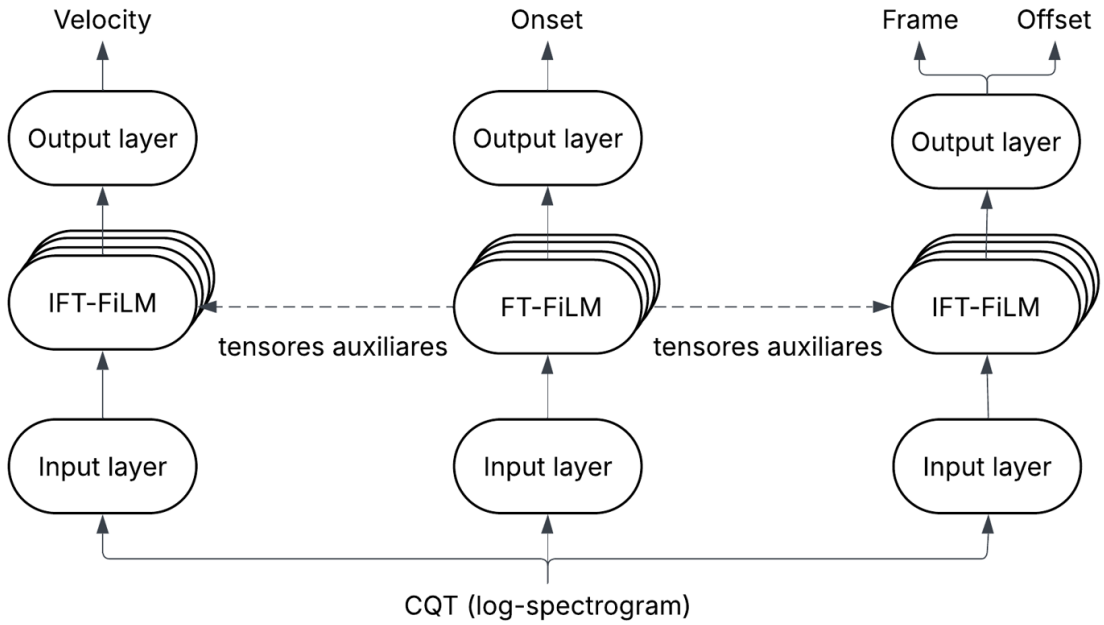


Figura 4.2: Esquema general del modelo MFT-FiLM para la transcripción multitarea de notas de piano, con las tres ramas de *onset*, *velocity* y *frame+offset*, y el flujo de información a través de los bloques FT-FiLM e IFT-FiLM. Ilustración adaptada de [13].

La entrada común a las tres ramas es el espectrograma CQT descrito en la Sección 4.2, con dimensiones (C_0, T, F) donde $C_0 = 4$, T es el número de *frames* y $F = 88$ corresponde a las teclas del piano. Como se aprecia en la parte inferior de la Figura 4.2, este tensor se distribuye de forma idéntica a las tres ramas a través de sus respectivas **capas de entrada** (*Input layer*), que proyectan los $C_0 = 4$ canales al espacio de características interno de cada rama mediante una transformación lineal.

El sistema se organiza en **tres ramas paralelas**, visibles en la Figura 4.2: la rama central de *onset*, la rama izquierda de *velocity* y la rama derecha de *frame+offset*. Cada rama apila **cuatro bloques de modulación** en cascada, representados como bloques apilados en el diagrama. El tipo de bloque varía según la rama: la rama *onset* emplea bloques **FT-FiLM**, mientras que las ramas *velocity* y *frame+offset* emplean bloques **IFT-FiLM**. Las diferencias funcionales entre ambos tipos de bloque se detallan en la Sección 4.4.

Las dimensiones de los canales internos son $C = 64$ en las ramas *onset* y *velocity*, y $C = 32$ en la rama *frame+offset* [13]. Esta asimetría deliberada permite reducir el número total de parámetros sin penalizar la capacidad expresiva de las ramas más discriminativas.

La interacción entre ramas se produce mediante **tensores auxiliares**, representados en la Figura 4.2 mediante flechas discontinuas. En cada uno de los cuatro niveles de procesamiento, los tensores intermedios generados por los bloques FT-FiLM de la rama *onset* se reenvían simultáneamente a los bloques IFT-FiLM de las ramas *velocity* y *frame+offset*. Este flujo es **estrictamente unidireccional**: la información viaja desde la rama *onset* hacia

las otras dos, pero no en sentido contrario. Esta decisión de diseño se justifica porque la señal de *frame* combina activaciones de notas sostenidas y de finales de nota, lo que podría interferir negativamente en la detección precisa de inicios si se empleara como entrada de la rama *onset* [13]. El resultado es un esquema con **cuatro niveles de interacción cruzada**, desde representaciones espectrales de bajo nivel hasta semánticas de alto nivel.

Cada rama dispone de una **capa de salida** (*Output layer*), visible en la parte superior de la Figura 4.2, que adapta las representaciones internas al objetivo de su subtarea. Las salidas del sistema son tres *piano rolls* de probabilidad: la rama *onset* produce un mapa de dimensiones $T \times 88$ con activación sigmoide; la rama *frame+offset* produce dos mapas de igual dimensión para la activación de trama y el final de nota, también con activación sigmoide; y la rama *velocity* produce un mapa de $T \times 88 \times 128$ valores sobre las 128 clases de velocidad MIDI, sin activación final. En conjunto, estas tres salidas permiten reconstruir una transcripción completa en formato MIDI con tono, duración y dinámica de cada nota. El diseño interno de las capas de salida se describe junto con el resto de componentes en la Sección 4.4.

Los *piano rolls* de probabilidad producidos por el modelo no constituyen por sí mismos una transcripción musical: son mapas densos de activaciones continuas sobre los que es necesario aplicar una etapa de postprocesado para obtener eventos de nota discretos con tono, inicio, duración y velocidad.

Durante la inferencia se aplica la estrategia de **medio paso** (*half stride*) [15], que procesa el audio en ventanas con un solapamiento del 50 %. Este solapamiento permite promediar las predicciones en las regiones de transición entre ventanas, mitigando los artefactos en bordes que aparecen al segmentar la señal.

Finalmente, se aplica el **algoritmo de decodificación heurístico de eventos de nota** [5], que convierte los *piano rolls* umbralizados en una secuencia de eventos discretos.

4.4 Módulos FiLM, FT-FiLM e IFT-FiLM

Esta sección describe el mecanismo de modulación que constituye el núcleo computacional del modelo y que da nombre al sistema. Se presentan en orden creciente de complejidad: primero el mecanismo FiLM base, después el módulo FT-FiLM que lo especializa para la extracción de información espectro-temporal, y finalmente el módulo IFT-FiLM que lo extiende para la interacción entre ramas de distintas tareas.

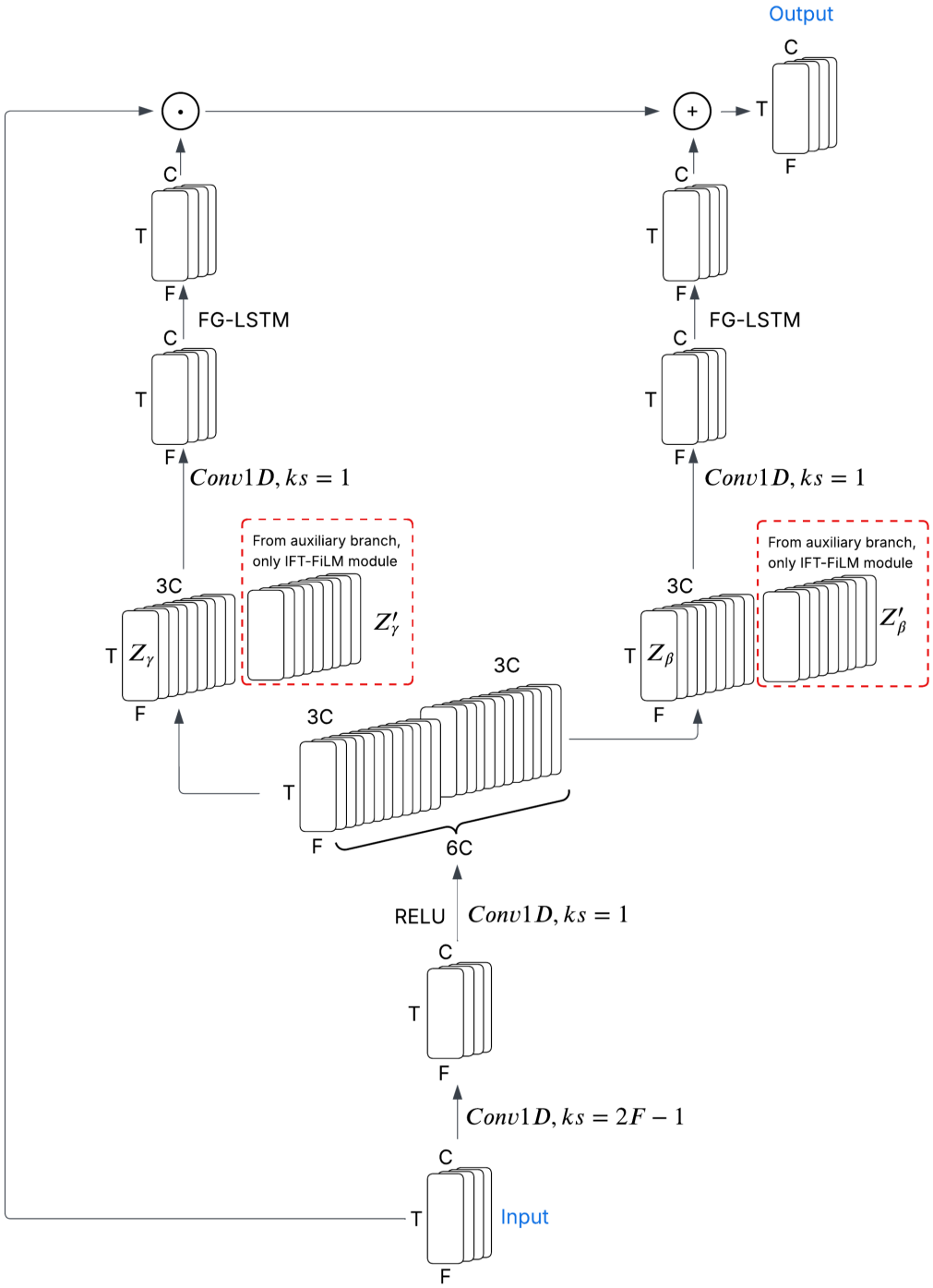


Figura 4.3: Arquitectura detallada de los módulos FT-FiLM e IFT-FiLM. La señal de **entrada** (azul, parte inferior) tiene dimensiones (C, T, F). Una convolución 1D con kernel de tamaño $2F - 1 = 175$ opera sobre el eje de frecuencia; a continuación, una convolución 1D con kernel de tamaño 1 ($k_s = 1$) expande los canales de C a $6C$ y el tensor resultante se divide en dos mitades de $3C$ canales: los parámetros de escala Z_γ y desplazamiento Z_β. En el módulo **IFT-FiLM** (ramas *velocity* y *frame+offset*), los tensores auxiliares Z'_γ y Z'_β procedentes de la rama *onset* (representados con borde discontinuo rojo) se concatenan a los tensores locales, alcanzando $6C$ canales cada uno; en el módulo **FT-FiLM** (rama *onset*) esta concatenación no se produce. Las convoluciones 1D reducen cada tensor de vuelta a C canales, tras lo cual dos **FG-LSTM** bidireccionales independientes de 32 unidades modelan las dependencias temporales, produciendo los parámetros finales γ y β. La transformación afín $\gamma \odot x + \beta$ se aplica sobre la entrada original mediante la conexión de salto (parte superior), generando la **salida** modulada (azul, arriba) de dimensiones (C, T, F). Los tensores auxiliares se desconectan del grafo de cómputo (`detach`) antes de su incorporación para evitar inestabilidades en la optimización. Ilustración adaptada de [13].

El mecanismo *Feature-Wise Linear Modulation* (FiLM) fue propuesto inicialmente por Perez *et al.* (2018) [11] para tareas de razonamiento visual. La idea central consiste en condicionar las activaciones de una red neuronal mediante una transformación afín elemento a elemento, usando información de contexto generada por un módulo externo denominado *FiLM Generator*. Definimos formalmente la operación como:

$$\text{FiLM}(x,\, \gamma,\, \beta) = \gamma \odot x + \beta \tag{4.1}$$

donde x es el mapa de características de entrada, γ y β son los parámetros de escala y desplazamiento generados dinámicamente, y $\odot$ es el producto de Hadamard (multiplicación elemento a elemento). El principal beneficio de este mecanismo respecto a la suma o concatenación directa de características es que γ y β se adaptan en función del contexto en cada paso de inferencia, permitiendo al modelo ponderar y suprimir canales de forma dinámica en lugar de aplicar pesos fijos.

El módulo **FT-FiLM** (*Frequency-Time FiLM*) adapta el mecanismo de la ecuación (4.1) para extraer dependencias espectrales y temporales de forma jerárquica a partir del espectrograma CQT. Es el tipo de bloque empleado en la rama *onset*, que opera exclusivamente sobre su propia representación interna (auto-condicionamiento). Su operación se define como:

$$\text{FT-FiLM}(x) = f_{FT}(x) \odot x + h_{FT}(x) \tag{4.2}$$

donde f_{FT} y h_{FT} son los generadores de los parámetros de modulación γ y β respectivamente. Como se puede seguir en la Figura 4.3, el generador opera en tres etapas secuenciales sobre el mapa de entrada de dimensiones (C, T, F). En primer lugar, el mapa de características se convoluciona con un kernel 1D de tamaño $2F - 1 = 175$ sobre el eje de frecuencia F, seguido de activación ReLU. Este kernel se diseña con una longitud igual al doble del eje completo menos uno para capturar simultáneamente las relaciones armónicas entre todos los *bins* de frecuencia del piano, algo que las convoluciones de kernel reducido no pueden lograr [13].

A continuación, los canales se expanden de C a $6C$ mediante una convolución 1D con kernel de tamaño 1. El tensor resultante se divide en dos mitades de $3C$ canales, denominadas Z_γ y Z_β, que posteriormente generarán los parámetros de escala y desplazamiento de la modulación afín. Sobre cada una de ellas se aplica una convolución 1D con kernel de tamaño 1 que reduce los canales de vuelta a C. Los tensores resultantes se procesan entonces con dos redes recurrentes **FG-LSTM** (*Frequency-Grouped LSTM*) bidireccionales independientes, cada una con 32 unidades ocultas, que capturan las dependencias temporales entre *frames* consecutivos a lo largo del eje T. El uso de LSTM bidireccional incorpora tanto el contexto pasado como el futuro en cada instante de la secuencia, produciendo los tensores γ y β finales del generador.

Tras estas tres etapas, el mapa de entrada original x se modula aplicando la transformación afín de la ecuación (4.2), con una capa de *dropout* de tasa 0,1 antes de la modulación final (visible en la Figura 4.3 como la conexión de salto que llega a los operadores $\odot$ y $+$). El resultado es una representación auto-condicionada que integra información espectral y temporal de forma jerárquica y separada, lo que permite mantener un número de parámetros reducido. El generador FiLM gestiona la información en tres dimensiones individuales (frecuencia, canal y tiempo) de forma desacoplada, en lugar de operarlas conjuntamente como haría un mecanismo de atención estándar.

Con el objetivo de permitir la interacción entre ramas de distintas tareas, el módulo **IFT-FiLM** (*Inter-Task FT-FiLM*) extiende el FT-FiLM incorporando información auxi-

liar procedente de otra rama. Es el tipo de bloque empleado en las ramas *velocity* y *frame+offset*. Su operación se define como:

$$\text{IFT-FiLM}(x,\, x') = f_{IFT}(x,\, x') \odot x + h_{IFT}(x,\, x') \tag{4.3}$$

donde x es el mapa de características de la rama principal y x' es la información auxiliar proveniente de la rama *onset*. La arquitectura del IFT-FiLM es idéntica a la del FT-FiLM salvo en el paso de transformación de canales: como se aprecia en la Figura 4.3 mediante los bloques con borde discontinuo rojo, los tensores Z'_γ y Z'_β generados por la rama auxiliar se concatenan a los tensores locales a lo largo del eje de canales, alcanzando $6C$ canales en cada tensor antes de que la convolución 1D con kernel de tamaño 1 los reduzca de vuelta a C. De este modo, la etapa de FG-LSTM y la modulación afín final operan sobre una representación que ya ha fusionado la información de ambas ramas.

Un detalle de implementación relevante es que los tensores auxiliares Z'_γ y Z'_β se **desconectan del grafo de cómputo** (`detach`) antes de ser incorporados [13]. Esta operación impide que el gradiente fluya de vuelta hacia la rama *onset* a través de esta conexión, evitando inestabilidades en la optimización. El módulo IFT-FiLM puede interpretarse como una red neuronal dinámica: los parámetros de modulación no son fijos, sino que se generan en cada paso en función de las entradas de ambas ramas, permitiendo que la red adapte su comportamiento de acuerdo con el contexto conjunto de las tareas involucradas [13].

4.5 Función de pérdida y eficiencia

La función de pérdida del sistema es la suma de las pérdidas individuales de cada subtarea:

$$\ell = \ell_{\text{onset}} + \ell_{\text{frame}} + \ell_{\text{offset}} + \ell_{\text{velocity}} \tag{4.4}$$

Las pérdidas de *onset*, *frame* y *offset* se calculan mediante **entropía cruzada binaria** (BCE), apropiada para clasificación binaria píxel a píxel del *piano roll*:

$$\ell_{\text{onset}} = \sum_{k=1}^{K} \sum_{t=1}^{T} \ell_{\text{BCE}}(I_{\text{onset}}(t,k),\ P_{\text{onset}}(t,k)) \tag{4.5}$$

y análogamente para ℓ_{frame} y ℓ_{offset}. La pérdida de velocidad se calcula mediante **entropía cruzada categórica** (CCE) sobre 128 clases, dado que la velocidad MIDI es un valor entero en el intervalo $[0, 127]$:

$$\ell_{\text{velocity}} = \sum_{k=1}^{K} \sum_{t=1}^{T} \ell_{\text{CCE}}\big(I_{\text{velocity}}(t,k),\ P_{\text{velocity}}(t,k)\big) \tag{4.6}$$

donde I y P representan los valores de *ground truth* y predicción respectivamente, $K = 88$ es el número de teclas del piano y T el número de *frames* [13]. La ausencia de factores de ponderación entre las cuatro pérdidas es una elección de diseño deliberada por los autores, cuya validez queda respaldada por el estudio de ablación presentado en la sección de resultados [13].

Una de las aportaciones más destacadas del modelo es su reducido tamaño. Con tan solo 1,2 **millones de parámetros**, el MFT-FiLM se sitúa como uno de los modelos de transcripción de piano más eficientes del estado del arte. En términos de operaciones de multiplicación-acumulación (MACs), calculadas para una salida de 128 *frames* con la

biblioteca `ptflops`, el MFT-FiLM registra una complejidad notablemente inferior a la de otros modelos como el hFT-Transformer o el SemiCRF V2 [13, 15, 18].

Esta eficiencia no sacrifica capacidad expresiva gracias al diseño jerárquico del generador FiLM, que desacopla el procesamiento en los ejes de frecuencia, canal y tiempo en lugar de operarlos conjuntamente. La separación de estas dimensiones permite reutilizar módulos ligeros (convoluciones 1D y LSTMs de baja dimensionalidad) en lugar de recurrir a mecanismos de atención de complejidad cuadrática, sin renunciar a la capacidad de capturar dependencias espectrales y temporales de largo alcance.

4.6 Experimentos

Los experimentos de entrenamiento y evaluación del modelo MFT-FiLM se han llevado a cabo en el clúster de cómputo de alto rendimiento `tensor` del DSIC. La descripción detallada del entorno hardware y software, así como de la configuración del experimento, se recoge en el Anexo A.

Antes del entrenamiento, se ejecuta una fase de **preprocesamiento y caché** del *dataset*. Durante esta fase se transforma cada audio del conjunto MAESTRO en su espectrograma CQT mediante la librería `librosa` y los archivos MIDI correspondientes son procesados mediante `pretty_midi` para generar los *piano rolls* de *ground truth* (véase la Sección A.4 del Anexo A para una descripción de estas herramientas).

Adicionalmente, la duración de cada nota en el *ground truth* se extiende en función de la información de los pedales de *sustain* y *sostenuto*, siguiendo el criterio habitual en transcripción automática de piano: una nota se prolonga hasta que el valor del pedal de *sustain* en el archivo MIDI cae por debajo de 64, o hasta que la misma tecla es pulsada de nuevo. Este paso es crucial para la fidelidad del *ground truth*, ya que en interpretaciones de piano con pedal las notas resuenan durante más tiempo del que dura la pulsación física de la tecla; ignorar esta información produciría un *ground truth* sistemáticamente más corto que el audio real.

El resultado de la fase de preprocesamiento es un conjunto de archivos binarios en disco que almacenan, para cada fragmento de audio, el espectrograma CQT y los correspondientes mapas de activación de nota. Estos archivos de caché se reutilizan directamente en todos los entrenamientos subsiguientes, de modo que el costoso cálculo de las representaciones tiempo–frecuencia y de los *piano rolls* solo se realiza una vez.

A continuación se analizan los resultados obtenidos por el modelo MFT-FiLM entrenado sobre MAESTRO v3.0.0. Las métricas empleadas coinciden con el protocolo estándar descrito en la Sección 3.9: se evalúa el rendimiento a nivel de trama y a nivel de nota mediante precisión, *recall* y puntuación F_1, tanto para la activación de teclas (*Frame*) como para las variantes *Note Onset*, *Note w/ Offset* y *Note w/ Offset & Velocity* [13, 18].

Para garantizar la comparabilidad directa con los trabajos originales, se utilizan los mismos umbrales que en el repositorio de referencia para las predicciones de MFT-FiLM. En todas las evaluaciones se fijan los umbrales de decisión a $t_{\text{onset}} = 0{,}4$, $t_{\text{frame}} = 0{,}48$ y $t_{\text{offset}} = 0{,}5$ [13].

Las Tablas 4.1, 4.2 y 4.3 muestran una comparación cuantitativa de los resultados obtenidos sobre MAESTRO v3.0.0, incluyendo las métricas de F_1, precisión y *recall*, respectivamente. Se compara el modelo entrenado con otros modelos SOTA.

Tabla 4.1: Resultados de transcripción de notas sobre MAESTRO v3.0.0. Se muestran el número de parámetros y las puntuaciones F_1 a nivel de trama (*Frame*) y de nota (*Note Onset*, *Note w/ Offset*, *Note w/ Offset & Velocity*). Todas las métricas se expresan en porcentaje (%).

Modelo	Parámetros	Frame F_1	Onset F_1	Onset+Off. F_1	Onset+Off.+Vel. F_1
Transformer [14]	54 M	—	96.01	83.94	82.75
SemiCRF [17]	9 M	90.75	96.11	88.42	87.44
HPPNet [16]	1.2 M	93.15	97.18	83.80	82.24
hFT-Transformer [15]	5.5 M	93.24	97.44	90.53	89.48
HATANet [12]	6.4 M	94.04	97.81	90.57	89.25
SemiCRF V2 [18]	12.9 M	95.35	98.32	93.48	92.94
MFT-FiLM (autores) [13]	1.2 M	94.75	98.39	92.46	91.79
MFT-FiLM (este trabajo)	1.2 M	94.36	98.17	91.60	90.78

En la Tabla 4.1 se observa que el modelo MFT-FiLM entrenado alcanza unas puntuaciones F_1 muy próximas a las reportadas por los autores originales en MAESTRO v3.0.0 [13]. A pesar de haber completado únicamente 30 de las 60 épocas de entrenamiento descritas en el artículo, el F_1 de trama se mantiene por encima del 94 %, mientras que las métricas de nota alcanzan valores superiores al 98 % en *Note Onset* y en torno al 91–91,5 % en *Note w/ Offset* y *Note w/ Offset & Velocity*. Estas diferencias de unas pocas décimas respecto a los resultados publicados pueden atribuirse tanto a la reducción del número de épocas como a las pequeñas divergencias en el entorno de cómputo (ver Anexo A).

Tabla 4.2: Resultados de precisión (P) de transcripción de notas sobre MAESTRO v3.0.0. Se muestran el número de parámetros y las métricas de precisión a nivel de trama (*Frame*) y de nota (*Note Onset*, *Note w/ Offset*, *Note w/ Offset & Velocity*). Todas las métricas se expresan en porcentaje (%).

Modelo	Parámetros	Frame P	Onset P	Onset+Off. P	Onset+Off.+Vel. P
Transformer [14]	54 M	—	98.61	86.19	84.95
SemiCRF [17]	9 M	93.79	98.69	90.79	89.78
HPPNet [16]	1.2 M	92.79	98.45	84.88	83.29
hFT-Transformer [15]	5.5 M	92.82	99.64	92.52	91.43
HATANet [12]	6.4 M	95.51	99.05	91.70	90.35
SemiCRF V2 [18]	12.9 M	95.75	99.53	94.61	94.07
MFT-FiLM (autores) [13]	1.2 M	95.77	99.33	93.33	92.65
MFT-FiLM (este trabajo)	1.2 M	95.27	99.37	92.70	91.86

Tabla 4.3: Resultados del *recall* (R) en transcripción de notas sobre MAESTRO v3.0.0. Se muestran el número de parámetros y las métricas de *recall* a nivel de trama (*Frame*) y de nota (*Note Onset*, *Note w/ Offset*, *Note w/ Offset & Velocity*). Todas las métricas se expresan en porcentaje (%).

Modelo	Parámetros	Frame R	Onset R	Onset+Off. R	Onset+Off.+Vel. R
Transformer [14]	54 M	—	93.60	81.86	80.70
SemiCRF [17]	9 M	88.36	93.96	86.46	85.51
HPPNet [16]	1.2 M	93.59	95.95	82.76	81.24
hFT-Transformer [15]	5.5 M	93.66	95.44	88.69	87.67
HATANet [12]	6.4 M	92.62	96.61	89.47	88.17
SemiCRF V2 [18]	12.9 M	95.01	97.16	92.39	91.87
MFT-FiLM (autores) [13]	1.2 M	93.84	97.48	91.62	90.95
MFT-FiLM (este trabajo)	1.2 M	93.56	97.03	90.54	89.73

Las Tablas 4.2 y 4.3 descomponen el F_1 en términos de precisión y *recall*. En la reproducción se observa una precisión especialmente alta en la detección de inicios de nota, cercana al 99 %, apenas introduce notas espurias que no existen en la partitura de referencia. El *recall* en onsets se sitúa ligeramente por debajo, en torno al 97 %, lo que sugiere

que los pocos errores restantes se deben principalmente a notas omitidas en pasajes complejos más que a activaciones falsas.

En las métricas que incluyen el *offset* y la velocidad, tanto la precisión como el *recall* descienden de forma moderada pero se mantienen en torno al 90 %, en línea con los resultados de MFT-FiLM reportados por Wang *et al.* [13] y con los mejores modelos del estado del arte. Esto refleja la mayor dificultad de estimar con precisión el final de las notas y su dinámica en presencia del pedal de *sustain*, pero confirma que la arquitectura MFT-FiLM es capaz de modelar de forma consistente la articulación temporal y la intensidad de la interpretación, incluso cuando el entrenamiento se detiene a mitad de las épocas previstas.

Finalmente, la comparación horizontal de las tres tablas evidencia que, con solo 1,2 millones de parámetros, la variante reproducida de MFT-FiLM ofrece un rendimiento muy cercano al de modelos considerablemente más pesados, como hFT-Transformer o las variantes SemiCRF. Desde la perspectiva de la relación entre eficiencia y precisión, este comportamiento respalda la tesis de los autores de que la modulación lineal frecuencia–tiempo permite explotar de forma eficaz la información compartida entre tareas sin necesidad de incrementar el tamaño del modelo.

De forma complementaria a la evaluación del rendimiento final sobre el conjunto de prueba, el análisis de las métricas durante el proceso de entrenamiento resulta fundamental para comprender el comportamiento analítico y la eficiencia de aprendizaje de la arquitectura MFT-FiLM. Con este propósito, se ha monitorizado la evolución de la función de pérdida (*loss*) y de las métricas de validación a lo largo de las 30 épocas completadas. En primer lugar se analiza la dinámica de la pérdida dentro de la primera época de entrenamiento (Figura 4.4), que proporciona información detallada sobre la fase de adaptación inicial del modelo.

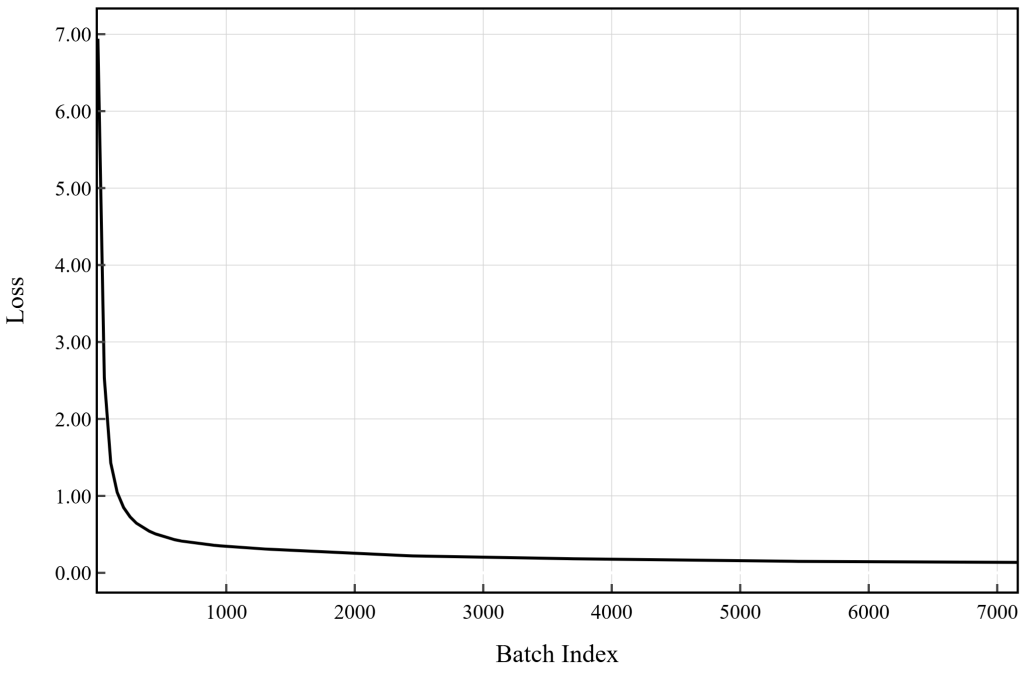


Figura 4.4: Evolución de la función de pérdida lote a lote durante la primera época de entrenamiento del modelo MFT-FiLM. Se aprecia un descenso abrupto en las primeras iteraciones seguido de una estabilización progresiva, lo que indica que el mecanismo FiLM facilita una adaptación inicial muy rápida a las representaciones espectro-temporales de la señal de audio.

La Figura 4.4 muestra que el modelo exhibe una convergencia inicial excepcionalmente rápida. Durante las primeras iteraciones de la primera época, la función de pérdida experimenta un descenso abrupto seguido de una estabilización progresiva, lo que sugiere que el mecanismo FiLM facilita una adaptación veloz a las representaciones espectrotemporales de la señal de audio. Este patrón es consistente con un régimen de aprendizaje en el que los parámetros del modelo se ajustan con rapidez a la estructura básica de los datos, mientras que las etapas posteriores de la época se dedican a refinar gradualmente las representaciones internas sin oscilaciones ni comportamientos inestables.

A continuación se analiza la evolución macroscópica de la pérdida de entrenamiento y validación a lo largo de las 30 épocas completadas (Figura 4.5), con el fin de valorar la estabilidad global del proceso de optimización y la posible aparición de sobreajuste.

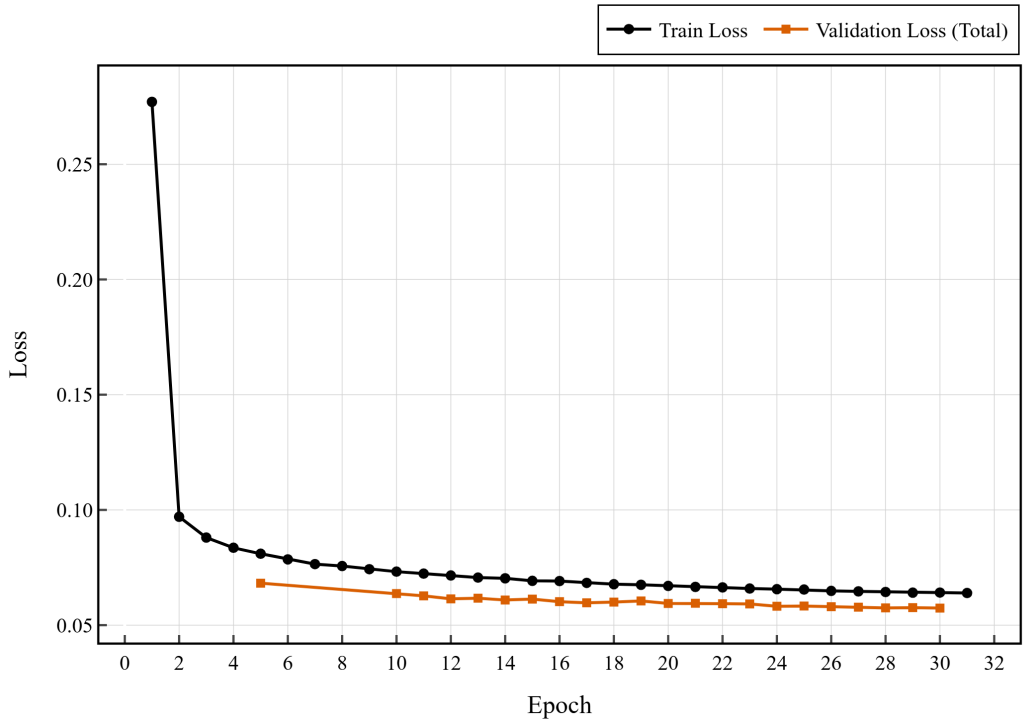


Figura 4.5: Pérdida promedio de entrenamiento frente a pérdida de validación a lo largo de las 30 épocas completadas (calculada como la suma de los errores de las cuatro subtareas: *onset*, *frame*, *offset* y *velocity*). La pérdida de entrenamiento decrece de forma asintótica de 0,277 en la época 1 a 0,063 en la época 30, y la pérdida de validación acompaña esta tendencia sin divergir, sin mostrar indicios claros de sobreajuste en esta etapa del entrenamiento.

Al analizar la Figura 4.5, se observa que la pérdida de entrenamiento continúa decreciendo de forma monótona y asintótica, reduciéndose de 0,277 en la época 1 a 0,063 en la época 30. La curva de validación acompaña esta trayectoria con valores ligeramente superiores pero paralelos, sin presentar la separación creciente característica del sobreajuste. Este comportamiento indica que, en el rango de épocas ejecutado, el modelo sigue mejorando su capacidad de generalización y no ha agotado aún su potencial de aprendizaje; de hecho, cabe esperar una ligera mejora adicional de las métricas si el entrenamiento se prolongara hasta las 60 épocas planificadas en el trabajo original.

Continuamos analizando métricas en validación. La Figura 4.6 desglosa la evolución de la métrica F_1 sobre el conjunto de validación para las cuatro subtareas evaluadas.

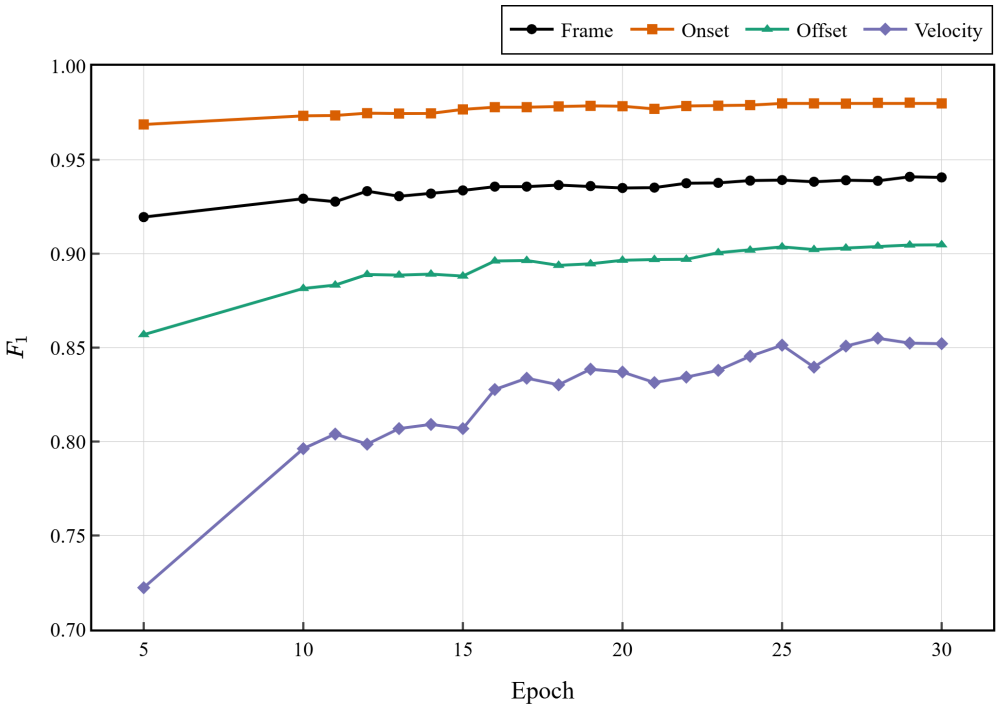


Figura 4.6: Evolución de la métrica F_1 en el conjunto de validación para la estimación de *Frame*, *Onset*, *Offset* y *Velocity* durante las 30 épocas de entrenamiento. Se aprecia una convergencia rápida y estable en las tareas de *Frame*, *Onset* y *Offset*, mientras que la tarea de *Velocity* presenta una mejora más gradual asociada a la mayor complejidad de modelar la dinámica de la interpretación.

En la Figura 4.6 se observa una estructura jerárquica de aprendizaje bien definida: la detección de *Note Onset* alcanza valores de convergencia superiores al 97 % de forma muy temprana (época 5). En contraste, la estimación de la velocidad (*Velocity*), al constituir un problema de regresión dependiente de la intensidad dinámica de la pulsación, requiere un proceso de optimización más prolongado, evolucionando desde un 72,22 % inicial hasta consolidarse por encima del 85 % hacia la culminación del entrenamiento. Esta diferencia de ritmos de convergencia es coherente con la dificultad relativa de cada subtarea y confirma que la arquitectura es capaz de capturar primero la estructura temporal de los eventos de nota y, en una segunda fase, refinar la estimación de su dinámica.

Para analizar rigurosamente la naturaleza de las predicciones del modelo, resulta necesario descomponer esta métrica en sus constituyentes básicos. La Figura 4.7 presenta en paralelo los niveles de precisión y *recall* obtenidos durante la fase de validación.

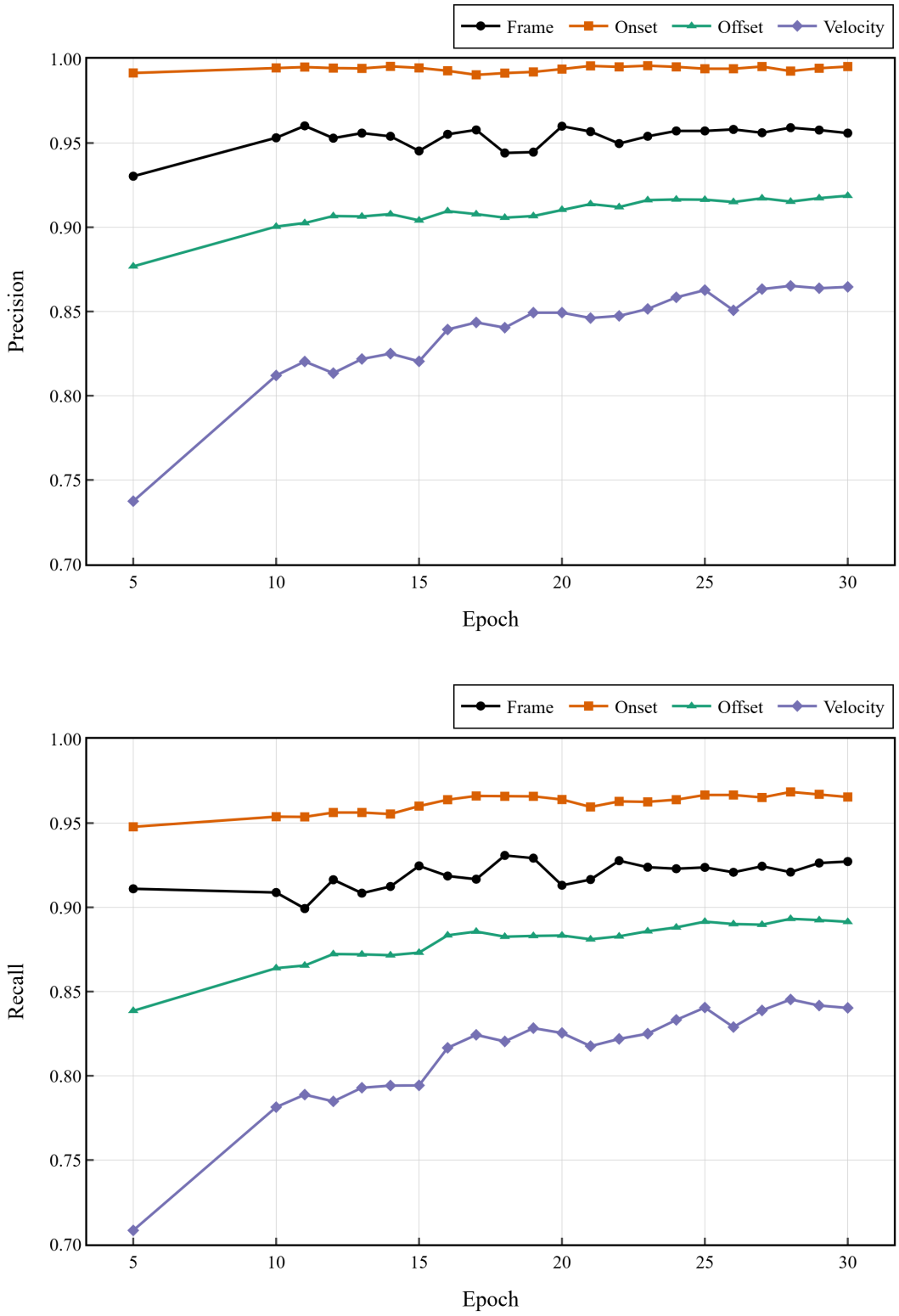


Figura 4.7: Evolución de la precisión (arriba) y del *recall* (abajo) en el conjunto de validación a lo largo de las 30 épocas de entrenamiento, desglosadas por subtarea: *Frame*, *Onset*, *Offset* y *Velocity*. La precisión se mantiene de forma sistemática por encima del *recall* en todas las tareas y épocas, reflejando un comportamiento conservador del modelo: ante la incertidumbre, el sistema prefiere omitir una nota a activar una inexistente. En la época 30, la precisión de *Onset* alcanza valores cuasi perfectos ($>$ 99,5 %), mientras que el *recall* de la misma subtarea se estabiliza en torno al 96,5 %. Esta divergencia indica que los errores residuales provienen principalmente de notas omitidas en pasajes de alta densidad polifónica, y no de activaciones falsas.

El contraste analítico entre ambas gráficas de la Figura 4.7 evidencia que el modelo MFT-FiLM presenta un comportamiento marcadamente **conservador**. A lo largo de todas las épocas y tareas evaluadas, los niveles de precisión se mantienen de forma sistemática por encima de los niveles de *recall*. A modo de ejemplo, en la época 30, la precisión para la detección de *Note Onset* alcanza valores cuasi perfectos ($> 99{,}5\,\%$), mientras que la exhaustividad (*recall*) se estabiliza en torno al 96,5 %.

Estadísticamente, esta divergencia indica que el modelo penaliza con mayor severidad los falsos positivos frente a los falsos negativos. En el contexto específico de la Transcripción Automática de Música (AMT), esto se traduce en que, ante la predicción de activación de una nota, existe una elevada probabilidad de que dicha nota esté efectivamente presente en la interpretación original. En consecuencia, los errores residuales del sistema provienen mayoritariamente de la omisión de notas (*missed notes*), fenómeno particularmente predecible en segmentos de alta densidad polifónica o ante pulsaciones de baja amplitud acústica enmascaradas por el uso del pedal de *sustain*.

Desde una perspectiva de percepción musical, este sesgo algorítmico resulta altamente favorable. En la reconstrucción auditiva de una partitura transcrita (por ejemplo, mediante la síntesis de los archivos MIDI resultantes), la inserción de notas incorrectas o inexistentes (falsos positivos) genera un impacto acústico muy disruptivo y disonante para el oyente. Por el contrario, la omisión puntual de una nota interna dentro de un clúster armónico complejo (falso negativo) tiende a resultar prácticamente imperceptible, garantizando así la preservación de la coherencia tonal y rítmica de la obra.

4.7 Conclusiones

El modelo MFT-FiLM abordado en este capítulo propone una arquitectura de transcripción multitarea de piano basada en la Transformada Q Constante como representación de entrada, una organización en ramas paralelas para *frame*, *onset*, *offset* y *velocity*, y un mecanismo de modulación FiLM especializado en los módulos FT-FiLM e IFT-FiLM. Este diseño permite desacoplar el procesamiento en los ejes de frecuencia, canal y tiempo, explotando de forma eficiente las dependencias espectrales y temporales sin recurrir a mecanismos de atención de alta complejidad [13].

Desde el punto de vista cuantitativo, la reproducción del experimento sobre MAESTRO v3.0.0 alcanza puntuaciones F_1, de precisión y de *recall* muy próximas a las reportadas por los autores originales, a pesar de que el entrenamiento solo completó 30 de las 60 épocas previstas. El modelo mantiene un F_1 de trama superior al 94 % y métricas de nota en torno al 91–91,5 % para las variantes más exigentes (*Note w/ Offset* y *Note w/ Offset & Velocity*), lo que sitúa a MFT-FiLM como una alternativa competitiva frente a modelos considerablemente más pesados, con únicamente 1,2 millones de parámetros [13, 18].

En cuanto al proceso de optimización, la evolución de la pérdida de entrenamiento y validación muestra una convergencia estable y sin indicios de sobreajuste en las 30 épocas completadas, con una reducción asintótica de la pérdida total y un acompañamiento estrecho por parte de la pérdida de validación. De forma complementaria, el análisis de las métricas en validación revela un comportamiento sistemáticamente conservador del modelo, con niveles de precisión que superan de forma consistente al *recall* en todas las subtareas, lo que indica que MFT-FiLM prioriza evitar falsas activaciones frente a la recuperación exhaustiva de todas las notas.

Este sesgo conservador resulta especialmente favorable desde una perspectiva perceptual: en la síntesis de las transcripciones resultantes, la ausencia ocasional de notas internas dentro de acordes densos tiende a ser poco perceptible, mientras que la introduc-

ción de notas espurias genera artefactos disonantes claramente audibles. En conjunto, los resultados numéricos y el comportamiento analítico del entrenamiento confirman que la arquitectura MFT-FiLM constituye una solución eficiente y robusta para la transcripción automática de piano, capaz de aproximarse al estado del arte con un coste computacional significativamente reducido.

CAPÍTULO 5
Transkun

5.1 Introducción

La transcripción automática de música de piano puede abordarse desde múltiples perspectivas metodológicas. El enfoque más extendido en la literatura consiste en trabajar a nivel de *frame* (como el MFT-FiLM estudiado en el capítulo anterior): el sistema produce, para cada instante de la señal de audio, una estimación de la activación de cada tecla del piano, del inicio y del final de cada nota (*onset* y *offset*), así como de la velocidad de pulsación (*velocity*) [5, 10]. Estas estimaciones se obtienen de forma independiente para cada subtarea y, a continuación, se ensamblan mediante algoritmos heurísticos de post-procesado (umbralización, búsqueda de picos, modelos de Markov de bajo orden) para recuperar los eventos de nota como intervalos discretos [20]. Por otro lado, hay modelos que tratan de aprender directamente (saltando el *frame level*) qué intervalos de tiempo corresponden a eventos musicales completos.

En este proyecto, la segunda alternativa explorada es **Transkun** (*Transcription Using Neural semi-CRFs*), propuesto por Yan y Duan [18]. La idea central consiste en tratar cada nota como un intervalo cerrado $[i, j]$ sobre el eje temporal, y asignar a cada candidato de intervalo una puntuación escalar que refleje su probabilidad de corresponder a un evento real del tipo de evento dado [18].

En este capítulo, la Sección 5.2 ofrece una visión general de la arquitectura del sistema. La Sección 5.3 describe el espectrograma de entrada, el submuestreo convolucional y los *event tracks*. A continuación, la Sección 5.4 detalla el *transformer* encoder no jerárquico. La Sección 5.5 presenta el *framework* semi-CRF y la formulación del *scoring* por producto interior escalado. La Sección 5.6 detalla la predicción de atributos por evento y la estrategia de procesado por segmentos. Después, la Sección 5.7 describe el preprocesamiento del *dataset*, el entrenamiento en el clúster y presenta y analiza los resultados obtenidos, comparando el rendimiento con el estado del arte y con el modelo MFT-FiLM. Finalmente, la Sección 5.8 recoge las conclusiones del capítulo.

5.2 Arquitectura general

La arquitectura de Transkun transforma el espectrograma log-mel de la señal de audio en un conjunto de intervalos temporales no solapados, uno por tipo de evento, en una única etapa de extremo a extremo. La Figura 5.1 ofrece un esquema general de la arquitectura que servirá de referencia para las explicaciones sobre este modelo.

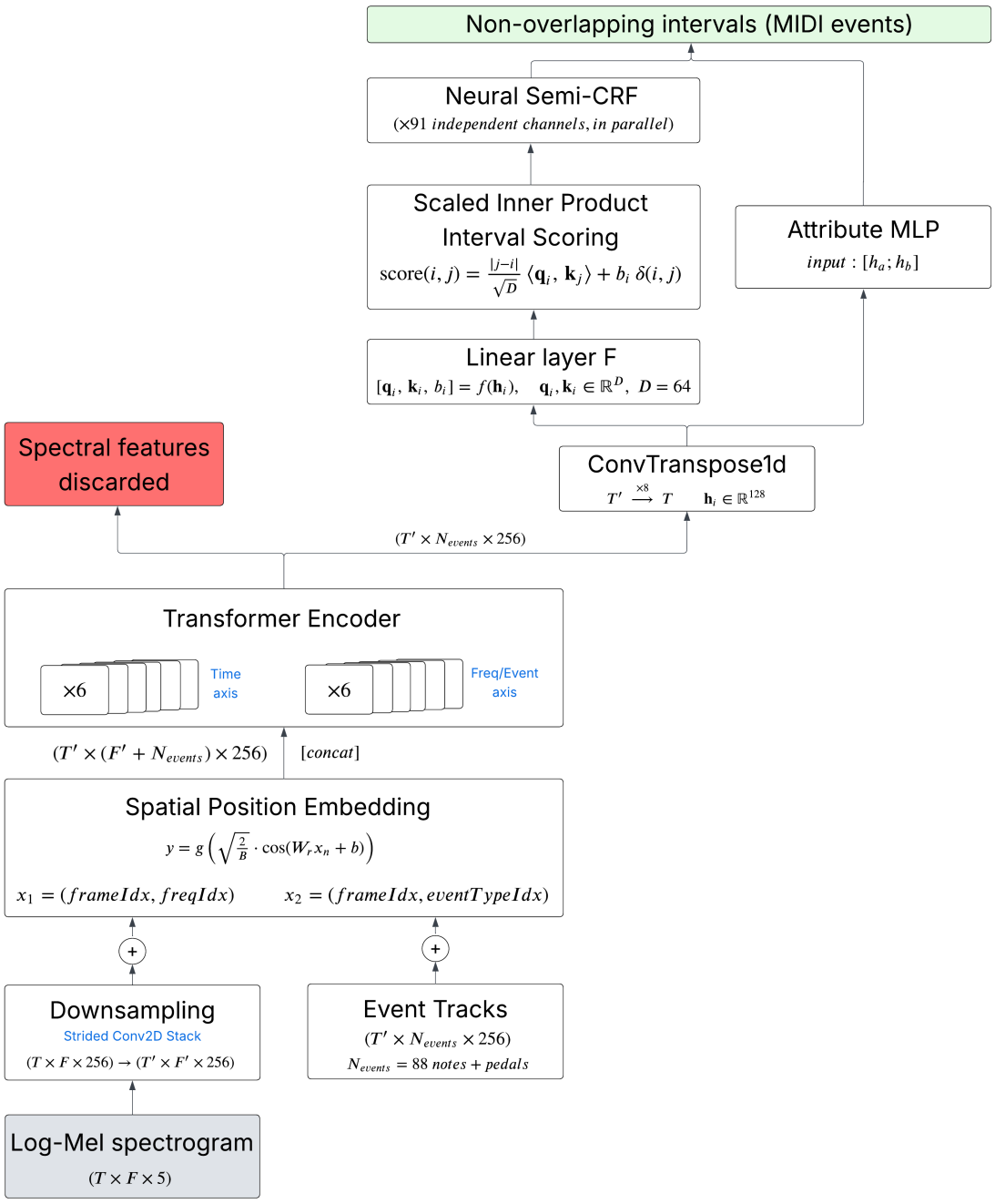


Figura 5.1: Esquema general de la arquitectura de Transkun. El espectrograma log-mel $(T \times F \times 5)$ se submuestrea mediante cuatro capas convolucionales *strided* hasta $(T' \times F' \times 256)$, al que se suma un *embedding* de posición espacial. Los *event tracks* $(T' \times N_{\text{events}} \times 256)$, inicializados también con *embedding* de posición (independiente), se concatenan a las representaciones del espectrograma y se procesan conjuntamente por el *transformer encoder* ($\times L$=6 capas). Tras el *encoder*, únicamente los *event tracks* se retienen y se remuestrean temporalmente mediante `ConvTranspose1d` hasta la resolución original T. Los *embeddings* $\mathbf{h}_i \in \mathbb{R}^{128}$ resultantes se usan en dos ramas paralelas: el módulo de *scoring* por producto interior escalado (que alimenta la capa semi-CRF para producir los intervalos no solapados) y el Attribute MLP (que predice velocidad, posiciones subframe e indicadores de segmento).

En la Figura 5.1 se pueden ver las cinco etapas funcionales que componen el modelo:

1. El cálculo del espectrograma de entrada con sobremuestreo de ventanas.
2. El submuestreo convolucional y la adición de *embeddings* de posición espacial, en paralelo con la inicialización de los *event tracks*.
3. El procesado conjunto mediante un *transformer encoder* no jerárquico.
4. El remuestreo temporal de los *event tracks* y el *scoring* por producto interior.
5. La decodificación mediante la capa semi-CRF y la predicción de atributos.

Las siguientes secciones desarrollan estas etapas en detalle.

5.3 Espectrograma de entrada, submuestreo y *event tracks*

Esta sección describe las tres primeras etapas del sistema, visibles en la mitad inferior de la Figura 5.1: el cálculo del espectrograma de entrada, el submuestreo convolucional con adición de embeddings de posición espacial, y la inicialización de los *event tracks*.

La señal de audio de entrada se muestrea a 44 100 Hz y se transforma mediante la Transformada de Fourier de Tiempo Corto (STFT) con ventana de *Hanning*, tamaño de ventana de 4096 muestras y un salto de trama (*hop size*) de 1024 muestras ($\approx$ 23,2 ms por trama). Se utilizan 229 bandas mel en el rango de 30 a 8000 Hz [18]. Aunque la etapa de preprocesamiento sigue una estructura análoga a la de MFT-FiLM (véase la Sección 4.3), Transkun no emplea la CQT sino el espectrograma log-mel, ya que opera a 44 100 Hz y su *backbone transformer* no requiere la estructura de octavas de la CQT.

Para mejorar la resolución temporal de la representación espectral sin reducir el salto de trama, se aplica una estrategia de **sobremuestreo de ventanas** (*multi-window oversampling*) heredada del semi-CRF original [17]: en lugar de una única ventana de *Hanning* por trama, se aplican **cinco subventanas de Hanning ajustables** (inicializadas con centros equidistantes y varianza constante) sobre cada trama de audio, y sus espectros log-mel resultantes se concatenan a lo largo del eje de canales. El resultado es una entrada de cinco canales espectrales por *bin* tiempo-frecuencia, con dimensiones $(T \times F \times 5)$, tal como indica la Figura 5.1 en la base del diagrama. Cada segmento de audio procesado tiene una duración de 16 segundos [18].

A diferencia de los sistemas SOTA previos (incluyendo el semi-CRF original [17], hFT-Transformer [15] y Onsets & Frames [5]), que preservan la resolución temporal completa de la entrada a lo largo de toda la red, Transkun adopta un diseño de **mapa de características de baja resolución**, análogo al empleado en los *Vision Transformers* (ViT) [19]. Esta elección de diseño se justifica en que la alta dimensionalidad de los embeddings permite mantener suficiente capacidad expresiva incluso con resolución temporal reducida, al tiempo que el coste computacional del *transformer* se reduce de forma drástica [18, 24].

El espectrograma de entrada $(T \times F \times 5)$ se submuestrea mediante **cuatro capas convolucionales *strided*** con kernel de tamaño 3 y canales de salida $[128, 256, 256, 256]$ respectivamente (la primera capa actúa como proyección inicial de los 5 canales de entrada). Los *strides* aplicados sobre los ejes de tiempo y frecuencia son $[(2,1), (2,2), (2,2), (1,1)]$, lo que produce un factor de submuestreo de $c_T = 8$ en tiempo y de $c_F = 4$ en frecuencia. El resultado es un mapa de características de baja resolución de dimensiones:

$$(T', F') = \left(\frac{T}{8}, \frac{F}{4}\right) \tag{5.1}$$

equivalente a parches de tamaño 8×4 sobre el espectrograma original [18]. Cada capa convolucional va seguida de una normalización **GroupNorm** con 4 grupos y una activación GELU, a excepción de la última capa. El espectrograma submuestreado tiene así dimensiones $(T' \times F' \times 256)$.

Tras el submuestreo, se añaden ***embeddings* de posición espacial aprendibles** basados en características de Fourier (*learnable Fourier features*) [23]. Este *embedding* transforma un vector de coordenadas $\boldsymbol{x} \in \mathbb{R}^C$ en un vector de *embedding* $\boldsymbol{y} \in \mathbb{R}^E$ de acuerdo con:

$$\boldsymbol{y} = g\left(\sqrt{\tfrac{2}{B}}\cos(\boldsymbol{W}_r\,\boldsymbol{x} + \boldsymbol{b})\right) \tag{5.2}$$

donde $\boldsymbol{W}_r \in \mathbb{R}^{B\times C}$ es una matriz aprendible inicializada desde $\mathcal{N}(0,\, \gamma^{-2})$; $\boldsymbol{b} \in \mathbb{R}^B$ es un sesgo aprendible inicializado desde $\mathcal{U}(-\pi,\, +\pi)$; B es la dimensión de las características de Fourier ($|B| = 256$); γ es un hiperparámetro de escala ($\gamma = 1$); y $g : \mathbb{R}^B \to \mathbb{R}^E$ es un perceptrón de dos capas con tamaño oculto 1024 [18]. Este *embedding* funciona como una red MLP que toma coordenadas espaciales como entrada, con la primera no linealidad siendo una función coseno escalada. Se elige por su sencillez y amplia compatibilidad con arquitecturas *transformer*.

El embedding se aplica de forma **independiente** a las representaciones del espectrograma submuestreado (con coordenadas $(\mathit{frameIdx}, \mathit{freqIdx})$) y a los *event tracks* (con coordenadas $(\mathit{frameIdx}, \mathit{eventTypeIdx})$). En ambos casos la operación es una **suma elemento a elemento** ($\oplus$) que enriquece las representaciones con información posicional sin modificar sus dimensiones.

Para obtener las representaciones necesarias para el *scoring* de intervalos, Transkun introduce el concepto de ***event track***: un tensor de parámetros aprendibles de dimensiones $(T' \times N_{\text{events}} \times 256)$, donde $N_{\text{events}} = 91$ (88 notas más 3 pedales), que actúa como representación aprendible de cada tipo de evento a lo largo del tiempo en la resolución reducida [18]. Esta idea es análoga a la de los tokens de detección (*detection tokens*) del modelo YOLOS [29], donde tokens aprendibles se añaden a las representaciones de imagen para que el *transformer* los especialice en tareas de predicción densa.

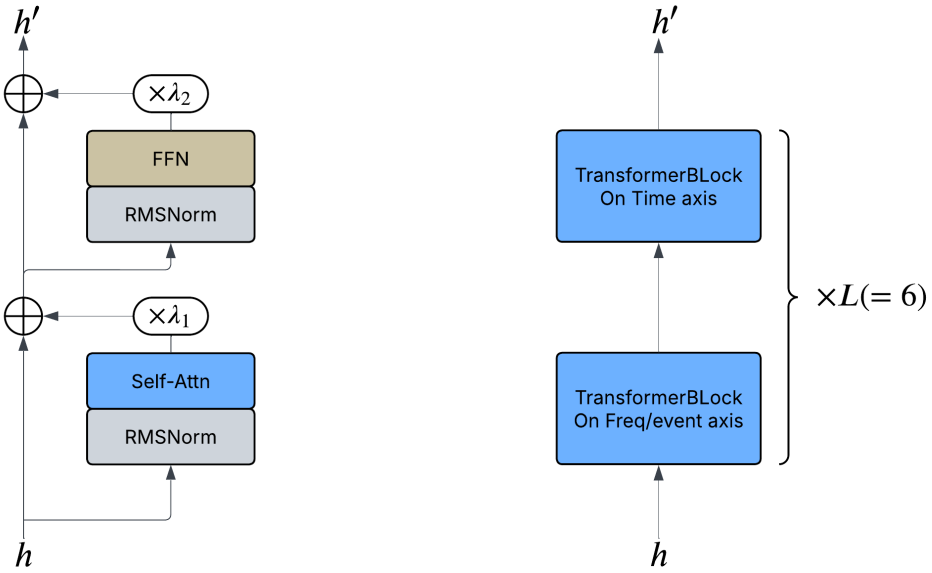
Los *event tracks* se inicializan sumándoles sus propios embeddings de posición espacial (Ecuación 5.2 con coordenadas $(\mathit{frameIdx}, \mathit{eventTypeIdx})$) y se **concatenan** al espectrograma submuestreado enriquecido a lo largo de la dimensión de secuencia [18]. El tensor resultante, de dimensiones $T' \times (F' + N_{\text{events}}) \times 256$, constituye la entrada al transformer encoder. Esta concatenación, indicada como `[concat]` en la Figura 5.1, permite que el *transformer* intercambie información entre las representaciones tiempo-frecuencia del audio y los *event tracks* a través del mecanismo de auto-atención, de forma completamente aprendible.

5.4 Transformer encoder no jerárquico

El *transformer encoder* es el componente central del sistema. Se trata de un ***encoder* no jerárquico** (sin submuestreo adicional ni estructura piramidal) compuesto por $L = 6$ capas, cuya elección como *backbone* responde a la coherencia entre su mecanismo de auto-atención y el módulo de *scoring* por producto interior que constituye la salida del sistema.

Cada una de las $L = 6$ capas del *encoder* contiene **dos bloques *transformer* en cascada**: uno que atiende a lo largo del **eje temporal** y otro a lo largo del **eje de frecuencia o tipo de evento**, alternando en cada capa [18]. Este esquema de **atención axial separable** reduce la complejidad computacional de $O(T^2(F' + N_{\text{events}})^2)$ a $O(T'^2 + (F' + N_{\text{events}})^2)$

por capa, y es análogo al empleado en otras arquitecturas *transformer* eficientes para datos multidimensionales [22, 25]. La organización de los bloques dentro de cada capa se ilustra en la Figura 5.2.



(a) `TransformerBlock` **(b)** Capa del encoder

Figura 5.2: Bloques constitutivos del *transformer encoder*. **(a)** Bloque *transformer* básico (`TransformerBlock`): aplica RMSNorm antes de la auto-atención y de la FFN, y escala la salida de cada subcapa mediante el factor aprendible λ (ReZero) antes de sumarlo al residual. **(b)** Organización de los bloques dentro de cada capa del encoder: un `TransformerBlock` atiende sobre el eje temporal y el siguiente sobre el eje de frecuencia o tipo de evento; esta configuración se repite $L = 6$ veces. Ilustración adaptada de [18].

Cada bloque *transformer*, ilustrado en la Figura 5.2, sigue el diseño estándar de *pre-normalización*: la entrada $\boldsymbol{h}$ se normaliza primero con **RMSNorm** [26], después se aplica la capa de auto-atención multi-cabeza (*Self-Attn*) o la red neuronal de avance (FFN, con tamaño de proyección interna 1024), y finalmente la salida de la subcapa se suma a la conexión residual. Para mejorar la estabilidad del entrenamiento de modelos profundos, se emplea **ReZero** [21]: un factor de escala aprendible λ_k ($k \in \{1, 2\}$), inicializado a 0,01, multiplica la salida de cada subcapa antes de sumarse al residual:

$$\tilde{\boldsymbol{h}} = \boldsymbol{h} + \lambda_1 \cdot \text{Self-Attn}(\text{RMSNorm}(\boldsymbol{h})) \tag{5.3}$$

$$\boldsymbol{h}' = \tilde{\boldsymbol{h}} + \lambda_2 \cdot \text{FFN}(\text{RMSNorm}(\tilde{\boldsymbol{h}})) \tag{5.4}$$

donde $\lambda_{1,0} = \lambda_{2,0} = 0{,}01$ son los valores de inicialización. Este mecanismo garantiza que, al inicio del entrenamiento, la salida de cada bloque sea casi idéntica a su entrada ($\boldsymbol{h}' \approx \boldsymbol{h}$), lo que facilita el flujo del gradiente a través de las 12 subcapas del *encoder* (2 subcapas $\times$ 6 capas) y acelera la convergencia. El *encoder* utiliza **8 cabezas de atención** y una dimensión de embedding de 256 [18].

Tras el procesado por las $L = 6$ capas del *encoder*, únicamente se retienen los *event tracks* (descartando las representaciones del espectrograma, como indica el bloque rojo de la Figura 5.1) ya que son estos vectores los que contienen la información especializada por tipo de evento necesaria para el *scoring* de intervalos. Las representaciones del espec-

trograma, aunque participan en el proceso de atención cruzada durante el *encoder*, no se utilizan en las etapas posteriores.

Los *event tracks* extraídos tienen dimensiones $(T' \times N_{\text{events}} \times 256)$ en la resolución temporal reducida $T' = T/8$, y se pasan a la etapa de remuestreo.

5.5 Scoring por producto interior y semi-CRF

Tras el *encoder*, los *event tracks* se remuestrean temporalmente y se emplean para calcular la matriz de *scoring* que alimenta la capa semi-CRF. Esta sección describe estas dos últimas etapas del sistema: el remuestreo temporal y el módulo de *scoring* por producto interior escalado, el marco semi-CRF y su justificación teórica.

Dado que los *event tracks* operan en la resolución temporal reducida $T' = T/8$, es necesario recuperar la resolución original T para asignar puntuaciones a intervalos con precisión de trama. Para ello se aplica una única **capa convolucional transpuesta 1D** (`ConvTranspose1d`) con tamaño de kernel y *stride* igual a 8 y 128 canales de salida, sin activación no lineal. Esta capa realiza un remuestreo **lineal** a lo largo del eje temporal, produciendo *event tracks* de dimensiones $(T \times N_{\text{events}} \times 128)$. Cada vector $\boldsymbol{h}_i \in \mathbb{R}^{128}$ representa la información de un tipo de evento en la trama i en la resolución temporal completa.

La validez de este remuestreo lineal queda garantizada teóricamente por el Teorema 3.2 del artículo [18]: si la dimensión D' del espacio de baja resolución supera el producto $c \cdot |\mathcal{Y}|$, donde $c = 8$ es el factor de remuestreo y $|\mathcal{Y}|$ es el número de intervalos activos del tipo de evento, entonces el operador de remuestreo lineal no implica ninguna pérdida de expresividad. Con $D' = 256$ y $c = 8$, esto equivale a requerir $|\mathcal{Y}| < 32$ intervalos activos simultáneos por tipo de evento, condición que se satisface ampliamente en la práctica para el piano [18].

Para una secuencia de T tramas de audio, la transcripción se formula como la inferencia de un conjunto $\mathcal{Y}$ de intervalos cerrados no solapados $[i, j]$, donde i y j son índices de trama de inicio y fin respectivamente, y cada intervalo se asocia a un tipo de evento específico. El semi-CRF modela directamente esta estructura definiendo, para cada tipo de evento, dos tipos de puntuaciones a partir de la señal de audio $\mathcal{X}$ [17, 18]:

1. $\text{score}(i, j)$: una matriz triangular $T \times T$ que asigna una puntuación a cada intervalo candidato $[i, j]$, indicando en qué medida corresponde a un evento activo. Los valores diagonales $\text{score}(i, i)$ representan eventos de trama única.

2. $\text{score}_\epsilon(i-1, i)$: un vector de dimensión $T - 1$ que asigna una puntuación a cada par de tramas consecutivas $[i-1, i]$ no cubiertas por ningún intervalo en $\mathcal{Y}$, representando la inactividad del tipo de evento en ese instante.

La puntuación total de un conjunto de intervalos $\mathcal{Y}$ dada la entrada $\mathcal{X}$ se define como [18]:

$$\Phi(\mathcal{Y} \mid \mathcal{X}) = \sum_{[i,j] \in \mathcal{Y}} \text{score}(i, j) \; + \sum_{\substack{[i-1,i] \\ \text{no cubierto en } \mathcal{Y}}} \text{score}_\epsilon(i-1, i) \tag{5.5}$$

Para la **inferencia**, se utiliza la estimación del máximo a posteriori (MAP), que recupera el conjunto óptimo de intervalos no solapados mediante el algoritmo de Viterbi [17]:

$$\mathcal{Y}^* = \arg\max_{\mathcal{Y}} \Phi(\mathcal{Y} \mid \mathcal{X}) \tag{5.6}$$

Para el **entrenamiento**, se emplea estimación de máxima verosimilitud (MLE), con la log-verosimilitud condicional:

$$\log p(\mathcal{Y} \mid \mathcal{X}) = \Phi(\mathcal{Y} \mid \mathcal{X}) - \log \sum_{\mathcal{Y}'} \exp \Phi(\mathcal{Y}' \mid \mathcal{X}) \tag{5.7}$$

donde la suma en el segundo término recorre todos los posibles conjuntos de intervalos no solapados. Este término de normalización, equivalente a la función de partición del modelo, se computa eficientemente mediante el algoritmo *forward-backward* [17]. Se instancia un semi-CRF independiente para cada tipo de evento, procesando en paralelo los 88 tipos de nota y los pedales, tal como indica la anotación $\times 91$ en la Figura 5.1.

El módulo de *scoring* calcula la matriz $\text{score}(i, j)$ a partir de los *event tracks* remuestreados. En el semi-CRF original [17], este módulo empleaba un MLP que tomaba como entrada los *embeddings* de los extremos del intervalo $\boldsymbol{h}_i$ y $\boldsymbol{h}_j$, su producto de Hadamard $\boldsymbol{h}_i \odot \boldsymbol{h}_j$ y los tres primeros momentos estadísticos sobre el intervalo:

$$\text{score}(i, j) = \text{MLP}([\boldsymbol{h}_i,\, \boldsymbol{h}_j,\, \boldsymbol{h}_i \odot \boldsymbol{h}_j,\, \boldsymbol{m}_1,\, \boldsymbol{m}_2,\, \boldsymbol{m}_3]) \tag{5.8}$$

Este enfoque es costoso en memoria: la matriz de *scoring* crece de forma cuadrática con T, y la evaluación del MLP para cada par (i, j) requería procesamiento por segmentos y *gradient checkpointing* durante el entrenamiento, limitando la profundidad y anchura del módulo [18].

Transkun propone sustituirlo por una operación de **producto interior escalado** entre vectores de baja dimensión, tal como muestra la Figura 5.1:

$$\text{score}(i, j) = \frac{|j - i|}{\sqrt{D}} \langle \boldsymbol{q}_i,\, \boldsymbol{k}_j \rangle + b_i\, \delta(i, j) \tag{5.9}$$

donde $\delta(i, j)$ es la delta de Kronecker (vale 1 si $i = j$, 0 en caso contrario), y los vectores $\boldsymbol{q}_i \in \mathbb{R}^D$, $\boldsymbol{k}_i \in \mathbb{R}^D$ y el escalar $b_i \in \mathbb{R}$ se obtienen del *event track* $\boldsymbol{h}_i$ mediante una única capa lineal f, como indica el bloque *Linear layer F* de la Figura 5.1:

$$[\boldsymbol{q}_i,\, \boldsymbol{k}_i,\, b_i] = f(\boldsymbol{h}_i) \tag{5.10}$$

La matriz resultante tiene estructura de **rango bajo más diagonal**: el término del producto interior produce una matriz de rango $D = 64$, y el término $b_i\, \delta(i, j)$ añade la diagonal de forma independiente. Esta formulación es notablemente más simple que el MLP original y no requiere acceder a los *embeddings* internos del intervalo [18]. El factor $|j - i|/\sqrt{D}$ combina el escalado estándar $1/\sqrt{D}$ para estabilizar los gradientes (análogo al escalado del mecanismo de atención) con un factor proporcional a la longitud del intervalo $|j - i|$, que refleja que los intervalos más largos acumulan más evidencia acústica [18].

Justificación teórica de la expresividad

Una pregunta natural es si la simplicidad del producto interior implica una pérdida de capacidad expresiva respecto al MLP original. Yan y Duan responden a esta pregunta formalmente a través del siguiente resultado [18].

Se define una **matriz de *scoring* ideal** $S_{\mathcal{Y}} \in \mathbb{R}^{T\times T}$ para un conjunto de intervalos $\mathcal{Y}$ como aquella en que:

$$S_{\mathcal{Y}}(i,j) > 0 \quad \forall\, [i,j] \in \mathcal{Y}, \qquad S_{\mathcal{Y}}(i,j) = -\epsilon \quad \text{en caso contrario}$$

con $\epsilon > 0$. Con esta matriz la decodificación MAP produce necesariamente $\mathcal{Y}$, ya que excluir cualquier intervalo activo o incluir cualquier intervalo inactivo reduciría la puntuación total.

Lema (Yan y Duan, 2024)**.** *El rango de una matriz de scoring ideal $S_{\mathcal{Y}}$ para un conjunto de intervalos no solapados $\mathcal{Y}$ es $M+1$, donde $M = |\mathcal{Y}|$ es el número de intervalos activos.*

Teorema (Yan y Duan, 2024)**.** *Sea $\mathcal{Y}$ un conjunto de M intervalos no solapados sobre T pasos. Una matriz de scoring ideal $S_{\mathcal{Y}}$ puede representarse como el producto interior por pares entre dos secuencias 1D $(k_i)_i$ y $(q_i)_i$ de vectores:*

$$S_{\mathcal{Y}}(i,j) = \langle q_i,\, k_j \rangle$$

con $\operatorname{rank}(Q_{\mathcal{Y}}) > M$ *y* $\operatorname{rank}(K_{\mathcal{Y}}) > M$*, donde* $Q_{\mathcal{Y}} = [q_0, \ldots, q_{T-1}]$ *y* $K_{\mathcal{Y}} = [k_0, \ldots, k_{T-1}]$.

Este resultado tiene dos implicaciones prácticas directas [18]. En primer lugar, la dimensión D de los vectores q_i y k_i debe superar el número total de intervalos activos $|\mathcal{Y}|$. En segundo lugar, dado que los vectores se obtienen aplicando el remuestreo lineal u_c de la capa `ConvTranspose1d` sobre la secuencia de baja resolución, la dimensión requerida en el espacio de baja resolución satisface $D' > c \cdot |\mathcal{Y}|$ con $c = 8$. Con $D' = 256$, esto garantiza que el diseño de baja resolución temporal no implica ninguna pérdida de expresividad.

La relación entre la Ecuación 5.9 y el mecanismo de atención por producto interior escalado de los *transformers* [27] es clara: ambos computan una puntuación escalar para cada par de posiciones (i,j) mediante el producto interior entre un vector de consulta q_i y un vector de clave k_j, escalado por $1/\sqrt{D}$. La diferencia estructural fundamental reside en la **normalización**: la atención normaliza por posición mediante *softmax*, produciendo una distribución de probabilidad local por fila, mientras que el semi-CRF normaliza de forma **global** sobre todos los posibles conjuntos de intervalos no solapados $\mathcal{Y}'$, garantizando que la decodificación sea globalmente óptima en lugar de localmente *greedy* [18].

Esta analogía no es solamente formal: motiva la elección del *transformer* como *backbone*, ya que puede interpretarse como una arquitectura que refina iterativamente las representaciones para que sean adecuadas para el *scoring* por producto interior. El estudio de ablación del artículo original confirma que una capa lineal para el cálculo de q, k y b (Ecuación 5.10) supera a un MLP, alineándose con cómo se calculan las consultas y claves en los *transformers* estándar [18].

5.6 Predicción de atributos y procesado por segmentos

La capa semi-CRF produce, tras la decodificación MAP, un conjunto de intervalos $[i,j]$ para cada tipo de evento con precisión de trama. Sin embargo, la representación MIDI final requiere información adicional: la **velocidad** de cada nota, las **posiciones subframe** de inicio y fin para obtener los instantes exactos de *onset* y *offset*, y un par de **indicadores binarios** para gestionar correctamente los eventos que cruzan los límites entre segmentos de audio consecutivos. Estos atributos se predicen, en paralelo con la decodificación semi-CRF, mediante el Attribute MLP visible en la rama derecha de la Figura 5.1.

Una vez decodificados los intervalos, cada evento $[a,b]$ se caracteriza mediante cuatro atributos predichos por un **MLP de dos capas** con tamaño oculto de 512 unidades y

dropout de tasa 0,1 [18]. La entrada al MLP es la concatenación de los *embeddings* del *event track* en los extremos del intervalo, $[\boldsymbol{h}_a^{\text{eventType}}; \boldsymbol{h}_b^{\text{eventType}}]$, tal como indica la anotación *input*: $[\mathbf{h}_a; \mathbf{h}_b]$ en la Figura 5.1:

- **Velocidad** (*velocity*) $\in \{0, \ldots, 127\}$: modelada como una distribución categórica sobre 128 clases mediante una capa *softmax*. La velocidad MIDI representa la intensidad de pulsación de cada tecla como un valor entero en el rango estándar MIDI [18].
- **Posiciones refinadas de *onset* y *offset***: dado que los índices de trama a y b son posiciones cuantizadas (resolución de $\approx$ 23,2 ms), el instante exacto de inicio y fin de cada nota puede caer en cualquier punto dentro de esa trama. El modelo predice un desplazamiento relativo $\in (-0{,}5,\ 0{,}5)$ para cada extremo, modelado mediante una **distribución de Bernoulli continua** desplazada en $-0{,}5$ [28]. Esta distribución proporciona soporte continuo sobre el intervalo abierto $(-0{,}5,\ 0{,}5)$ con gradientes no nulos en todo el dominio [18].
- **Indicadores** `hasOnset` **y** `hasOffset` $\in \{0, 1\}$: dos variables binarias modeladas mediante distribuciones de Bernoulli, cuyo significado se detalla a continuación.

La pérdida de entrenamiento del módulo de predicción de atributos se suma a la log-verosimilitud del semi-CRF (Ecuación 5.7), de forma que el objetivo total es [18]:

$$\mathcal{L} = -\left[\log p(\mathcal{Y} \mid \mathcal{X}) + \sum_{e \in \mathcal{Y}} \log p(\text{atributos} \mid e)\right] \tag{5.11}$$

donde $\mathcal{X}$ es el espectrograma de entrada, $\mathcal{Y}$ el conjunto de intervalos ground truth, y e un evento concreto. El segundo término se descompone en:

$$\begin{aligned} \log p(\text{atributos} \mid e) = {} & \log p(\text{velocity} \mid e) \\ & + \log p(\text{onset refinado} \mid e) \\ & + \log p(\text{offset refinado} \mid e) \end{aligned} \tag{5.12}$$

Procesado por segmentos y gestión de eventos incompletos

El *transformer* encoder opera sobre segmentos de audio de duración fija de **16 segundos**. Para transcribir grabaciones de mayor duración, el audio se divide en segmentos consecutivos con un **solapamiento del 50 %** (8 segundos de solape), y los eventos decodificados en cada segmento se ensamblan en una secuencia global [18].

El principal desafío de esta estrategia es la gestión de los eventos que **cruzan el límite** de un segmento. El semi-CRF original [17] descartaba directamente estos eventos durante el entrenamiento. Transkun adopta una solución más precisa: **truncar** el evento en el límite del segmento e introducir los indicadores binarios `hasOnset` y `hasOffset` para registrar si el truncamiento ha tenido lugar [18]:

- `hasOnset` $= 1$: el *onset* del evento pertenece al segmento actual (no ha sido truncado por el límite izquierdo).
- `hasOffset` $= 1$: el *offset* del evento pertenece al segmento actual (no ha sido truncado por el límite derecho).

Con estos indicadores, el algoritmo de ensamblado entre segmentos opera de la siguiente forma para cada tipo de evento [18]:

1. La decodificación dentro de cada segmento comienza desde el máximo entre: (a) el límite izquierdo del segmento actual, y (b) el *offset* del último evento en el conjunto resultado con `hasOffset` $= 1$.
2. Los eventos decodificados con `hasOnset` $= 1$ que no se solapan con el último evento del conjunto resultado se añaden directamente.
3. Para los eventos que se solapan con el último evento del conjunto resultado:
   - Si el nuevo evento tiene `hasOnset` $= 1$, reemplaza al anterior (el anterior era un fragmento truncado que ya no tiene soporte en el segmento nuevo).
   - Si el nuevo evento tiene `hasOnset` $= 0$, es la continuación del evento anterior, y ambos fragmentos se **fusionan** en un único evento.

Este mecanismo es especialmente relevante para los **pedales**, que con frecuencia superan los 16 segundos de duración de un segmento (en particular el pedal *sustain*), mientras que las notas individuales raramente exceden este límite. El estudio de ablación del artículo original confirma que omitir este tratamiento produce una degradación notable en las métricas de pedal, con un impacto prácticamente nulo en las métricas de notas [18].

5.7 Experimentos

Los experimentos de entrenamiento y evaluación del modelo Transkun se han llevado a cabo en el clúster de cómputo de alto rendimiento del DSIC. La descripción detallada del entorno *hardware* y *software*, así como de las dificultades de configuración del entorno y la finalización del entrenamiento, se recoge en el Anexo B.

Antes del entrenamiento, se ejecuta una fase de preprocesamiento del *dataset*. A diferencia del flujo empleado en MFT-FiLM, Transkun calcula sus espectrogramas log-mel en tiempo de entrenamiento a partir de los audios originales de MAESTRO, sin necesidad de almacenar una caché intermedia de representaciones tiempo–frecuencia. Los archivos MIDI se procesan para generar los conjuntos de intervalos de *ground truth* que alimentan la capa semi-CRF durante el entrenamiento [18].

A continuación se analizan los resultados obtenidos por el modelo Transkun entrenado sobre MAESTRO v3.0.0. Las métricas empleadas en el conjunto de *test* coinciden con el protocolo estándar descrito en la Sección 3.9: precisión, *recall* y F_1 a nivel de trama y de nota, en sus variantes *Note Onset*, *Note w/ Offset* y *Note w/ Offset + Velocity* [18, 13]. Para garantizar la comparabilidad directa con el trabajo original, las predicciones se obtienen con los parámetros de inferencia predeterminados del repositorio de referencia.

Comenzamos con los datos obtenidos durante el entrenamiento del modelo. La Figura 5.3 muestra la evolución lote a lote de la función de pérdida durante la primera época de entrenamiento del modelo Transkun sobre MAESTRO v3.0.0. Su análisis permite verificar que la red neuronal arranca correctamente y que el mecanismo de optimización responde de forma adecuada desde el primer instante en que el modelo comienza a procesar datos.



Figura 5.3: Evolución de la función de pérdida de entrenamiento (*Train Loss*) lote a lote durante la primera época (época 0) del modelo Transkun. El eje horizontal representa el índice de lote procesado de forma secuencial y el eje vertical el valor instantáneo de la pérdida *meanNLL* calculado sobre ese lote. Se aprecia una caída abrupta en los primeros 160 lotes (desde valores próximos a 6000 hasta el entorno de 1200), seguida de una fase de oscilación ruidosa con tendencia decreciente sostenida que lleva la pérdida hasta valores en torno a 290–340 al final de la época.

La curva de la Figura 5.3 muestra el comportamiento típico de un *transformer* con pesos inicializados aleatoriamente: una caída inicial muy pronunciada en los primeros lotes, característica de modelos que parten de predicciones casi uniformes sobre el espacio de intervalos del semi-CRF y donde cualquier ajuste de parámetros produce una ganancia inmediata y elevada en la verosimilitud, seguida de un descenso progresivo pero ruidoso. Las oscilaciones de amplitud considerable son el comportamiento esperado cuando se entrena con lotes de tamaño moderado, pues la dificultad intrínseca de cada lote varía según la densidad polifónica y el *tempo* de la interpretación, introduciendo varianza natural en la estimación del gradiente. A lo largo de toda la época, la tendencia decreciente se mantiene sin señales de estancamiento ni de divergencia. Cabe destacar que la magnitud absoluta de la pérdida es intrínseca a la función objetivo de Transkun: a diferencia de la suma de entropías cruzadas binarias empleada en MFT-FiLM, el *meanNLL* del semi-CRF normaliza sobre el espacio de todos los conjuntos de intervalos no solapados posibles, por lo que sus valores no son directamente comparables en escala con los del capítulo anterior.

Continuamos con la pérdida general. La Figura 5.4 presenta la evolución de la función de pérdida a lo largo de las 112 épocas completas de entrenamiento, comparando el error cometido sobre el conjunto de entrenamiento (*Train Loss*, media del *meanNLL* por época) con el error sobre el conjunto de validación (*Val meanNLL*). Esta gráfica constituye la referencia principal para analizar la dinámica de aprendizaje global del modelo y detectar la presencia de sobreajuste (*overfitting*).

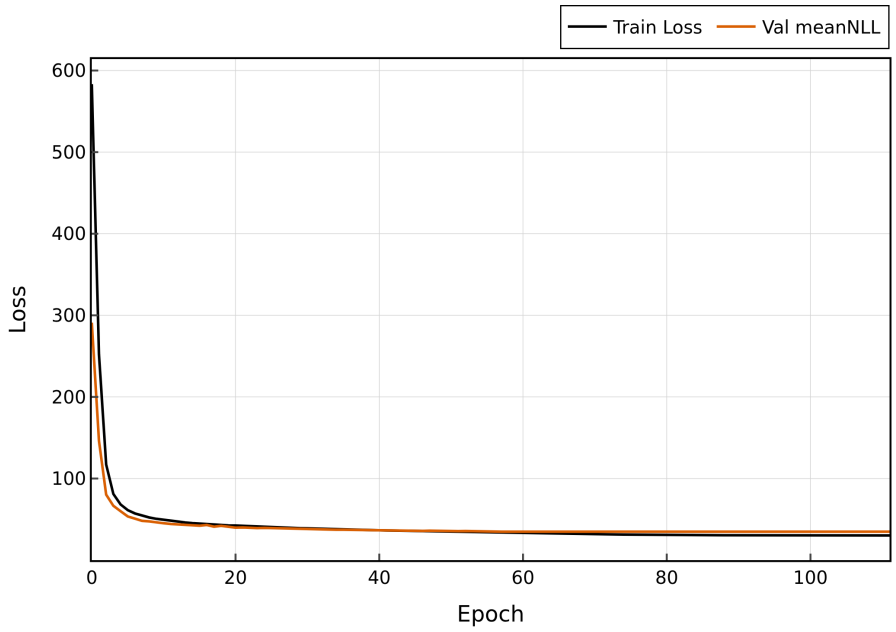


Figura 5.4: Evolución de la pérdida de entrenamiento (*Train Loss*) y de validación (*Val meanNLL*) por época a lo largo de las 112 épocas de entrenamiento del modelo Transkun sobre MAESTRO v3.0.0. El eje horizontal representa la época y el eje vertical el valor medio del *meanNLL* calculado sobre el conjunto correspondiente. Ambas curvas parten de valores elevados en las primeras épocas y descienden de forma pronunciada hasta aproximadamente la época 40, tras la cual la pérdida de entrenamiento continúa disminuyendo lentamente mientras la de validación se estabiliza.

En la Figura 5.4 se observa que ambas curvas descienden de forma casi exponencial durante las primeras épocas. En este tramo inicial, la pérdida de validación se mantiene sistemáticamente por debajo de la de entrenamiento, comportamiento habitual en modelos con *dropout* activo: las máscaras aleatorias inflan la estimación del *meanNLL* sobre el conjunto de entrenamiento, mientras que durante la validación el modelo opera con todos sus pesos activos. A medida que el entrenamiento avanza, las dos curvas invierten su posición relativa (señal de que el modelo ha comenzado a ajustarse a las particularidades del conjunto de entrenamiento) y continúan descendiendo de forma cada vez más gradual. La pérdida de validación alcanza su mínimo en torno a la época 72 y a partir de ahí se estabiliza oscilando en una banda estrecha, mientras la pérdida de entrenamiento sigue disminuyendo muy lentamente describiendo una asíntota práctica. No se aprecia sobreajuste severo: la brecha entre ambas curvas crece de forma moderada y la validación no muestra tendencia creciente en ningún momento del entrenamiento.

Además de la función de pérdida principal, durante el entrenamiento se monitoriza la capacidad del modelo para estimar la velocidad MIDI y el *offset* de cada nota. La calidad de estas dos métricas se evalúa mediante el error cuadrático medio (*Mean Squared Error*, MSE) calculado por época. Dado que ambas magnitudes operan en escalas muy distintas (la velocidad se expresa en unidades MIDI al cuadrado y el *offset* en segundos al cuadrado), se presentan en dos gráficas separadas, Figuras 5.5 y 5.6.

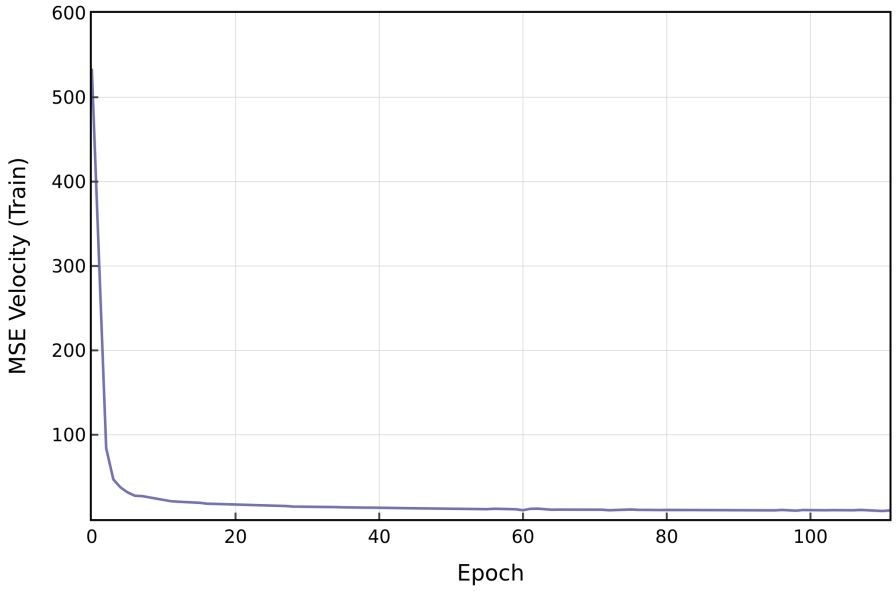


Figura 5.5: Evolución del error cuadrático medio (*MSE*) de la estimación de velocidad MIDI sobre el conjunto de entrenamiento a lo largo de las 112 épocas del modelo Transkun. El eje horizontal representa la época y el eje vertical el MSE en unidades MIDI^2. La curva parte de un valor de 533,93 en la época 0 y desciende de forma pronunciada durante las primeras épocas, estabilizándose progresivamente en torno a 10–11 unidades MIDI^2 a partir de la época 70, con oscilaciones crecientes en la segunda mitad del entrenamiento que reflejan la variabilidad intrínseca de la dinámica entre piezas del conjunto de entrenamiento.

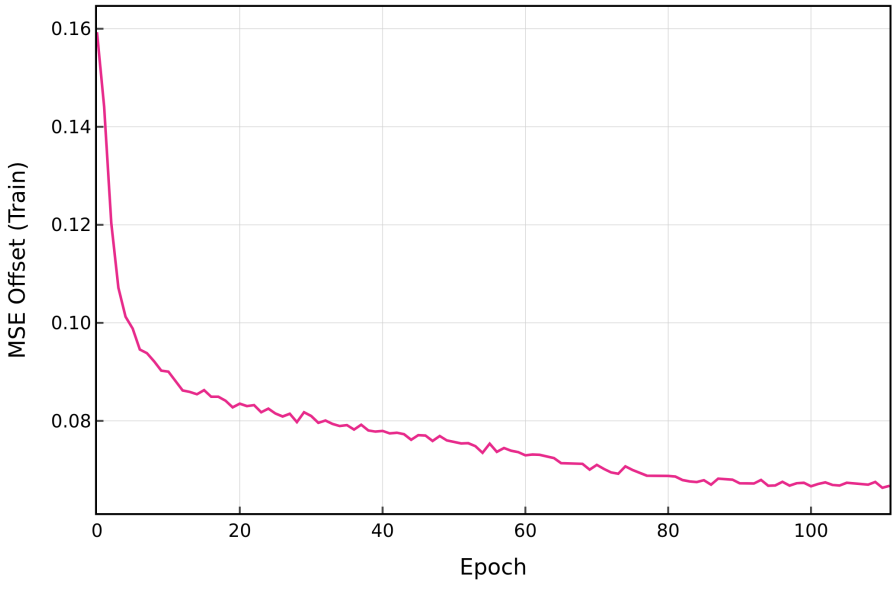


Figura 5.6: Evolución del error cuadrático medio (*MSE*) de la estimación del instante de *offset* sobre el conjunto de entrenamiento a lo largo de las 112 épocas del modelo Transkun. El eje horizontal representa la época y el eje vertical el MSE en segundos^2. La curva desciende de forma suave y continua desde 0,1593 s^2 en la época 0 hasta 0,0663 s^2 en la época 110, mostrando una reducción del 58 % sin signos de estancamiento a lo largo de todo el entrenamiento.

El análisis conjunto de ambas figuras revela comportamientos cualitativamente distintos para cada uno de los dos atributos. El MSE de velocidad (Figura 5.5) presenta la caída más pronunciada del entrenamiento, con una reducción superior al 90 % en las primeras épocas que evidencia que el *Attribute MLP* adquiere rápidamente la capacidad de estimar el orden de magnitud de la dinámica de cada nota. Esto se explica por la naturaleza del regresor: la red aprende a evitar predicciones de velocidad extremas o uniformes y converge rápidamente hacia la media estadística del dataset. A partir de entonces, la curva se estabiliza en una banda estrecha sin tendencia decreciente clara, lo que indica que el modelo podría haber alcanzado el límite de lo que puede aprender sobre la dinámica de las interpretaciones a partir de la representación espectral de entrada. El MSE de *offset* (Figura 5.6), en cambio, muestra un comportamiento radicalmente diferente: desciende de forma lenta, suave y prácticamente monótona durante todo el entrenamiento, sin saturarse en ningún momento. Este contraste pone de manifiesto que la estimación del instante de finalización de una nota es intrínsecamente más compleja: a diferencia del *onset*, el *offset* depende del uso del pedal de *sustain*, de la resonancia del instrumento y de la decisión interpretativa del pianista, factores difíciles de inferir a partir del espectrograma.

Como último análisis de indicadores durante el entrenamiento, la Figura 5.7 presenta la evolución de las métricas de validación del modelo. La métrica reportada es la *Note w/ Offset* (precisión, *recall* y F_1), que exige tanto la detección correcta del *onset* como la estimación del *offset* dentro de la tolerancia definida por el protocolo `mir_eval`, y representa un criterio de evaluación más exigente que el simple *Note Onset* (véase Sección 3.9).

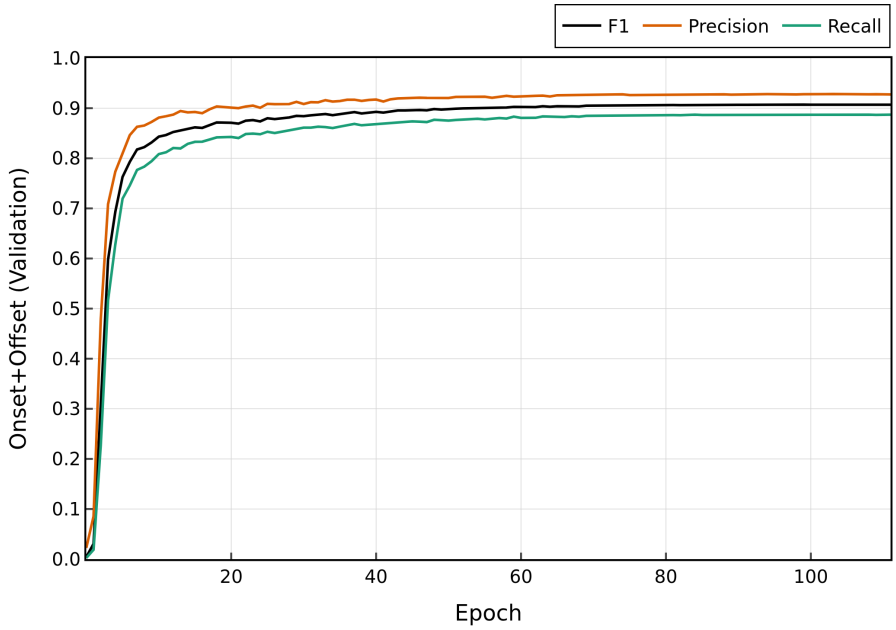


Figura 5.7: Evolución de la precisión, *recall* y F_1 de la métrica *Note w/ Offset* sobre el conjunto de validación a lo largo de las 112 épocas del modelo Transkun. El eje horizontal representa la época y el eje vertical la puntuación en el rango $[0, 1]$. Las tres métricas ascienden rápidamente durante las primeras épocas, superan 0,90 de F_1 en la época 54 y se estabilizan en una banda estrecha en torno a 0,906–0,908 a partir de la época 79, sin tendencia decreciente apreciable hasta el final del entrenamiento. La precisión se mantiene de forma consistente por encima del *recall* a lo largo de todo el proceso.

En la Figura 5.7 se aprecia que el F_1 de validación asciende de forma casi exponencial durante las primeras épocas, lo que confirma que el modelo construye representaciones genuinamente generalizables desde el primer momento. En este tramo inicial, la

precisión crece más lentamente que el *recall*, indicando que el modelo aprende primero a activar los intervalos de nota correctos antes de suprimir las predicciones erróneas; a partir de la época 10 esta relación se invierte de forma estable, consolidando un sesgo conservador análogo al observado en MFT-FiLM [13]: el modelo prefiere omitir notas antes que predecir notas incorrectas. La mejora continúa de forma sostenida hasta superar el 90 % de F_1, en coherencia con el descenso prolongado del MSE de *offset* de la Figura 5.6, y la brecha entre precisión y *recall* se estrecha progresivamente a medida que el modelo aprende a detectar notas de mayor dificultad. A partir de ese punto, el F_1 entra en un régimen de oscilación estable sin tendencia creciente apreciable, con un máximo absoluto registrado en la época 110. Esta compresión del rango de variación indica que el modelo está muy cerca de su límite de generalización y que épocas adicionales no producirían mejoras significativas. La elección del *checkpoint* final para la evaluación sobre el conjunto de test queda plenamente justificada: la curva de validación no muestra ningún indicio de degradación que aconseje una parada anticipada.

A continuación se analizan los resultados finales obtenidos por el modelo. Las Tablas 5.1, 5.2 y 5.3 recogen los resultados de evaluación del modelo Transkun entrenado en este trabajo sobre el conjunto de *test* de MAESTRO v3.0.0, junto con los valores reportados por los autores del artículo original [18] y los obtenidos en el capítulo anterior por el modelo MFT-FiLM. Se presentan, respectivamente, las puntuaciones F_1, la precisión y el *recall*, tanto a nivel de trama como a nivel de nota, en sus tres variantes de complejidad creciente.

Tabla 5.1: Resultados de transcripción de notas sobre MAESTRO v3.0.0. Se muestran el número de parámetros y las puntuaciones F_1 a nivel de trama (*Frame*) y de nota (*Note On.*, *Note w/ Off.*, *Note w/ Off. & Vel.*). Todas las métricas se expresan en porcentaje (%).

Modelo	**Parámetros**	**Frame** F_1	**On.** F_1	**On.+Off.** F_1	**On.+Off.+Vel.** F_1
Transformer [14]	54 M	—	96.01	83.94	82.75
SemiCRF [17]	9 M	90.75	96.11	88.42	87.44
HPPNet [16]	1.2 M	93.15	97.18	83.80	82.24
hFT-Transformer [15]	5.5 M	93.24	97.44	90.53	89.48
HATANet [12]	6.4 M	94.04	97.81	90.57	89.25
MFT-FiLM (autores) [13]	1.2 M	94.75	98.39	92.46	91.79
MFT-FiLM (este trabajo)	1.2 M	94.36	98.17	91.60	90.78
Transkun / SemiCRF V2 (autores) [18]	12.9 M	95.35	98.32	93.48	92.94
Transkun (este trabajo)	12.9 M	94.69	98.03	92.06	91.52

En la Tabla 5.1 se aprecia que la reproducción de Transkun alcanza un F_1 de trama del 94,69 %, frente al 95,35 % reportado por los autores, lo que representa una diferencia de 0,66 puntos porcentuales. En las métricas de nota, la brecha es de magnitud similar: el F_1 de *Note On.* se sitúa en 98,03 % frente al 98,32 % original, mientras que en *Note w/ Off.* y *Note w/ Off. & Vel.* los valores obtenidos son 92,06 % y 91,52 %, frente a 93,48 % y 92,94 % respectivamente. Estas diferencias son atribuibles a la detención anticipada del entrenamiento por limitaciones de cómputo y tiempo: aunque las métricas de validación a nivel de nota mostraban una mejora prácticamente negligible en las últimas épocas, otras métricas intermedias, como el MSE del *offset*, presentaban todavía una tendencia decreciente apreciable en las curvas de entrenamiento, lo que indica que el modelo podría no haber alcanzado su convergencia completa.

En comparación con el modelo MFT-FiLM entrenado en el capítulo anterior, la reproducción de Transkun obtiene resultados superiores en prácticamente todas las métricas. La única excepción es el F_1 de *Note On.*, donde MFT-FiLM alcanza 98,17 % frente a 98,03 % de Transkun, una diferencia de tan solo 0,14 puntos porcentuales que puede considerarse marginal. En el resto de métricas, Transkun supera a MFT-FiLM en 0,33 puntos en *Frame*, en 0,46 puntos en *Note w/ Off.* y en 0,74 puntos en *Note w/ Off. & Vel.*. Siendo esta última

la métrica más representativa de la calidad de una transcripción completa (debido a que incorpora la evaluación conjunta del tono, el inicio, la duración y la dinámica de cada nota) puede concluirse que Transkun ofrece en términos generales una transcripción de mayor calidad que MFT-FiLM. Esta ventaja se obtiene, sin embargo, a costa de un coste computacional considerablemente mayor: con 12,9 M de parámetros frente a los 1,2 M del MFT-FiLM, el entrenamiento de Transkun fue notablemente más extenso en duración y en recursos, consecuencia directa de la mayor complejidad de su arquitectura *transformer* y del esquema de decodificación secuencia a secuencia que emplea.

Tabla 5.2: Resultados de precisión (*P*) de transcripción de notas sobre MAESTRO v3.0.0. Se muestran el número de parámetros y las métricas de precisión a nivel de trama (*Frame*) y de nota (*Note On.*, *Note w/ Off.*, *Note w/ Off. & Vel.*). Todas las métricas se expresan en porcentaje (%).

Modelo	Parámetros	Frame P	On. P	On.+Off. P	On.+Off.+Vel. P
Transformer [14]	54 M	—	98.61	86.19	84.95
SemiCRF [17]	9 M	93.79	98.69	90.79	89.78
HPPNet [16]	1.2 M	92.79	98.45	84.88	83.29
hFT-Transformer [15]	5.5 M	92.82	99.64	92.52	91.43
HATANet [12]	6.4 M	95.51	99.05	91.70	90.35
MFT-FiLM (autores) [13]	1.2 M	95.77	99.33	93.33	92.65
MFT-FiLM (este trabajo)	1.2 M	95.27	99.37	92.70	91.86
Transkun / SemiCRF V2 (autores) [18]	12.9 M	95.75	99.53	94.61	94.07
Transkun (este trabajo)	12.9 M	95.21	99.68	93.58	93.03

Tabla 5.3: Resultados del *recall* (*R*) en transcripción de notas sobre MAESTRO v3.0.0. Se muestran el número de parámetros y las métricas de *recall* a nivel de trama (*Frame*) y de nota (*Note On.*, *Note w/ Off.*, *Note w/ Off. & Vel.*). Todas las métricas se expresan en porcentaje (%).

Modelo	Parámetros	Frame R	On. R	On.+Off. R	On.+Off.+Vel. R
Transformer [14]	54 M	—	93.60	81.86	80.70
SemiCRF [17]	9 M	88.36	93.96	86.46	85.51
HPPNet [16]	1.2 M	93.59	95.95	82.76	81.24
hFT-Transformer [15]	5.5 M	93.66	95.44	88.69	87.67
HATANet [12]	6.4 M	92.62	96.61	89.47	88.17
MFT-FiLM (autores) [13]	1.2 M	93.84	97.48	91.62	90.95
MFT-FiLM (este trabajo)	1.2 M	93.56	97.03	90.54	89.73
Transkun / SemiCRF V2 (autores) [18]	12.9 M	95.01	97.16	92.39	91.87
Transkun (este trabajo)	12.9 M	94.69	96.48	90.62	90.10

Las Tablas 5.2 y 5.3 permiten profundizar en el análisis desagregando la F_1 en sus componentes de precisión y *recall*. En la métrica de *Note On.*, la precisión obtenida en este trabajo es de 99,68 %, superando incluso al valor reportado por los autores originales (99,53 %), mientras que el *recall* correspondiente se sitúa en 96,48 %, 0,68 puntos por debajo del original (97,16 %). Este desequilibrio entre precisión y *recall* indica que la mayor parte de los errores residuales consisten en notas omitidas (especialmente en pasajes de elevada densidad polifónica o con articulaciones rápidas) y no en activaciones espurias, lo que apunta a un comportamiento conservador del modelo coherente con los resultados presentados previamente. En las métricas que incorporan el *offset* y la velocidad, tanto la precisión como el *recall* descienden moderadamente respecto a los valores de *onset*, siguiendo el patrón habitual observado tanto en MFT-FiLM como en el resto de modelos del estado del arte. La precisión de *Note w/ Off. & Vel.* alcanza 93,03 % frente al 94,07 % de los autores, y el *recall* correspondiente es de 90,10 % frente al 91,87 % original. La penalización proporcionalmente mayor en *recall* respecto a la precisión sugiere que las discrepancias se concentran principalmente en la estimación de la duración de las notas, subtarea sobre la que la detención anticipada del entrenamiento habría tenido

un impacto más directo, en consonancia con la evolución todavía decreciente del MSE de *offset* observada en las gráficas de entrenamiento.

5.8 Conclusiones

Este capítulo ha presentado la reproducción y evaluación del modelo Transkun [18] sobre el conjunto de *test* de MAESTRO v3.0.0, siguiendo el protocolo estándar de evaluación descrito en la Sección 3.9. El modelo, basado en una arquitectura *transformer* que aborda la transcripción como un problema de traducción de secuencias, ha sido entrenado hasta su detención anticipada por limitaciones de cómputo y tiempo, si bien las métricas de validación a nivel de nota mostraban una convergencia prácticamente estable en las últimas épocas.

Los resultados obtenidos se sitúan entre 0,66 y 1,42 puntos porcentuales por debajo de los reportados por los autores originales en F_1, diferencia atribuible principalmente al entrenamiento incompleto, cuyo impacto es más pronunciado en las métricas de *offset* y velocidad que en las de *onset*, en consonancia con la evolución todavía decreciente del MSE de *offset* observada en las curvas de entrenamiento. A pesar de ello, la reproducción supera en precisión de *Note On.* al propio valor original (99,68 % frente a 99,53 %), lo que confirma la robustez del entrenamiento realizado.

En comparación con el modelo MFT-FiLM reproducido en el capítulo anterior, Transkun obtiene resultados superiores en la mayoría de las métricas, siendo la única excepción las métricas de *Note On.*, donde la diferencia es marginal. Atendiendo a la métrica más representativa de la calidad de una transcripción completa ()*Note w/ Off. & Vel.* F_1) Transkun (91,52 %) supera a MFT-FiLM (90,78 %), si bien esta ventaja se obtiene por un número de parámetros aproximadamente 10 veces superior y de un entrenamiento notablemente más costoso en tiempo y recursos computacionales.

Finalmente, ambos modelos (Transkun y MFT-FiLM) se posicionan por encima de todos los modelos de referencia incluidos en la comparativa en las métricas más exigentes, superando a Transformer [14], SemiCRF [17], HPPNet [16], hFT-Transformer [15] y HATANet [12] en *Note w/ Off.* y *Note w/ Off. & Vel.*. Esto confirma que ambos modelos, evaluados sobre repertorio de música clásica occidental, representan fielmente el estado del arte en transcripción automática de piano.

CAPÍTULO 6
Evaluación fuera del clasicismo: *jazz*

6.1 Introducción

Los capítulos anteriores han presentado los fundamentos de la transcripción automática de piano y las dos arquitecturas exploradas en este trabajo: MFT-FiLM y Transkun. Tras adaptarlos a entornos actuales y entrenarlos, ambos modelos han sido evaluados bajo un protocolo común. Sin embargo, una evaluación limitada al dominio habitual de entrenamiento no permite responder por completo a una cuestión relevante: qué comportamiento mantienen estos sistemas cuando se aplican a música para piano con características estilísticas y acústicas distintas.

Este capítulo aborda dicha cuestión mediante una comparación fuera del dominio clásico, centrada en el piano *jazz*. El objetivo no es sustituir la evaluación estándar sobre MAESTRO, sino complementarla con un análisis que permita observar qué modelo se adapta mejor a las particularidades del *jazz* una vez entrenado. Por tanto, PiJAMA no se emplea como conjunto de entrenamiento ni como base para realizar *fine-tuning*, sino exclusivamente como medio para estudiar la generalización relativa de MFT-FiLM y Transkun.

La Sección 6.2 parte de los resultados obtenidos sobre MAESTRO y reflexiona sobre hasta qué punto este conjunto permite extraer conclusiones generales acerca de la transcripción de piano. En particular, se discute la relación entre el rendimiento de los modelos y factores musicales como la densidad de notas, la polifonía y el estilo compositivo. La Sección 6.3 presenta PiJAMA como conjunto de datos de piano *jazz* solo, describiendo su utilidad para este trabajo y las diferencias acústicas que introduce respecto al repertorio clásico o romántico. Posteriormente, la Sección 6.4 detalla el protocolo experimental seguido para evaluar ambos modelos sobre este conjunto y los resultados que reportan. Finalmente, la Sección 6.5 resume las conclusiones obtenidas.

6.2 Análisis del comportamiento sobre MAESTRO

MAESTRO se ha consolidado como uno de los conjuntos de datos fundamentales para la transcripción automática de piano. Su importancia se debe a que combina interpretaciones reales de piano acústico con una referencia MIDI de alta precisión, obtenida mediante pianos Yamaha Disklavier [6, 3]. Esto permite entrenar y evaluar modelos de transcripción sobre una colección extensa, homogénea y cuidadosamente alineada, convirtiéndolo en un punto de referencia natural para entrenar sistemas de AMT.

No obstante, el hecho de que MAESTRO sea de gran calidad no implica que abarque todos los escenarios posibles de transcripción de piano. La distribución musical del conjunto está fuertemente vinculada al repertorio clásico occidental, especialmente al contexto de interpretaciones de competición pianística (como se comentó en la Sección 3.7). Por esto, un modelo que obtiene buenos resultados sobre MAESTRO puede haber aprendido patrones altamente eficaces para ese dominio concreto, pero no necesariamente equivalentes a una comprensión general de toda la música para piano.

Para comprender en profundidad el comportamiento de ambos modelos sobre este corpus, se ha llevado a cabo un análisis estadístico de sus resultados. Concretamente, se ha evaluado la métrica F_1 *note onset* sobre la partición de prueba (*test split*) en función de diversas variables musicales: notas por segundo, polifonía máxima, duración de la pieza y compositor. Los resultados de este análisis indican que factores como la densidad de notas, la polifonía y el autor afectan de manera significativa al rendimiento. Esto sugiere que, en el estado actual de los sistemas de AMT, una parte sustancial de la dificultad de transcripción no depende exclusivamente de la arquitectura empleada, sino de la propia naturaleza acústica y musical de la obra.

La densidad de notas por segundo constituye una medida directa de la actividad temporal de la pieza. Cuando el número de eventos por unidad de tiempo aumenta, se reduce el espacio disponible para distinguir ataques consecutivos y se incrementa la probabilidad de solapamiento entre notas. De forma complementaria, la polifonía máxima aumenta la complejidad vertical de la textura pianística. Una mayor cantidad de notas simultáneas provoca una superposición más intensa de componentes espectrales, armónicos y colas de resonancia, dificultando la asignación precisa de la energía acústica a eventos MIDI individuales.

Estos dos factores poseen una interpretación física clara. El piano no produce eventos aislados y perfectamente separables, sino sonidos con ataque, decaimiento, resonancia, armónicos y posibles interacciones con el pedal. En consecuencia, el problema de la transcripción no consiste únicamente en detectar la presencia de energía en determinadas frecuencias, sino en decidir si dicha energía corresponde a un nuevo ataque, a una nota ya sostenida, a una resonancia simpática o a la acumulación de varios eventos simultáneos. Esta ambigüedad aumenta con la densidad temporal y con la complejidad polifónica.

El efecto de la época histórica observado en MAESTRO refuerza esta interpretación. Las obras barrocas y clásicas tienden a presentar un rendimiento superior, mientras que el repertorio romántico y moderno introduce una mayor dificultad. Esta diferencia no debe entenderse únicamente como una clasificación cronológica, sino como una manifestación de la evolución de la escritura pianística. A medida que avanza el repertorio, aparecen texturas más densas, mayor uso expresivo del pedal, acordes de gran amplitud, figuraciones rápidas, pasajes de carácter orquestal, trémolos y una mayor saturación del espacio sonoro. Todos estos elementos afectan directamente a la representación tiempo–frecuencia utilizada por los modelos, sin estar necesariamente relacionados con la dificultad de la pieza. Uno de los resultados más interesantes del estudio es la representación de medias e intervalos LSD (*Least Significant Difference*) al 95 % del F_1 respecto al factor compositor (ordenado cronológicamente, con excepciones por época estilística), que podemos ver en la Figura 6.1.

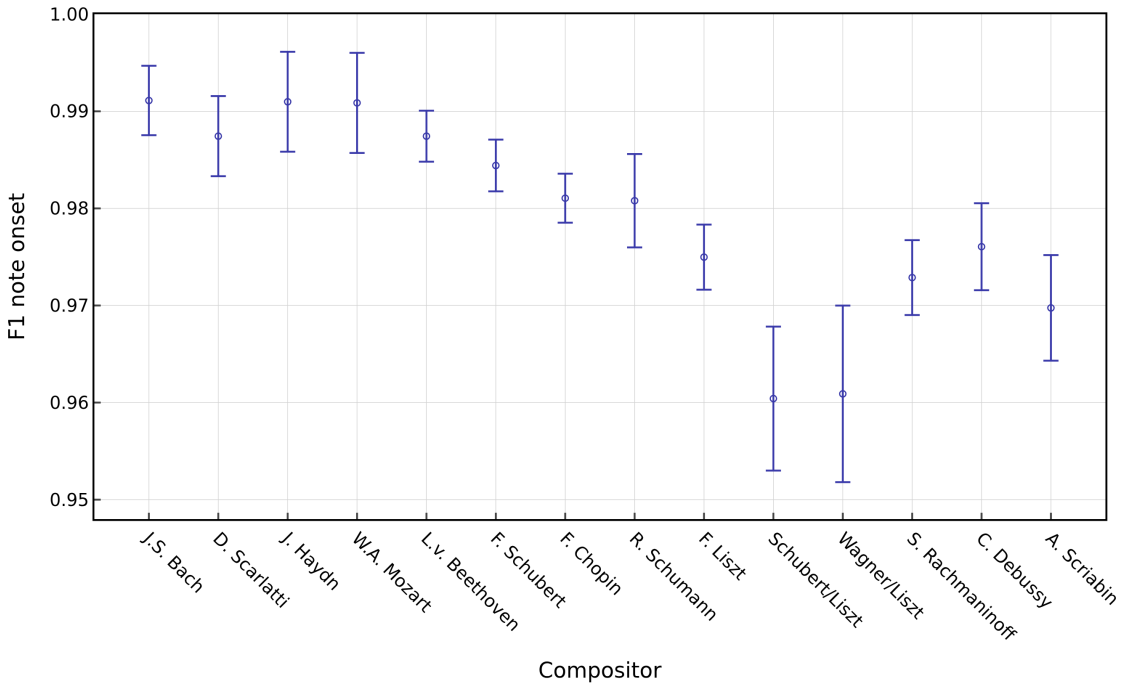


Figura 6.1: Intervalos LSD al 95 % para el F_1 de *note onset* por compositor, ordenados de forma aproximadamente cronológica. La figura resume la tendencia decreciente del rendimiento conforme el repertorio se aproxima a lenguajes pianísticos más densos, complejos y modernos.

La Figura 6.1 muestra esta tendencia de forma visual. Al ordenar los compositores cronológicamente, se aprecia que la calidad de la transcripción no se mantiene constante a lo largo del repertorio. Los autores asociados a una escritura más clara y contrapuntística presentan, en general, mejores puntuaciones, mientras que compositores vinculados a texturas más densas o a una explotación más extrema del instrumento tienden a situarse en regiones de menor F_1. Aunque existen diferencias individuales y el número de piezas por compositor no siempre es equilibrado, la tendencia global resulta coherente con la interpretación musical del problema.

Como alternativa para mitigar el desequilibrio, se pueden considerar los cinco compositores que abarcan la mayoría de grabaciones del *dataset*, presentados en la Sección 3.4, Tabla 3.1.

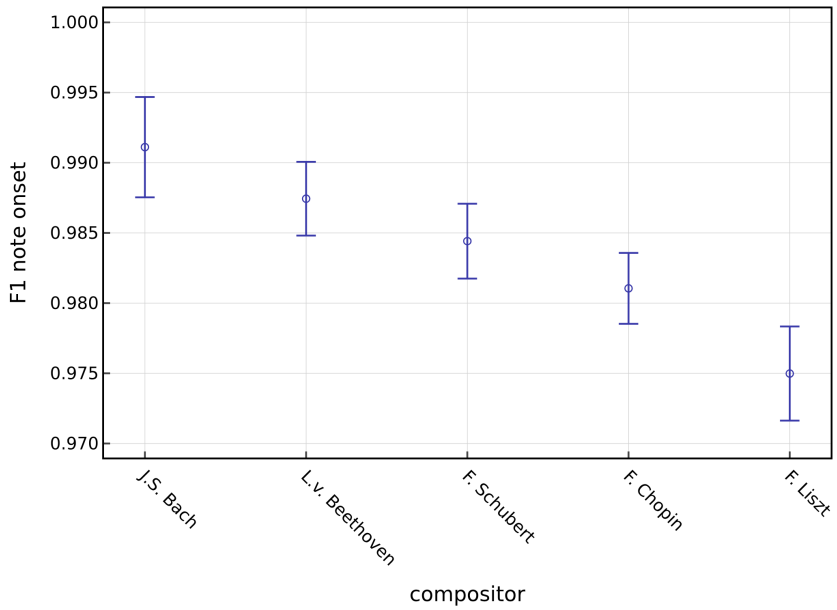


Figura 6.2: Intervalos LSD al 95 % para el F_1 de *note onset* por compositor (los más representativos de MAESTRO), ordenados de forma aproximadamente cronológica. Observamos una clara tendencia decreciente.

En la Figura 6.2 se observa con mayor claridad la tendencia decreciente del F_1 *note onset* para los compositores más representados en el *dataset* MAESTRO, mostrando etapas sucesivas de la música clásica occidental.

Esta observación resulta relevante porque indica que la dificultad de transcripción está relacionada con el lenguaje pianístico. Si dentro del propio *dataset* MAESTRO se detectan diferencias asociadas a la época de composición, entonces es razonable plantear que un cambio más acusado pueda modificar también el comportamiento relativo de los modelos. En este sentido, el *jazz* constituye un caso especialmente interesante: sigue tratándose de piano, pero la distribución musical, la acústica y el periodo difieren de forma notable del repertorio clásico utilizado habitualmente para entrenar y evaluar estos sistemas.

Además del efecto de los factores principales, el análisis sobre el *split* de *test* de MAESTRO mostró una interacción significativa entre duración y densidad de notas por segundo. En particular, las piezas largas con baja densidad presentaban un comportamiento anómalo (un descenso del F_1). Este fenómeno puede explicarse por el procesamiento fragmentado del audio que realizan ambos modelos estudiados. En piezas con baja densidad, las notas pueden presentar colas de resonancia largas, especialmente cuando interviene el pedal. Una ventana puede contener entonces energía acústica procedente de ataques ocurridos en una ventana anterior, generando ambigüedad entre un nuevo evento y la continuación de una nota previa.

Este tipo de errores es especialmente importante al estudiar repertorios fuera del dominio clásico. La música *jazz* utiliza el silencio, la suspensión temporal, los ataques desplazados, la síncopa y la irregularidad expresiva de una forma distinta. Por ello, una arquitectura que se comporta de forma estable en fragmentos densos y regulares puede enfrentarse a dificultades diferentes cuando el discurso musical alterna pasajes muy cargados con espacios de silencio o con resonancias aisladas.

Bajo esta perspectiva, la evaluación sobre *jazz* se plantea como un experimento de generalización. El objetivo no es afirmar que el *jazz* sea necesariamente más difícil que el

repertorio clásico en todos los casos, sino observar si las características propias de este estilo modifican la relación entre MFT-FiLM y Transkun. La comparación deja de centrarse únicamente en el rendimiento absoluto y pasa a estudiar qué modelo conserva mejor su capacidad de transcripción ante una distribución musical distinta, además de comprobar si los resultados obtenidos se sitúan de acuerdo con la tendencia decreciente observada en la Figura 6.1.

6.3 PiJAMA

Para estudiar el comportamiento de los modelos fuera del dominio clásico se ha utilizado PiJAMA (*Piano Jazz with Automatic MIDI Annotations*). Se trata de un *dataset* de piano solo compuesto por interpretaciones reales de pianistas de *jazz*, acompañado de anotaciones MIDI generadas automáticamente [30]. Su interés para este trabajo reside en que proporciona una colección amplia y estilísticamente coherente de grabaciones de piano *jazz*, un género ausente en el *dataset* MAESTRO.

PiJAMA contiene 2 777 interpretaciones únicas, procedentes de 120 pianistas y 244 álbumes, con una duración total superior a 200 horas. A diferencia de MAESTRO, no se construye a partir de interpretaciones registradas en un entorno controlado, sino a partir de grabaciones discográficas reales [30]. Esto introduce una variabilidad acústica mayor: diferencias de microfonía, sala, mezcla, reverberación, ruido, aplausos o fragmentos hablados en algunas interpretaciones en directo. Desde el punto de vista de la transcripción automática, esta variabilidad hace que PiJAMA sea un *dataset* especialmente útil para analizar robustez.

El conjunto fue construido a partir de una selección de pianistas representativos de la tradición del piano *jazz*. Los autores identificaron álbumes de piano solo, localizaron grabaciones correspondientes y aplicaron procesos de filtrado para asegurar que el contenido fuera realmente piano solo. También se estimaron los intervalos temporales correspondientes a la interpretación musical, con el objetivo de excluir regiones no pianísticas al inicio o al final de algunas pistas [30]. Esta información resulta importante porque muchas grabaciones de *jazz*, especialmente las procedentes de conciertos, no comienzan ni terminan necesariamente con música: pueden contener aplausos, presentaciones o silencios prolongados.

Las anotaciones simbólicas de PiJAMA se proporcionan en formato MIDI y fueron obtenidas mediante sistemas de transcripción automática de piano. Esta característica debe tenerse en cuenta al interpretar los resultados: PiJAMA no ofrece una referencia capturada mediante Disklavier ni una transcripción manual nota a nota, sino pseudo-etiquetas automáticas. En este trabajo, por tanto, las métricas obtenidas sobre PiJAMA se interpretan como una medida de concordancia frente a una referencia común en dominio *jazz*, no como una medida absoluta equivalente a la evaluación sobre MAESTRO.

Esta precisión metodológica no reduce la utilidad del conjunto. Permite definir con claridad qué pregunta se está respondiendo. Dado que ambos modelos se evalúan sobre las mismas grabaciones y frente al mismo tipo de referencia, PiJAMA permite comparar cuál de ellos se adapta mejor a las características acústicas y musicales del piano *jazz*. El conjunto no se emplea para entrenar los modelos ni para ajustar sus parámetros, de modo que la evaluación refleja su capacidad de generalización una vez finalizado el entrenamiento principal.

La elección de PiJAMA se justifica también por la dificultad de encontrar un equivalente jazzístico o moderno de MAESTRO que resulte adecuado para la evaluación de transcripción polifónica de piano. Durante la preparación del experimento se conside-

raron otros conjuntos de datos relacionados con *jazz* o con transcripción de piano, pero ninguno reunía simultáneamente las condiciones necesarias para este trabajo: piano solo, grabaciones reales, escala suficiente y una representación simbólica comparable a la salida de los modelos evaluados.

Uno de los recursos jazzísticos más relevantes es la *Weimar Jazz Database*, desarrollada en el marco del proyecto *jazzomat*. Este conjunto contiene transcripciones detalladas de solos improvisados de *jazz*, con información de alturas, *onsets*, *offsets*, dinámica e incluso anotaciones de compás y armonía [34]. Sin embargo, su objetivo principal es el análisis de líneas melódicas improvisadas, no la transcripción polifónica completa de piano. Por ello, aunque constituye una referencia de gran valor musical, no resulta adecuada para comparar modelos como MFT-FiLM y Transkun.

También se consideró la *RWC Jazz Music Database*, que forma parte de la colección *RWC Music Database* [33]. Esta base de datos es relevante por incluir material de *jazz* y haber sido diseñada como recurso común para investigación en procesamiento musical. No obstante, su utilidad para este experimento es limitada: el número de interpretaciones de piano *jazz* solo disponibles es muy reducido, insuficiente para comparar de forma estable dos modelos de transcripción. En consecuencia, puede resultar útil como recurso complementario o histórico, pero no como corpus principal de evaluación.

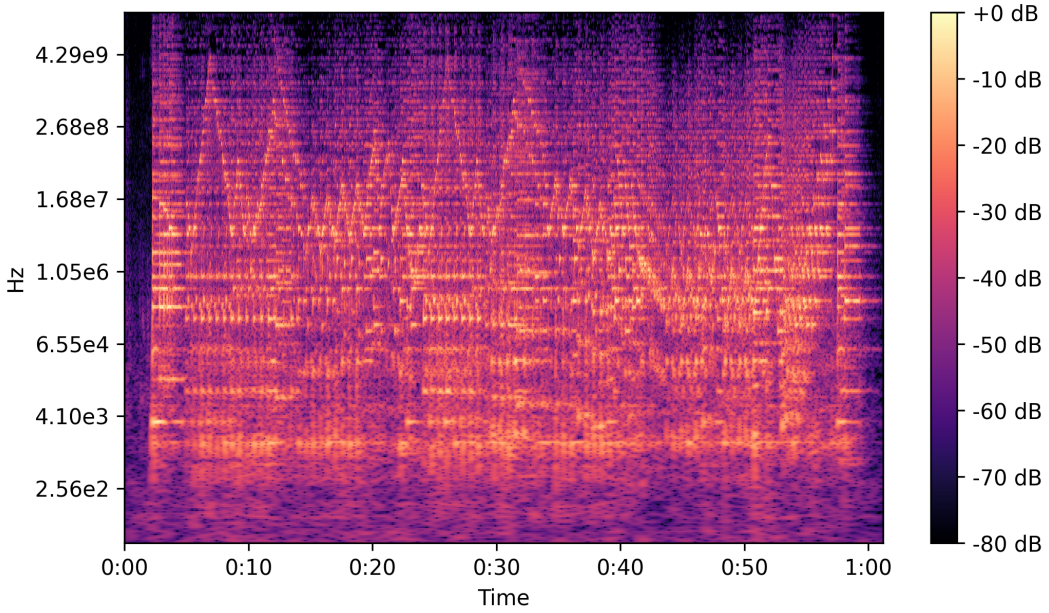
Por último, una alternativa reciente es la *Jazz Trio Database*, compuesta por grabaciones de trío de *jazz* con piano, contrabajo y batería, anotadas mediante un proceso automático que incluye separación de fuentes y extracción de eventos musicales [31]. Aunque este conjunto se aproxima estilísticamente al dominio buscado, no corresponde a piano solo, sino a interpretaciones en conjunto. Esto introduce una dificultad adicional: la transcripción del piano depende de la calidad de la separación de fuentes y queda condicionada por la presencia de otros instrumentos. Dado que MFT-FiLM y Transkun se evalúan en este trabajo como modelos de transcripción de piano solo, utilizar un conjunto con acompañamiento de bajo y batería habría mezclado el problema de generalización estilística con el problema de separación instrumental.

En este contexto, PiJAMA representa el compromiso más adecuado para el objetivo del experimento. Proporciona una colección extensa de piano *jazz* solo, con más de 200 horas de grabaciones reales y una representación MIDI que permite calcular métricas de transcripción nota a nota [30].

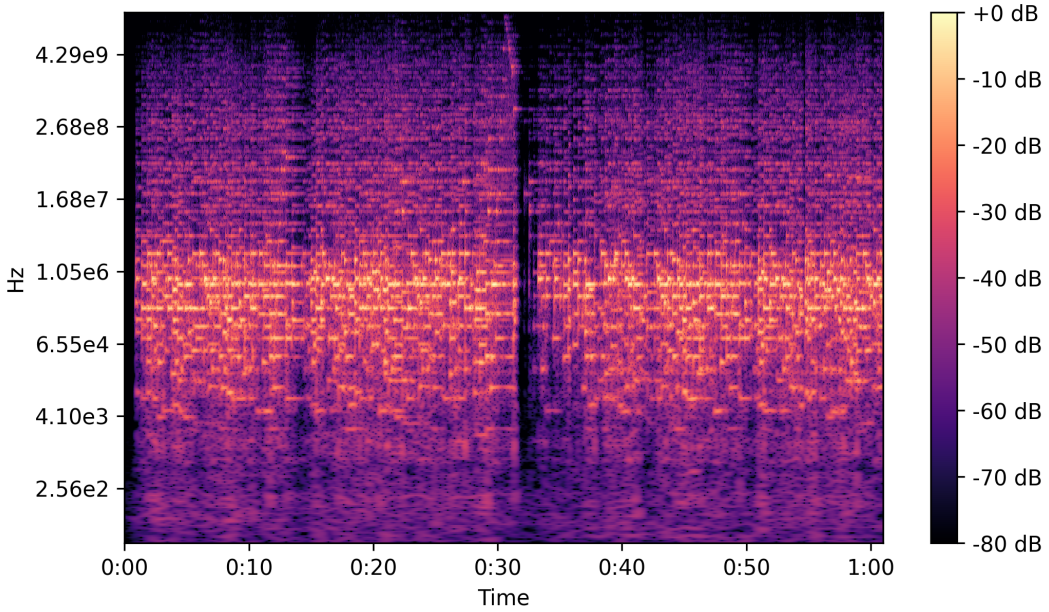
Desde el punto de vista musical, el piano *jazz* introduce una serie de rasgos que lo convierten en un dominio especialmente interesante. La escritura y la interpretación jazzística hacen un uso frecuente de síncopas, desplazamientos rítmicos, acordes extendidos, tensiones armónicas, *voicings*[1] abiertos, ornamentación, improvisación y variaciones expresivas del pulso. Además, el silencio desempeña un papel estructural más marcado que en buena parte del repertorio clásico o romántico: no aparece simplemente como ausencia de sonido, sino como elemento rítmico y discursivo.

Estas propiedades afectan directamente a la señal de audio. En una interpretación clásica o romántica, incluso cuando la textura es compleja, suele existir una organización formal más continua: voces con distribución estable, flujo sonoro con progresión clara y silencios extensos poco frecuentes. En muchas interpretaciones de *jazz*, por el contrario, pueden alternarse pasajes de alta densidad en registros medios con espacios de silencio, ataques aislados, acordes breves, resonancias prolongadas o respuestas rítmicas irregulares. Esta alternancia modifica la distribución temporal de los eventos y, por tanto, la información que reciben los modelos en cada ventana de análisis. La Figura 6.3 presenta de forma gráfica estas propiedades diferenciales que se han comentado.

[1] En el contexto del *jazz* y la armonía moderna, el término *voicing* hace referencia a la disposición vertical específica, distribución y espaciado de las notas que componen un acorde.



(a) Interpretación clásica/romántica



(b) Interpretación de piano *jazz*

Figura 6.3: Comparación cualitativa entre el espectrograma de una interpretación clásica/romántica y una interpretación de piano *jazz*. En la primera se observa una estructura más regular, con separación más clara entre voces y continuidad sonora. En la segunda aparece una distribución más irregular y concentrada en tesituras medias, con mayor densidad local, silencios más marcados y una organización temporal característica del lenguaje jazzístico.

En la Figura 6.3 se observa que el espectrograma correspondiente a la pieza clásica/romántica presenta una estructura más continua, con líneas de energía relativamente organizadas y una separación más clara entre registros. Aunque puede existir complejidad polifónica, la distribución de los eventos tiende a seguir una lógica más estable, lo que facilita al modelo asociar patrones espectrales con ataques concretos.

En el espectrograma de *jazz*, en cambio, la energía aparece con una organización menos regular. Es habitual encontrar una concentración importante en tesituras medias, donde se sitúan muchos *voicings* y acompañamientos armónicos, junto con ataques más aislados, silencios expresivos y cambios bruscos de densidad. Esta combinación puede generar regiones en las que el sonido parece estar más mezclado, no necesariamente por una mayor polifonía global, sino por la superposición de acordes, resonancias, pedal y variaciones de articulación. Para un modelo entrenado principalmente sobre repertorio clásico, estas configuraciones pueden representar un cambio relevante en la distribución de entrada.

El silencio merece una consideración particular. En un análisis puramente cuantitativo, una pieza con espacios sin sonido puede presentar una densidad media de notas menor. Sin embargo, desde el punto de vista de la transcripción, esos silencios no simplifican necesariamente el problema. Si la interpretación alterna silencios con ataques densos o con acordes de gran riqueza armónica, el modelo debe resolver transiciones abruptas y distinguir con precisión entre el final de una resonancia, la ausencia real de eventos y la aparición de un nuevo ataque. Este comportamiento se relaciona directamente con la anomalía observada en MAESTRO para piezas largas con baja densidad de notas (Sección 6.2): una baja actividad media no siempre implica menor dificultad.

El uso de PiJAMA permite, por tanto, explorar una dimensión que no queda plenamente cubierta por MAESTRO. La cuestión no es únicamente si los modelos detectan bien notas en piano acústico, sino si los patrones aprendidos durante el entrenamiento son suficientemente flexibles para adaptarse a otro lenguaje pianístico. La comparación entre MFT-FiLM y Transkun adquiere interés precisamente porque ambos modelos presentan un rendimiento similar en el dominio estándar. Si uno de ellos obtiene una mayor concordancia sobre PiJAMA, esto puede interpretarse como una señal de mejor adaptación relativa a un dominio externo al entrenamiento.

6.4 Experimentos

En primer lugar, se descargaron las grabaciones de audio asociadas a las anotaciones MIDI de PiJAMA. De las 2 777 piezas con referencia simbólica, se obtuvieron correctamente 2 626 archivos de audio, por lo que 151 interpretaciones tuvieron que descartarse al no disponer de la señal necesaria para realizar la inferencia. Antes de evaluar los modelos, se comprobó que esta pérdida no alteraba de forma significativa la distribución original de las particiones del conjunto. La Tabla 6.1 muestra la cobertura obtenida tras la descarga.

Tabla 6.1: Cobertura de audio en las particiones de PiJAMA tras la descarga de las grabaciones. Se muestra el número de piezas con audio disponible, el número de piezas descartadas por ausencia de audio y el porcentaje final disponible en cada partición.

Partición	Con audio	Sin audio	Disponible
train	2 099	128	94,3 %
validation	261	11	95,9 %
test	266	12	95,7 %

Como puede observarse en la Tabla 6.1, la proporción de piezas disponibles se mantiene muy próxima en las tres particiones, siempre por encima del 94 %. Por tanto, la pérdida de audios no introduce un sesgo relevante en la estructura del conjunto. En consecuencia, se respetaron las particiones originales de PiJAMA y se utilizó el subconjunto de *test* disponible, formado por 266 pares audio–MIDI.

Una vez construido el conjunto evaluable, ambos modelos se evaluaron siguiendo el mismo protocolo descrito en la Sección 3.9 para MAESTRO. Se emplearon las métricas estándar, manteniendo la misma tolerancia temporal de 0,05 segundos para la comparación de *onsets*.

Una primera ejecución del experimento produjo resultados anómalos. En particular, MFT-FiLM obtenía un F_1 de *frame* razonable, pero valores prácticamente nulos en las métricas basadas en *onset*. El caso más claro fue un F_1 de *frame* de 0,6797 frente a un F_1 de *note onset* de solo 0,0120. Esta combinación no era coherente desde el punto de vista musical: si el modelo detectaba de forma aceptable qué notas estaban activas, no resultaba plausible que fallara casi por completo en la localización de sus pulsaciones.

Para investigar esta anomalía se compararon manualmente predicciones y referencias en piezas individuales. El análisis mostró que las notas estimadas eran compatibles con las anotaciones MIDI, pero aparecían desplazadas temporalmente. Es decir, el problema no parecía deberse a un fallo de transcripción del modelo, sino a una desincronización sistemática entre el audio y la referencia simbólica. Esta hipótesis se confirmó al variar la tolerancia temporal de *onset* en una pieza concreta: con la tolerancia estándar de 0,05 segundos el resultado era casi nulo, mientras que al aumentar la tolerancia el F_1 crecía de forma abrupta, como se muestra en la Tabla 6.2.

Tabla 6.2: Efecto de la tolerancia temporal sobre el F_1 de *note onset* en una pieza de PiJAMA antes de corregir el alineamiento entre audio y MIDI.

Tolerancia de *onset*	**F_1 de *note onset***
0,05 s	0,0048
0,10 s	0,5958
0,20 s	0,9488
0,50 s	0,9488

Este comportamiento evidenciaba que las predicciones y la referencia coincidían musicalmente, pero no dentro de la ventana temporal estricta usada por la métrica. No obstante, aumentar la tolerancia no habría sido una solución adecuada, ya que habría modificado el protocolo respecto a MAESTRO. Por ello, se decidió mantener la tolerancia de 0,05 segundos y corregir previamente el alineamiento temporal entre audio y MIDI.

El desfase se estimó sobre el conjunto de *test* y resultó ser muy estable: el *offset* medio fue de aproximadamente 0,0982 segundos, con una mediana de 0,0983 segundos y una desviación estándar de 0,0028 segundos. Esta baja variabilidad permitió tratarlo como una desincronización sistemática del conjunto. Tras aplicar la corrección temporal, se reconstruyó la referencia alineada para las 266 piezas de *test* y se calcularon de nuevo las métricas manteniendo exactamente la misma tolerancia que en MAESTRO.

La Tabla 6.3 presenta los resultados finales de MFT-FiLM y Transkun sobre PiJAMA, junto con los resultados obtenidos previamente sobre MAESTRO.

Tabla 6.3: Resultados comparativos de MFT-FiLM y Transkun sobre MAESTRO y PiJAMA. Se muestran la precisión (*P*), el *recall* (*R*) y el F_1 para las principales métricas de transcripción automática de piano. Todas las métricas se expresan en porcentaje (%).

Modelo	Frame			Note			Note w/ Offset			Note w/ Off. & Vel.		
	P(%)	R(%)	F1(%)	P(%)	R(%)	F1(%)	P(%)	R(%)	F1(%)	P(%)	R(%)	F1(%)
	MAESTRO											
Transkun	94.25	95.21	94.69	99.68	96.48	98.03	93.58	90.62	92.06	93.03	90.10	91.52
MFT-FiLM	95.27	93.56	94.36	99.37	97.03	98.17	92.70	90.54	91.60	91.86	89.73	90.78
	PiJAMA											
Transkun	79.62	84.13	81.56	88.15	94.54	91.15	65.48	70.17	67.69	57.42	61.51	59.35
MFT-FiLM	82.67	83.29	82.44	89.84	94.01	91.73	65.44	68.54	66.89	57.68	60.37	58.94

En la Tabla 6.3 se puede observar una degradación clara al pasar de MAESTRO a PiJAMA. Aunque ambos conjuntos contienen piano solo, el cambio de dominio afecta de forma notable a la calidad de las transcripciones. En MAESTRO, Transkun y MFT-FiLM obtienen valores muy próximos y elevados en todas las métricas, lo que confirma que ambos modelos resuelven de forma competitiva la transcripción en el dominio clásico. En PiJAMA, en cambio, todos los indicadores disminuyen, reflejando que el repertorio *jazz* introduce dificultades que no están igualmente representadas en el dominio clásico. Estos resultados son coherentes con la tendencia que se observaba en la Figura 6.2.

La caída es especialmente relevante porque, en transcripción musical, diferencias de uno o varios puntos porcentuales no son necesariamente pequeñas desde el punto de vista perceptivo. Un MIDI puede conservar una parte importante de las notas y, aun así, resultar musicalmente pobre si omite ataques relevantes, desplaza duraciones, corta notas antes de tiempo o introduce eventos adicionales. De forma similar a como una palabra incorrecta puede alterar el sentido de una frase, una proporción aparentemente reducida de errores puede modificar de manera clara el fraseo, el ritmo y la coherencia musical de una transcripción.

La métrica de *note onset* es la que mejor resiste el cambio de dominio. Transkun pasa de un F_1 de 98,03 % en MAESTRO a 91,15 % en PiJAMA, mientras que MFT-FiLM pasa de 98,17 % a 91,73 %. Aunque estos valores siguen siendo altos en términos absolutos, la pérdida de casi siete puntos porcentuales es muy perceptible al escuchar o visualizar los MIDIs generados. Los modelos continúan detectando muchos ataques correctamente, pero el número de omisiones y errores aumenta lo suficiente como para que la transcripción pierda parte de la claridad que presentaba en repertorio clásico.

En la métrica de *frame* la degradación también es importante. Transkun desciende de 94,69 % a 81,56 %, y MFT-FiLM de 94,36 % a 82,44 %. Esto indica que los modelos no son completamente incapaces de decidir si una nota está presente o no, pero sí tienen más dificultades para mantener una representación temporal estable de las notas activas. En términos musicales, esto se traduce en transcripciones menos limpias, con más notas omitidas, activaciones espurias o continuidades mal representadas.
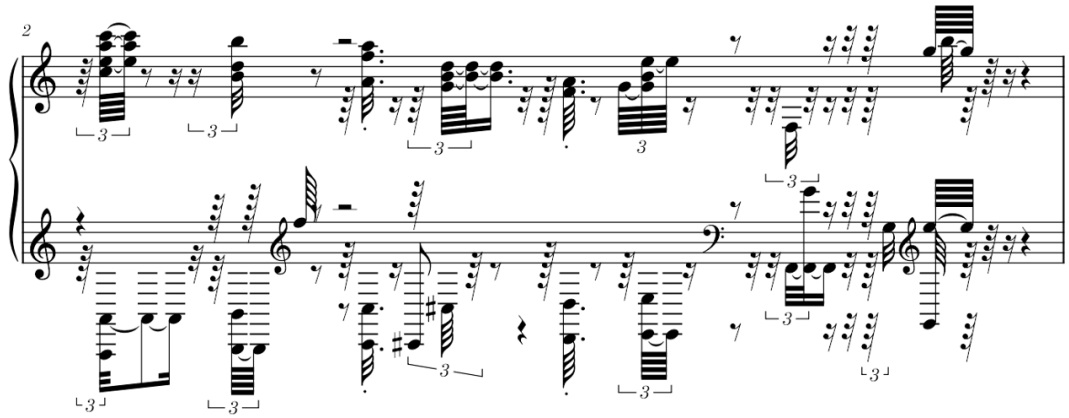
Las métricas que incorporan *offset* y velocidad muestran el deterioro más severo. En *Note w/ Offset*, ambos modelos caen desde valores cercanos al 92 % en MAESTRO hasta aproximadamente el 67 % en PiJAMA. En la métrica más restrictiva, *Note w/ Off. & Vel.*, Transkun obtiene 59,35 % y MFT-FiLM 58,94 %. Estos resultados confirman que el principal problema en el dominio *jazz* no es solo detectar qué notas existen, sino reconstruir correctamente su duración, su articulación y su intensidad. Desde el punto de vista auditivo, estos errores afectan de forma directa al ritmo percibido, al fraseo y a la naturalidad de la interpretación transcrita.

Por tanto, los resultados cuantitativos permiten extraer dos conclusiones. En primer lugar, ambos modelos generalizan peor al piano *jazz* que al repertorio clásico, y esta pér-
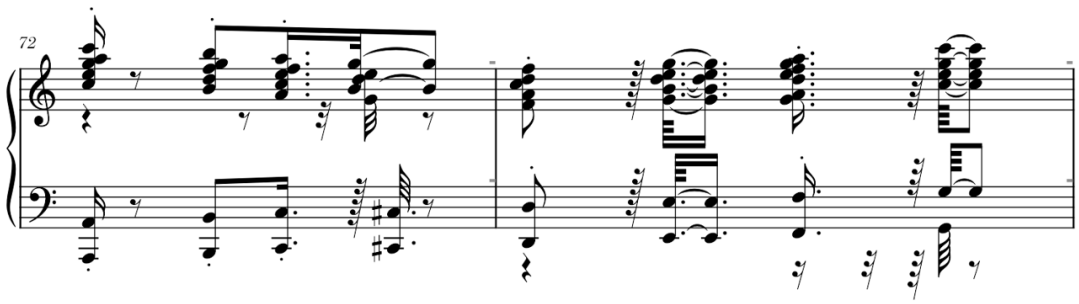
dida se aprecia de forma clara en los MIDIs generados. En segundo lugar, aunque las métricas no muestran una superioridad notable de un modelo sobre el otro, sí sugieren que las diferencias deben analizarse también cualitativamente.

Con el objetivo de comprender mejor los resultados de las métricas, cuya interpretación resulta menos concluyente que en el caso de MAESTRO, se propone analizar un fragmento de una pieza de *jazz* procesado por ambos modelos. De este modo, se busca obtener conclusiones más representativas sobre su funcionamiento y determinar si las ligeras variaciones en los resultados cuantitativos se traducen en diferencias perceptibles a nivel auditivo y visual que permitan discernir qué modelo ofrece un rendimiento superior.

Tras procesar el fragmento de audio (correspondiente a una interpretación original de un arreglo de *jazz* de *Fly Me to the Moon*, de Bart Howard) mediante los modelos evaluados, se obtiene un archivo MIDI cuya representación en un visor de partituras se ilustra en la Figura 6.4.



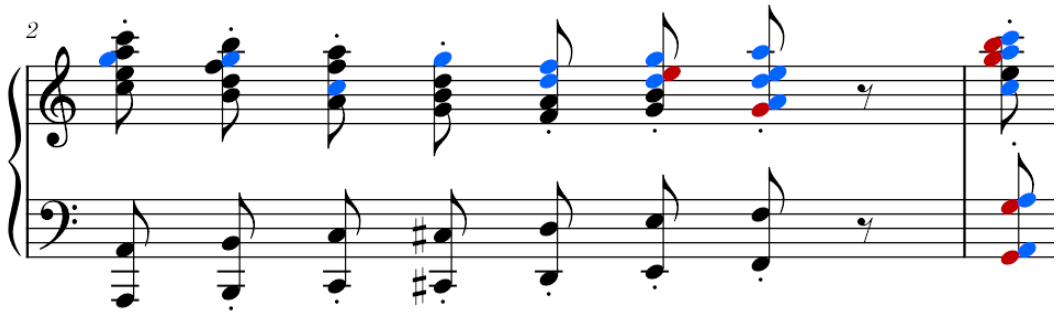
(a) Modelo MFT-FiLM
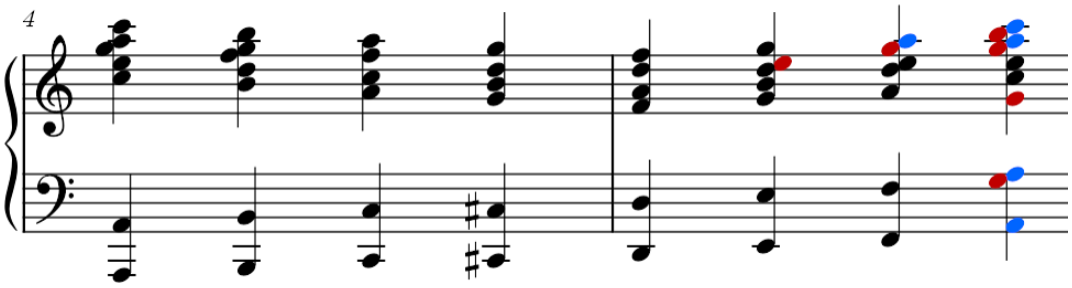


(b) Modelo Transkun

Figura 6.4: Salidas de MFT-FiLM y Transkun sobre fragmento de *jazz*. Se observa una representación MIDI del audio poco legible y difícil de interpretar, particularmente en el caso del modelo MFT-FiLM. Cabe recalcar que la salida de los modelos no es una partitura, sino un MIDI que debería sonar como el audio de entrada.

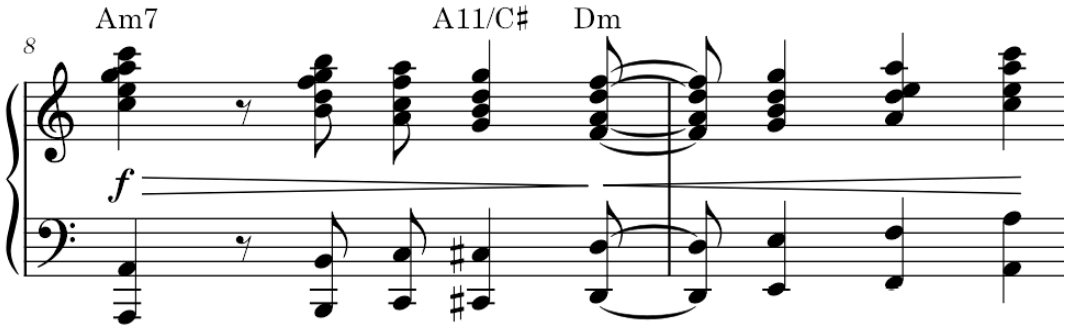
Como se puede observar en la Figura 6.4, la representación resulta bastante ilegible y de difícil interpretación, particularmente en el caso del modelo MFT-FiLM. Con el fin de obtener una visualización más comprensible, la salida de los modelos se procesa mediante la herramienta MIDI2ScoreTransformer, presentada y adaptada en la Sección 2.7. De este modo, se genera un archivo MIDI cuya transcripción se aproxima en mayor medida a una partitura real y legible, tal y como se muestra en la Figura 6.5.



(a) Modelo MFT-FiLM



(b) Modelo Transkun



(c) Referencia MIDI

Figura 6.5: Salidas de MFT-FiLM y Transkun procesadas por MIDI2ScoreTransformer. En las transcripciones se observan notas azules, que representan notas omitidas por los modelos pero que se encuentran en el MIDI original; y notas rojas que representan notas equivocadas, que no forman parte del MIDI original. Observamos que se trata de una secuencia de acordes con gran disonancia, muy característica del *jazz*. Respecto a la duración y velocidad de las notas, se puede ver que ninguno de los modelos acierta de forma consistente, aunque Transkun se acerca más a la representación original. En cuanto a las notas, los primeros acordes se transcriben bastante bien por ambos modelos: Transkun reproduce la referencia sin omisiones y MFT-FiLM omite unas pocas notas. A medida que avanza el fragmento, aumentan los errores y omisiones, ya que se van concentrando los armónicos y colas de audio de las notas pulsadas previamente. Al final del fragmento observamos que ambos modelos fallan y omiten bastantes notas. Este fragmento sugiere que Transkun se comporta mejor en situaciones características del *jazz*.

El análisis detallado de las transcripciones ilustradas en la Figura 6.5 revela diferencias significativas en el rendimiento de ambos modelos frente a la complejidad armónica y las disonancias características del *jazz*. En primer lugar, respecto a la dimensión temporal, se observa que ninguno de los sistemas logra transcribir el ritmo con exactitud. Sin embargo, las duraciones predichas por el modelo Transkun se aproximan de manera considerablemente mayor a la interpretación original, superando a MFT-FiLM en este aspecto.

En cuanto a la precisión tonal, la evolución del fragmento evidencia cómo la acumulación de armónicos y las colas de audio de las notas pulsadas previamente dificultan la tarea de transcripción a medida que avanza el tiempo. Aunque los dos modelos presentan dificultades en la sección final, los datos demuestran una clara diferencia entre ambos. Específicamente, ambos sistemas registran un total de 6 notas incorrectas (falsos positivos, notas de color rojo), las cuales se concentran de forma exclusiva en los tres últimos acordes de la secuencia. No obstante, la discrepancia principal reside en el volumen de notas omitidas, es decir, falsos negativos representados en color azul. El modelo MFT-FiLM omite un total de 17 notas a lo largo del fragmento evaluado, fallando en la detección de al menos un evento en cada uno de los acordes presentes. Por el contrario, Transkun transcribe los acordes iniciales a la perfección y reduce sus omisiones a un total de 5 notas, las cuales se localizan únicamente en los dos últimos acordes, coincidiendo con el punto de mayor saturación acústica.

Este comportamiento resulta coherente con los valores de *recall* observados en la Tabla 6.3. En PiJAMA, Transkun obtiene un *recall* superior al de MFT-FiLM en prácticamente todas las métricas evaluadas, lo que indica que el modelo tiende a detectar una mayor proporción de los eventos presentes en la señal de referencia. Esta tendencia se traduce directamente en el análisis perceptivo: las 5 omisiones de Transkun frente a las 17 de MFT-FiLM reflejan, a escala de fragmento, la misma diferencia sistemática que las métricas cuantitativas apuntan a nivel de corpus.

Cabe destacar que, en el contexto del *jazz*, esta consideración adquiere una dimensión especialmente relevante. A diferencia del repertorio clásico, donde la priorización de la precisión sobre la exhaustividad es adecuada e incluso conveniente, el *jazz* hace un uso extensivo de la disonancia como recurso expresivo fundamental. Los acordes propios de este género incorporan con frecuencia extensiones armónicas de novena, undécima y decimotercera, cuya presencia simultánea define en gran medida la identidad tímbrica y estilística del fragmento. En consecuencia, la omisión de alguno de estos componentes puede resultar más perjudicial para la musicalidad del resultado que la inclusión de notas adicionales, dado que la supresión de tensiones armónicas características altera de forma sustancial el carácter del acorde original. Desde esta perspectiva, un modelo con un *recall* elevado resulta particularmente deseable en el dominio del *jazz*, puesto que garantizar la detección completa de los eventos presentes en la señal es condición necesaria para preservar la riqueza armónica inherente a este estilo musical.

En definitiva, este análisis perceptivo del fragmento corrobora que, si bien la alta densidad armónica del final de la secuencia supone un desafío complejo para ambas arquitecturas, el modelo Transkun ofrece una transcripción notablemente más robusta, precisa y fiel a la referencia MIDI.

6.5 Conclusiones

Este capítulo ha abordado la evaluación de MFT-FiLM y Transkun fuera del dominio clásico, utilizando PiJAMA como corpus de referencia para el piano *jazz*. Los resultados obtenidos permiten extraer conclusiones tanto a nivel cuantitativo como cualitativo.

En primer lugar, el análisis estadístico sobre MAESTRO ha evidenciado que el rendimiento de los sistemas de AMT está condicionado por factores musicales inherentes a las obras: la densidad de notas por segundo, la polifonía máxima y el compositor presentan un efecto significativo sobre el F_1 de *note onset*. En particular, se observa una tendencia decreciente conforme el repertorio se aproxima a lenguajes pianísticos más densos y modernos, coherente con la evolución estilística de la escritura para piano.

La evaluación sobre PiJAMA sugiere que dicha tendencia se mantiene al considerar un dominio jazzístico. La comparación con los resultados en MAESTRO muestra una degradación clara en todas las métricas al cambiar de dominio, siendo especialmente acusada en las métricas que incorporan *offset* y velocidad. Este deterioro refleja que el *jazz* introduce dificultades que van más allá de la detección de ataques, afectando también a la estimación de duraciones, articulación e intensidad.

En cuanto a la comparación entre modelos, los resultados cuantitativos sobre PiJAMA no permiten establecer una superioridad clara de ninguno de los dos sistemas. Sin embargo, el análisis cualitativo del fragmento de *Fly Me to the Moon* procesado mediante MIDI2ScoreTransformer revela diferencias relevantes: Transkun omite significativamente menos notas, concentra sus errores en las regiones de mayor saturación acústica y reproduce con mayor fidelidad las duraciones originales. En consecuencia, aunque la brecha cuantitativa es reducida, el análisis perceptivo posiciona a Transkun como mejor candidato en términos de generalización al dominio jazzístico.

Cabe señalar, por último, que los resultados obtenidos sobre PiJAMA no son directamente comparables a los de MAESTRO en términos de rendimiento absoluto, dado que las anotaciones de PiJAMA son pseudo-etiquetas generadas automáticamente. No obstante, el conjunto resulta de gran utilidad para comparar el comportamiento relativo de ambos modelos, sirve como precedente para una comparación práctica y permite analizar la tendencia de generalización observada a lo largo del capítulo.

CAPÍTULO 7
Conclusiones

????? ????????????? ????????????? ????????????? ????????????? ?????????????

Bibliografía

[1] Valentin Emiya, Nancy Bertin, Bertrand David y Roland Badeau. MAPS - A piano database for multipitch estimation and automatic transcription of music. *INRIA Technical Report*, 2010.

[2] Francesco Foscarin, Katharina Hoedt, Gerhard Widmer y Maarten Grachten. Towards musically informed evaluation of piano transcription models. *arXiv preprint arXiv:2406.08454*, 2024.

[3] Google Magenta. The MAESTRO *dataset* v3.0.0. 2020.

[4] Yamaha Corporation. *Disklavier pianos*. Disponible en: https://usa.yamaha.com/products/musical_instruments/pianos/disklavier/index.html Consultado en: mayo de 2026.

[5] Curtis Hawthorne, Erich Elsen, Jialin Song, Adam Roberts, Ian Simon, Colin Raffel, Jesse Engel, Sageev Oore y Douglas Eck. Onsets and frames: Dual-objective piano transcription. *Proceedings of the International Society for Music Information Retrieval (ISMIR)*, 2018.

[6] Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman y Douglas Eck. Enabling factorized piano music modeling and generation with the MAESTRO *dataset*. *International Conference on Learning Representations (ICLR)*, 2019.

[7] Colin Raffel, Brian McFee, Eric J. Humphrey, Justin Salamon, Oriol Nieto, Dawen Liang y Daniel P. W. Ellis. mir_eval: A transparent implementation of common MIR metrics. *Proceedings of the 15th International Conference on Music Information Retrieval*, 2014.

[8] Sebastian Ruder. An overview of multi-task learning in deep neural networks. *arXiv preprint arXiv:1706.05098*, 2017.

[9] Kevin Patrick Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023.

[10] Qiuqiang Kong, Bochen Li, Xuchen Song, Yuan Wan y Yuxuan Wang. High-resolution piano transcription with pedals by regressing onset and offset times. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2021.

[11] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin y Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[12] Qi Wang, Mingkuan Liu, Changchun Bao y Maoshen Jia. Harmonic-aware frequency and time attention for automatic piano transcription. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2024.

[13] Qi Wang, Mingkuan Liu y Maoshen Jia. Multitask frequency-time feature-wise linear modulation for piano transcription with pedal. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2025.

[14] C. Hawthorne, I. Simon, R. Swavely, E. Manilow y J. Engel. Sequence-to-sequence piano transcription with transformers. *Proceedings of the International Society for Music Information Retrieval (ISMIR)*, 2021.

[15] K. Toyama, T. Akama, Y. Ikemiya, Y. Takida, W.-H. Liao y Y. Mitsufuji. Automatic piano transcription with hierarchical frequency-time transformer. *Proceedings of the International Society for Music Information Retrieval (ISMIR)*, 2023.

[16] W. Wei, P. Li, Y. Yu y W. Li. HPPNet: Modeling the harmonic structure and pitch invariance in piano transcription. *Proceedings of the International Society for Music Information Retrieval (ISMIR)*, 2022.

[17] Y. Yan, F. Cwitkowitz y Z. Duan. Skipping the frame-level: Event-based piano transcription with neural semi-CRFs. *Proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2021.

[18] Y. Yan y Z. Duan. Scoring time intervals using non-hierarchical transformer for automatic piano transcription. *Proceedings of the International Society for Music Information Retrieval (ISMIR)*, 2024.

[19] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit y Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations (ICLR)*, 2021.

[20] Taegyun Kwon, Dasaem Jeong y Juhan Nam. Polyphonic piano transcription using autoregressive multi-state note model. *Proceedings of the International Society for Music Information Retrieval (ISMIR)*, 2020.

[21] T. Bachlechner, B. P. Majumder, H. Mao, G. Cottrell y J. McAuley. ReZero is all you need: Fast convergence at large depth. *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence (UAI)*, 2021.

[22] J. Ho, N. Kalchbrenner, D. Weissenborn y T. Salimans. Axial attention in multidimensional transformers. *arXiv preprint arXiv:1912.12180*, 2019.

[23] Y. Li, S. Si, G. Li, C.-J. Hsieh y S. Bengio. Learnable Fourier features for multi-dimensional spatial positional encoding. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[24] Y. Li, H. Mao, R. Girshick y K. He. Exploring plain vision transformer backbones for object detection. *European Conference on Computer Vision (ECCV)*, 2022.

[25] N.-C. Ristea, R. T. Ionescu y F. S. Khan. Septr: Separable transformer for audio spectrogram processing. *Proceedings of INTERSPEECH*, 2022.

[26] B. Zhang y R. Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser e Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[28] G. Loaiza-Ganem y J. P. Cunningham. The continuous Bernoulli: Fixing a pervasive error in variational autoencoders. *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[29] Y. Fang, B. Liao, X. Wang, J. Fang, J. Qi, R. Wu, J. Niu y W. Liu. You only look at one sequence: Rethinking transformer in vision through object detection. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[30] Drew Edwards, Simon Dixon y Emmanouil Benetos. PiJAMA: Piano *jazz* with automatic MIDI annotations. *Transactions of the International Society for Music Information Retrieval*, 2023.

[31] Huw Cheston *et al.* *jazz* trio database: Automated annotation of *jazz* piano trio recordings processed using audio source separation. *Transactions of the International Society for Music Information Retrieval*, 2024.

[32] Francesco Foscarin, Andrew McLeod, Philippe Rigaux, Florent Jacquemard y Masahiko Sakai. ASAP: A dataset of aligned scores and performances for piano transcription. *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2020.

[33] Masataka Goto, Hiroki Hashiguchi, Takuichi Nishimura y Ryuichi Oka. RWC music database: Popular, classical, and *jazz* music databases. *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2002.

[34] Martin Pfleiderer, Klaus Frieler, Jakob Abeßer, Wolf-Georg Zaddach y Benjamin Burkhart. *Inside the jazzomat: New perspectives for jazz research*. Schott Campus, 2017.

[35] Tim Beyer y Angela Dai. End-to-end piano performance-MIDI to score conversion with transformers. *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2024.

[36] Kevin Patrick Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.

[37] E. Benetos, S. Dixon, Z. Duan y S. Ewert. Automatic music transcription: An overview. *IEEE Signal Processing Magazine*, vol. 36, pp. 20–30, 2019.

[38] S. Davis y P. Mermelstein. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, pp. 357–366, 1980.

[39] M. Good. MusicXML: An Internet-friendly format for sheet music. *Proceedings of XML*, 2001.

[40] MIDI Manufacturers Association. *The Complete MIDI 1.0 Detailed Specification*. MMA, 1996.

[41] A. V. Oppenheim y R. W. Schafer. *Discrete-Time Signal Processing*. Prentice Hall, tercera edición, 2010.

APÉNDICE A
Configuración del experimento MFT-FiLM

A.1 Entorno de ejecución y dependencias

El entrenamiento del modelo MFT-FiLM se llevó a cabo en el clúster de cómputo de alto rendimiento **tensor**, perteneciente al Departamento de Sistemas Informáticos y Computación (DSIC) de la Universidad Politécnica de Valencia. Este sistema está formado por un único nodo equipado con 503 **GB de memoria RAM** y cuatro unidades de procesamiento gráfico **NVIDIA L40S** con 48 GB de memoria GDDR6 ECC cada una. Las GPUs L40S pertenecen a la familia *Ada Lovelace*, orientadas a cargas de trabajo de inteligencia artificial en entornos de servidor. El hardware empleado en el artículo original consistía en dos tarjetas NVIDIA GeForce RTX 4090 de 24 GB de VRAM cada una [13]; la L40S cuenta individualmente con 48 GB, lo que permitió alojar el modelo con el tamaño de lote requerido sin restricciones de memoria.

El sistema de gestión de trabajos del clúster es **SLURM**, que organiza la ejecución mediante dos colas principales: `docencia`, para trabajos de duración inferior a 24 horas, y `long`, para trabajos de hasta 72 horas. Dada la duración estimada de entrenamiento reportada por los autores [13], fue necesario emplear la cola `long`.

El entorno software se configuró sobre Python 3.12 con `PyTorch` y `CUDA` compatibles con las GPUs L40S, y se instalaron las dependencias específicas del repositorio original (incluyendo `librosa`, `pretty_midi`, `mireval` y `joblib`) en un entorno virtual aislado para garantizar la reproducibilidad del experimento.

Durante la preparación del entorno surgieron dos incidencias relevantes. La primera fue un problema de **obsolescencia de dependencias**: para reproducir exactamente los espectrogramas CQT del trabajo original se requirió la versión **0.9.2** de `librosa`, pero esta versión dependía del módulo `pkg_resources`, eliminado en las versiones recientes de `setuptools` asociadas a Python 3.12. El problema se solventó manteniendo `librosa` 0.9.2 y degradando las herramientas de construcción del entorno a la última versión compatible, lo que permitió generar CQT idénticos a los descritos en [13] sin modificar el código original.

La segunda incidencia fue una **incompatibilidad de la API de PyTorch**. Los autores definieron la clase `CacheDataset` heredando de `TensorDataset` pero invocando al constructor padre sin tensores, algo que las versiones modernas de PyTorch (serie 2.x) rechazan con una excepción de desajuste dimensional. La solución consistió en sustituir la herencia de `TensorDataset` por la clase base estándar `torch.utils.data.Dataset`, manteniendo inalterada la lógica interna del conjunto de datos. Adicionalmente, fue necesario

regenerar los archivos de caché `.pth` tras algunos intentos fallidos de entrenamiento que habían corrompido parcialmente dichos ficheros.

A.2 Rendimiento del entrenamiento en el clúster

Durante el entrenamiento se observó un patrón de utilización de la GPU con oscilaciones periódicas de uso entre el 0 % y el 100 %, en lugar de un valor cercano al máximo continuado. Este fenómeno, conocido como *GPU starvation* o cuello de botella de entrada/salida (*I/O bottleneck*), se produce cuando la velocidad de cómputo de la GPU es significativamente superior a la velocidad con que la CPU puede preparar y transferir nuevos lotes de datos a la VRAM.

En la práctica, una vez que la GPU terminaba de procesar un lote de 10 espectrogramas (operación que, dada la compacidad del modelo, se completaba en pocos segundos), el proceso quedaba bloqueado a la espera de que la CPU leyera el siguiente lote desde los archivos `.pth` en disco y lo transfiriera a la VRAM a través del bus PCIe. Durante ese intervalo de espera la utilización de la GPU caía al 0 %. En la configuración empleada, este fenómeno fue particularmente pronunciado debido a que `tensor` solo dispone de un nodo compartido por todos los usuarios, lo que impide reservar de forma dedicada todos los recursos de entrada/salida disponibles para un único `DataLoader`.

Como consecuencia directa de estas limitaciones de entrada/salida, la duración efectiva de cada época de entrenamiento fue de aproximadamente **83 minutos**, frente a los 43,5 minutos reportados por los autores originales con dos GPUs RTX 4090. En el *setup* original, la paralelización entre dos GPUs permitía solapar parcialmente la fase de transferencia de datos con el cómputo sobre la segunda tarjeta, mitigando significativamente este efecto y casi duplicando el rendimiento efectivo.

El clúster `tensor` impone una duración máxima de **72 horas** para los trabajos sometidos a la cola `long`. Dado que cada época requirió aproximadamente 83 minutos, el trabajo de entrenamiento fue interrumpido por SLURM al alcanzar dicho límite temporal, habiendo completado únicamente **30 de las 60 épocas planificadas**. A pesar de no haber completado el ciclo de entrenamiento descrito en el artículo original, el mecanismo de validación dinámica y guardado automático de *checkpoints* preservó el mejor modelo alcanzado durante las 30 épocas ejecutadas, que es el que se emplea en el análisis de resultados.

A.3 Parámetros de configuración del experimento

Una vez resueltos los problemas de compatibilidad, se ajustaron los hiperparámetros en el archivo central de configuración `constant_macro.py` para replicar las condiciones del artículo original. La Tabla A.1 resume los parámetros empleados [13].

Tabla A.1: Parámetros de configuración establecidos en `constant_macro.py` para el entrenamiento de MFT-FiLM.

Parámetro	Valor	Descripción
`EPOCH_RANGE`	60	Número total de épocas
`BATCH_SIZE`	10	Muestras por lote
`LEARNING_RATE`	0,0005	Tasa de aprendizaje
`DATASET_NAME`	`maestro-v3.0.0`	Versión del *dataset*
`DATASET_PATH`	ruta absoluta	Directorio raíz de los datos
`MODEL_PATH`	ruta absoluta	Directorio de salida del modelo
`MODEL_NAME`	`MTFiLM`	Identificador del modelo
`SCHEDULE_IDX`	`2x`	Objetivo: transcripción de notas

A.4 Herramientas de preprocesamiento de audio y MIDI

El preprocesamiento del conjunto MAESTRO se apoya en dos librerías ampliamente utilizadas en la comunidad de *Music Information Retrieval*: `librosa` para el tratamiento del audio y `pretty_midi` para el manejo de archivos MIDI.

La librería `librosa` es un paquete de Python orientado al análisis de señales de audio musicales. Entre otras funcionalidades, proporciona rutinas para el cálculo de representaciones tiempo–frecuencia como la Transformada de Fourier de tiempo corto (STFT) y la Transformada Q Constante (CQT), así como herramientas para el remuestreo, el filtrado y la extracción de descriptores espectrales. En este trabajo, `librosa` se utiliza para convertir cada grabación de MAESTRO en su correspondiente espectrograma CQT con los parámetros descritos en la Sección 4.2, que constituye la entrada del modelo MFT-FiLM.

Por otro lado, `pretty_midi` es una librería de Python diseñada para facilitar la lectura, escritura y manipulación estructurada de archivos MIDI. La librería ofrece una representación de alto nivel de las notas y de los eventos de control (por ejemplo, los pedales de *sustain* y *sostenuto*), lo que permite generar de forma directa los *piano rolls* de activación de nota a partir de una partitura MIDI. En el contexto de este proyecto, `pretty_midi` se utiliza para extraer las notas, extender su duración en función de los pedales y construir los mapas binarios de *ground truth* que se alinean temporalmente con los espectrogramas CQT preprocesados mediante `librosa`.

En conjunto, la combinación de `librosa` y `pretty_midi` permite establecer un flujo de preprocesamiento consistente entre el dominio del audio y el del MIDI, garantizando que las representaciones de entrada y las etiquetas de salida estén correctamente sincronizadas y sean comparables con las del trabajo original.

APÉNDICE B
Configuración del experimento Transkun

B.1 Entorno de ejecución y dependencias

El entrenamiento del modelo Transkun se llevó a cabo en el clúster de cómputo de alto rendimiento **tensor**, perteneciente al Departamento de Sistemas Informáticos y Computación (DSIC) de la Universidad Politécnica de Valencia. Dado que las características generales del clúster ya se describen en el Anexo A, en este anexo se recogen únicamente los aspectos específicos necesarios para reproducir el experimento con Transkun.

El entrenamiento se ejecutó sobre una única GPU **NVIDIA L40S** de 48 GB mediante el gestor de colas **SLURM**, empleando la cola `long`. El artículo original de Transkun emplea dos GPUs NVIDIA RTX 4090; en este trabajo se utilizó una sola GPU L40S, lo que permitió alojar el modelo con el tamaño de lote requerido, aunque a costa de un mayor tiempo de entrenamiento efectivo.

El entorno software se configuró en un entorno virtual de Python con `PyTorch`, `TorchAudio`, `moduleconf`, `adabelief-pytorch`, `mir_eval`, `tqdm` y las dependencias propias del repositorio original. Para evitar inconsistencias entre el paquete instalado mediante `pip` y el código fuente del repositorio, la ejecución se realizó siempre desde la copia local de Transkun.

Durante la preparación del entorno surgieron dos incidencias relevantes. En primer lugar, se detectó una incompatibilidad entre las versiones de CUDA con las que habían sido compilados `PyTorch` y `TorchAudio`. El problema se resolvió reinstalando `TorchAudio` desde el índice oficial de PyTorch correspondiente a la misma versión CUDA que `PyTorch`. En segundo lugar, fue necesario adaptar una llamada al `DataLoader` de PyTorch, sustituyendo el argumento obsoleto o incorrecto `dropout_last` por `drop_last`, compatible con la API actual.

B.2 Preparación del conjunto MAESTRO

El experimento se realizó sobre **MAESTRO v3.0.0**. Antes del entrenamiento se generaron los archivos de metadatos requeridos por Transkun: `train.pickle`, `val.pickle` y `test.pickle`. Estos ficheros contienen la información necesaria para indexar las piezas musicales y sus correspondientes anotaciones MIDI.

Durante la carga del conjunto de entrenamiento, Transkun registró un total de **962 piezas** en el subconjunto de entrenamiento, con una duración acumulada de aproxima-

damente **573277 segundos**, equivalentes a unas **159 horas** de audio. El subconjunto de validación contenía **137 piezas**, con una duración aproximada de **69946 segundos**. Estos valores corresponden al número de piezas indexadas, no al número de segmentos temporales utilizados internamente durante el entrenamiento.

Como parte de la preparación de los datos, se verificó que los audios estuvieran disponibles en formato `wav` y a la frecuencia de muestreo esperada por la configuración del modelo, **44100 Hz**. Además, se homogeneizó la lectura de audio para evitar la mezcla de señales mono y estéreo dentro de un mismo lote. Para ello, las señales estéreo se convirtieron a mono durante la carga, promediando ambos canales. Esta operación garantiza que todos los fragmentos procesados por el `DataLoader` tengan una forma compatible.

B.3 Configuración del modelo y del entrenamiento

La arquitectura de Transkun se definió mediante el fichero `conf.json`, generado a partir de la clase `transkun.ModelTransformer`. Este fichero contiene únicamente la configuración estructural del modelo; los hiperparámetros de entrenamiento se especifican como argumentos del módulo `transkun.train`.

La configuración inicial generada por defecto era distinta a la empleada por los autores. Para aproximar el experimento a la configuración del artículo original se ajustaron los parámetros como se presentan en la Tabla B.1.

Tabla B.1: Parámetros principales de configuración empleados en el experimento Transkun.

Parámetro	Valor	Descripción
`fs`	44100	Frecuencia de muestreo
`n_mels`	229	Número de bandas mel
`segmentSizeInSecond`	16	Duración del segmento de entrada
`segmentHopSizeInSecond`	8	Salto entre segmentos
`windowSize`	4096	Tamaño de ventana espectral
`hopSize`	1024	Salto entre ventanas
`baseSize`	64	Tamaño base de representación
`nHead`	8	Cabezas de atención
`fourierSize`	256	Tamaño de características Fourier
`nLayers`	6	Capas del transformador
`enabledAttn`	`F,T`	Atención en frecuencia y tiempo
`-batchSize`	12	Segmentos por lote
`-nIter`	500000	Iteraciones máximas planificadas
`-max_lr`	$4 \cdot 10^{-4}$	Tasa máxima de aprendizaje
`-weight_decay`	10^{-2}	Decaimiento de pesos
`-gradClippingQuantile`	0,8	Recorte adaptativo del gradiente

El trabajo de entrenamiento se lanzó mediante un script de `SLURM` que reservaba una GPU completa, necesaria para alojar el modelo. El fichero de salida del modelo, `model.pt`, se utilizó también como punto de guardado para permitir la reanudación del entrenamiento en caso de interrupción por el límite temporal de la cola.

Aunque la configuración establecía un máximo de 500000 iteraciones, el entrenamiento se detuvo cuando los *checkpoints* evaluados alcanzaron métricas competitivas para los objetivos del trabajo. La evolución de las métricas sugería que aún podía existir cierto margen de mejora con más tiempo de entrenamiento; sin embargo, se optó por detener la ejecución para evitar un uso prolongado de una GPU completa en un clúster de uso com-

partido. El modelo empleado en los resultados corresponde, por tanto, al mejor *checkpoint* evaluado antes de dicha parada.